JSC-61777-models/interactions/radiation_pressure

JSC Engineering Orbital Dynamics Radiation Pressure Model

Simulation and Graphics Branch (ER7)
Software, Robotics, and Simulation Division
Engineering Directorate

Package Release JEOD v5.4

Document Revision 1.1.1
January 2026



National Aeronautics and Space Administration
Lyndon B. Johnson Space Center
Houston, Texas

JSC Engineering Orbital Dynamics
Radiation Pressure Model

Document Revision 1.1.1
January 2026

Gary Turner

Simulation and Graphics Branch (ER7)
Software, Robotics, and Simulation Division
Engineering Directorate

National Aeronautics and Space Administration
Lyndon B. Johnson Space Center
Houston, Texas

Abstract

The magnitude of the radiation pressure force on a spacecraft is generally small; it has negligible effect on short-duration operations, and is typically insignificant – compared to aerodynamic drag and gravitational variabilities – in Low Earth Orbit. Nevertheless, the cumulative effect over long-duration operations can produce a significant perturbation to the vehicle trajectory, and to orbital characteristics. The Radiation Pressure Model allows for the incorporation of radiative forces into to JEOD v5.4 package for state propagation.

Contents

1 Introduction **1**

1.1 Purpose and Objectives of the Radiation Pressure Model 1

1.2 Context within JEOD 2

1.3 Documentation History 2

1.4 Documentation Organization 2

2 Product Requirements **4**

2.1 Modeling Requirements 4

2.2 Functional Requirements 6

2.3 Requirements Traceability 8

3 Product Specification **9**

3.1 Conceptual Design 9

3.1.1 Photon Interaction 9

3.1.2 Radiation Field 10

3.1.3 Surface Model 11

3.1.4 Temperature 11

3.2 Mathematical Formulations 12

3.2.1 Reduction of Flux by Intervening Body 12

3.2.2 Conical Eclipse Calculator 18

3.2.3 Response to Flux 24

3.2.4 Response of Temperature and Thermal Emission to Incident Flux 29

3.2.5 Coefficient of Reflection 32

3.3 Detailed Design 35

3.3.1 Process Architecture 35

3.3.2 Functional Design . . . 52
3.4 Inventory . . . 61

4 User Guide **71**
4.1 Analysis . . . 71
4.1.1 Identifying the Radiation Pressure Model . . . 71
4.1.2 Editing the Radiation Pressure Model . . . 72
4.1.3 Output Data . . . 76
4.2 Integration . . . 77
4.2.1 Generating the S_define . . . 77
4.2.2 Generating the Input File . . . 79
4.3 Extension . . . 83

5 Verification and Validation **84**
5.1 Verification . . . 84
5.1.1 Top-level Requirement . . . 84
5.1.2 Inspection of Modeling Requirements . . . 84
5.1.3 Verification of Surface Representations . . . 85
5.1.4 Verification of Force Calculations . . . 85
5.1.5 Verification of the Temperature Integration and Thermal Emission Procedures 90
5.1.6 Verification of the Flux Calculations . . . 97
5.2 Validation . . . 103
5.2.1 Validation of Flux-Vehicle Interaction-Model Principles . . . 103
5.2.2 Force Comparisons . . . 109
5.2.3 Validation of the Plate Model Assumptions . . . 118
5.2.4 Investigation of the Effect of the Isothermal Approximation on Forces and Temperature-Time Variation . . . 132
5.2.5 Validation of Orbital Simulations . . . 133
5.3 Metrics . . . 139
5.3.1 Code Metrics . . . 139

Chapter 1

Introduction

1.1 Purpose and Objectives of the Radiation Pressure Model

The Radiation Pressure Model allows for the incorporation of radiative forces into the JEOD v5.4 package for state propagation.

The magnitude of the radiation pressure force on a spacecraft is generally small, and has negligible effect on short-duration operations. Nevertheless, the cumulative effect over long-duration operations can produce significant perturbations to the orbital characteristics. The significance of those perturbations, compared against the perturbations generated by other environmental factors, depends on the specific orbital characteristics. While the effects of atmospheric drag and geo-potential terms generally decrease with altitude, the radiation pressure resulting from incident solar radiation is largely altitude independent, and so becomes increasingly important at higher altitudes. For Earth orbits with semimajor axes around 7000 km (altitude 600 km), radiation pressure effects become comparable to atmospheric drag effects [3]. They dominate at higher orbits, becoming comparable to the higher order ($l \geq 2$) geopotential effects at orbit altitudes comparable to geosynchronous orbits.

Smaller than the direct solar radiation effects, there is also indirect radiation (or albedo pressure) to consider – that is, solar radiation reflected off a third body, such as Earth. There is also the radiation emitted by the vehicle itself (thermal emission). For highly reflective surfaces in direct sunlight, this effect is typically two to three orders of magnitude smaller than the effect of direct solar radiation. Nevertheless, it becomes significant in several situations:

- Surfaces that absorb almost all of the incident radiation have emission forces comparable to the forces resulting from direct solar radiation pressure.
- Unlit portions of the vehicle (such as on the dark side of the vehicle, or in the shadow of Earth) obviously have no direct solar pressure, but will continue to experience forces resulting from thermal emission.
- Deep-space simulations that involve a significant, localized, and well-insulated heat source, such as a power unit have a particularly strong, localized thermal emission profile, and a solar radiation pressure that is highly dependent upon the distance from Sun.

The Radiation Pressure Model in JEOD v5.4 calculates the effect of the direct solar radiation pressure, and is structured to allow the easy inclusion of albedo pressure as a user-specified extension, or in a future release. With the incorporation of the Thermal Rider [6] model, the temperature of any surface can be accurately simulated; with this temperature evolution profile, the Radiation Pressure Model can further model the dynamic effects of radiation emitted from the vehicle.

The surface of the vehicle can be modeled in two ways. The default setting is that of a perfectly symmetric, isothermal sphere. Alternatively, the Surface Model [5] can be used; this provides the user with the capability of defining any surface geometry / topology using any combination of materials.

The full Surface Model implementation requires the user to define – for each material used in the surface – the albedo and the fraction of reflected light that is reflected in a diffuse pattern (as opposed to a specular pattern). The same can also be specified for the default surface (which uses only one material over the entire surface). The default surface also offers the alternative option of specifying the Coefficient of Reflection instead of the albedo and diffuse parameter. This option IS NOT AVAILABLE for the full Surface Model, because the Coefficient of Reflection is geometry-specific and cannot be easily adapted to the total flexibility in geometry provided by the Surface Model.

1.2 Context within JEOD

The following document is parent to this document:

- *JSC Engineering Orbital Dynamics* [2]

The Radiation Pressure Model forms a component of the interactions suite of models within JEOD v5.4. It is located at models/interactions/RadiationPressure. It relies on functionality provided as part of the *Thermal-Rider* model [6] and the Surface Model [5].

1.3 Documentation History

Author	Date	Revision	Description
Gary Turner	November, 2009	1.0	JEOD 2.0.0 release
Gary Turner	April, 2011	1.1	Description of refactoring of model for JEOD 2.2-Alpha2.
Gary Turner	January, 2012	1.1.1	Change from use of picins to wrapfig.

1.4 Documentation Organization

This document is formatted in accordance with the NASA Software Engineering Requirements Standard [4] and is organized into the following chapters:

Chapter 1: Introduction - This introduction contains four sections: objective and purpose, context within JEOD, document history, and document organization.

Chapter 2: Product Requirements - Describes requirements for the Radiation Pressure Model.

Chapter 3: Product Specification - Describes the underlying theory, architecture, and design of the Radiation Pressure Model in detail. It will be organized in four sections: Conceptual Design, Mathematical Formulations, Detailed Design, and Version Inventory.

Chapter 4: User Guide - Describes how to use the Radiation Pressure Model in a Trick simulation. It is broken into three sections to represent the JEOD defined user types: Analysts or users of simulations (Analysis), Integrators or developers of simulations (Integration), and Model Extenders (Extension).

Chapter 5: Verification and Validation - Contains Radiation Pressure Model verification and validation procedures and results.

Chapter 2

Product Requirements

2.1 Modeling Requirements

Requirement RadiationPressure_1: Top-level requirement

Requirement:
This model shall meet the JEOD project requirements specified in the JEOD v5.4 top-level document [2].

Rationale:
This model shall, at a minimum, meet all external and internal requirements applied to the JEOD v5.4 release.

Verification:
Inspection RadiationPressure_1 on page 84

Requirement RadiationPressure_2: State Encapsulation

Requirement:
The Radiation Pressure Model shall encapsulate the positional state of the vehicle and the data necessary to calculate the orientation and position of the vehicle with respect to the sun, and planetary bodies.

Rationale:
The extent to which radiation pressure acts on the vehicle depends on the radiation flux and the orientation of the vehicle with respect to the radiation flux. The magnitude of the radiation flux is a function of distance from Sun, and potential shadowing effects of intervening bodies. The direction of the force resulting form the interaction depends on the relative orientation of the surfaces of the vehicle to the flux vector.

Verification:
Inspection RadiationPressure_2 on page 84

Requirement RadiationPressure_3: Surface Representation

Requirement:

The Radiation Pressure Model shall offer the capability to model the vehicle surface as a simple approximation, and as a high-fidelity geometric structure comprising multiple materials.

Rationale:

A driving design consideration across the development of JEOD v5.4 is the ability to do simple tasks simply. The inclusion of a radiation pressure model to test the significance of its inclusion must be a simple task. Conversely, a simple model may not offer the desired fidelity, particularly in the situation in which a vehicle is made of multiple materials, or where there is significant diversion from an isothermal surface. These capabilities must be provided for maximal utility of the model.

Verification:

Test RadiationPressure_1 on page 85

Requirement RadiationPressure_4: Temperature Structure

Requirement:

The Radiation Pressure Model shall incorporate realistic time-dependent temperature modeling of its surface.

Rationale:

Although thermal emission is typically a relatively small effect of radiation pressure modeling, the interaction between the vehicle and radiation is a primary contributor to the temperature structure of the vehicle, which may be desired by other models. Furthermore, there are situations in which spatial temperature variations do play a significant role in the radiation pressure calculation.

Verification:

Tests RadiationPressure_3 on page 90 and RadiationPressure_4 on page 94.

Requirement RadiationPressure_5: Albedo Pressure Option

Requirement:

The Radiation Pressure Model shall provide the ability to add – at a future time – a model to simulate the radiation pressure from sources other than the sun.

Rationale:

The solar radiation pressure will typically dominate, but secondary pressure (also known as albedo pressure) from planetary objects is necessary for a complete description and more accurate modeling. These other sources will provide a flux that will interact with the vehicle in exactly the same way as the solar flux, and should be handled by the same routines.

Verification:

Inspection RadiationPressure_3 on page 84.

Requirement RadiationPressure_6: Force and Torque

Requirement:
The data structures shall include storage of the force and torque on the vehicle.

Rationale:
The force and torque can be collected within Trick – or within any other simulation engine – from multiple interaction models to provide the overall force and torque on the vehicle for integration within the dynamics package for purposes of state propagation.

Verification:
Inspection RadiationPressure_4 on page 84.

2.2 Functional Requirements

Requirement RadiationPressure_7: Interaction Modeling

Requirement:
The model shall be capable of modeling the radiation interaction by appropriate combination of the effects of photon absorption, diffuse photon reflection (rough surface), specular photon reflection (smooth surface), and photon emission (thermal emission).

7.1 Temperature
The model shall be capable of independently modeling the temperature of each thermally independent component (facet) of the overall surface representation.

7.2 Plate Properties
Each component of the overall surface representation shall be described in terms of its material properties to allow the combination of these terms to produce the most physically realistic description of the surface-photon interaction.

Rationale:
Each type of interaction behaves in such a way to produce a different force resulting from the same incident flux. The relative "weighting" of each interaction-type is a function of the material properties, which may not be constant over an entire vehicle (e.g. the characteristics of solar array panels may be very different from those of a thermal shield). By assigning the material values to each facet, these local variations can be modeled and accounted for. The temperature of each plate will be particularly influential on the photon emission, which varies with temperature to the 4th power. The sides of the vehicle in shadow will tend to be significantly cooler than those that are in direct solar illumination, and this spatial temperature variation can be modeled on a facet-by-facet basis.

Verification:
Tests RadiationPressure_2 on page 85, RadiationPressure_3 on page 90, and RadiationPressure_4 on page 94

Requirement RadiationPressure_8: Temperature Integration

Requirement:

The model shall be capable of integrating the time-dependent temperature structure in order to maintain an energy balance between thermal energy, and net radiative power, derived from consideration of absorption and emission processes.

Rationale:

The temperature structure should have physical validity. Its integration must be independent of the overall state integration because of the influence that temperature variability can have on the state integration.

Verification:

Tests RadiationPressure_3 on page 90 and RadiationPressure_4 on page 94.

Requirement RadiationPressure_9: Flux Calculation

Requirement:

The model shall be capable of determining the magnitude and direction of the solar flux as a function of the current position with respect to the sun.

9.1 Orientation

The model shall be capable of calculating the orientation of the vehicle with respect to the flux vector.

Rationale:

For any given vehicle and vehicle-orientation, the force exerted on the vehicle is proportional to the magnitude of the incident flux. The interaction between the flux and the vehicle depends on the respective angles between the vehicle plates and the flux vector.

Verification:

Tests RadiationPressure_5 on page 97, RadiationPressure_6 on page 99, and RadiationPressure_11 on page 133.

Requirement RadiationPressure_10: Eclipse Calculation

Requirement:

The Radiation Pressure Model shall be capable of determining whether the vehicle is in full illumination, in the shadow of a third body, or in a position that is partially eclipsed (receiving partial illumination). In the case of a partial eclipse, the model shall be capable of calculating an adjustment to the background solar flux to account for that part of the solar disk that is not visible.

Rationale:

The radiation force depends on the flux, which can be affected by intervening objects. To accurately model the flux, the model must have the capability to account for those objects.

Verification:

Tests RadiationPressure_5 on page 97 and RadiationPressure_6 on page 99.

Requirement RadiationPressure_11: Eclipsing Bodies

Requirement:

The Radiation Pressure Model shall have the capability to calculate the shadowing effect of any one intervening body, and the option to extend that capability to multiple intervening bodies.

Rationale:

There are limited situations where more than one body could simultaneously provide significant shadowing, so typically only one object need be identified at any particular time. However, for simulations such as an Earth-Moon transit scenario, that object may change (from Earth to Moon, for example).

For future development, it is conceivable that it may be desirable to add the capability to monitor the eclipsing of non-planetary third bodies (such as an intervening vehicle). An example of this application would be the temperature modeling of components of a larger, composite vehicle, such as International Space Station (ISS).

Verification:

Inspection RadiationPressure_5 on page 102

2.3 Requirements Traceability

Table 2.1: Requirements Traceability

Requirement	Inspection and Testing
RadiationPressure_1 - Top-level Requirements	Insp. RadiationPressure_1
RadiationPressure_2 - State Encapsulation	Insp. RadiationPressure_2
RadiationPressure_3 - Surface Representation	Test RadiationPressure_1
RadiationPressure_4 - Temperature Structure	Test RadiationPressure_3 Test RadiationPressure_4
RadiationPressure_5 - Albedo Pressure Option	Insp. RadiationPressure_3
RadiationPressure_6 - Force and Torque	Insp. RadiationPressure_4
RadiationPressure_7 - Interaction Modeling	Test RadiationPressure_2 Test RadiationPressure_3 Test RadiationPressure_4
RadiationPressure_8 - Temperature Integration	Test RadiationPressure_3 Test RadiationPressure_4
RadiationPressure_9 - Flux Calculation	Test RadiationPressure_5 Test RadiationPressure_6 Test RadiationPressure_11
RadiationPressure_10 - Eclipse Calculation	Test RadiationPressure_5 Test RadiationPressure_6
RadiationPressure_11 - Eclipsing Bodies	Insp. RadiationPressure_5

Chapter 3

Product Specification

3.1 Conceptual Design

This section describes the key concepts found in the Radiation Pressure Model. For an architectural overview, see the Radiation Pressure Model *Reference Manual* [1].

3.1.1 Photon Interaction

The Radiation Pressure Model models the interaction between a vehicle and the radiation field in which it is located. This overall interaction can be divided and subdivided into four processes, each of which is well understood.

- Absorption

  When a photon strikes a surface, there is some probability that it will be absorbed by the surface, thereby raising the energy (observed as a rise in temperature) of that surface. The momentum of the photon must also be absorbed by the surface, since momentum is conserved; this change in momentum is interpreted as a force, called radiation pressure.

- Reflection

  There is some probability that the photon will simply bounce off the surface. While this will not directly affect the material itself, the momentum of the incident photon is absorbed, and the surface will also recoil in response to the momentum that the reflected photon carries in the opposite direction. There are two distinct types of reflection, *specular* and *diffuse*. The actual reflection profile of a surface can, in most cases, be modeled to fairly good precision as a combination of these two types, weighted according to the specific material.

  - Specular

    Specular reflection is the type of reflection seen from a smooth surface. The reflected photon leaves the surface at the same angle as the incident photon. The result of such reflection is that:

    * Parallel to the surface there is no net force, the photon retains its momentum in that direction.

    - Perpendicular to the surface, the photon momentum changes by double the component of the incident photon.
  - Diffuse

    Diffuse reflection is the type of reflection seen from dull, matte finishes. On a microscopic level, the surface is not smooth, and the reflected photon is reflected at some unpredictable angle. Statistically, diffuse reflection has been well modeled, and we have the Lambertian profile, which forms a cosine density function centered on the normal to the macroscopic surface. Integrating over that profile produces an effect whereby the entire diffuse reflection profile can be modeled as a single reflection along the normal to the macroscopic surface, but carrying only 2/3 of the photon momentum. The result of such reflection is that:
    - Parallel to the surface the photon momentum changes by the component of the incident photon in that direction.
    - Perpendicular to the surface, the photon momentum changes by the component of the incident photon in that direction, plus 2/3 of the magnitude of the photon momentum.

- Emission

  All bodies at temperatures above 0 Kelvin radiate to some extent, with a profile that varies with temperature to the 4th power. The thermal emission is also modeled with a Lambertian profile, i.e. 2/3 of the power radiated is considered, and it is all emitted normal to the macroscopic surface.

An alternative representation considers the vehicle to be an isothermal sphere with some fixed and isotropic combination of the above characteristics. It is then possible to represent all four interaction methods with a single value, called the Coefficient of Reflection. It must be realized that this is only relevant to a spherical, isotropic, isothermal surface; the mathematics describing the relation between the four interaction methods and this Coefficient of Reflection are developed in section *Coefficient of Reflection* (on page 32).

3.1.2 Radiation Field

The radiation field is the environment in which the surface is located. The field has two important effects on the vehicle: it provides energy and momentum. The energy imbalance (between absorbed and emitted radiation) is critically important when considering the thermal response of a vehicle to the environment. The momentum imbalance produces a net force and corresponding torque on the vehicle; although this force is typically dominated by aerodynamic uncertainties for low-Earth-orbits, it does become significant for lunar and deep-space missions.

The strength of that field is described by a vector quantity, called the flux, which (somewhat ambiguously) can be defined as the rate at which energy, or momentum, is transmitted through space. Often, when the intent is to interpret flux as being associated with the momentum, the flux is deliberately identified as the momentum-flux.

Often, there is a dominant source of radiation (typically, that is the sun) that is so overwhelming that any other sources provide negligible effect. If other sources are to be utilized, it is very

important to realize that the flux is not an additive vector. For example, two bright lights shining at each other do not add to produce zero flux in the middle. Even in situations where the forces perfectly cancel, it is unlikely that the torques also cancel, and impossible for the thermal responses to cancel, since those are a scalar effect. Therefore, it is important to consider all sources of flux as being inherently independent, and the radiation field must be considered to be comprised of multiple independent subfields rather than a single accumulated field.

When extending the model to include planetary albedo effects, this concept is particularly important. While for distant sources it is reasonable to make the assumption that the incident radiation is all parallel, that is an invalid assumption where the source covers a significant solid angle of the environment. In the latter case, a single source can simultaneously illuminate opposite sides of a single vehicle, and the magnitude of the source itself is a function of the vehicle orientation, as different surfaces 'see' more or less of the source.

3.1.3 Surface Model

The interaction is between the radiation field and the surface of the vehicle, so we must have some model of the surface. This can be modeled simply, using the default spherical isothermal surface, or with greater fidelity using the JEOD Surface Model (first available in JEOD version 2.0.1).

3.1.4 Temperature

The emission from the vehicle depends strongly on surface temperature, and so temperature must be modeled for an accurate representation. For this, the Radiation Pressure Model relies on the JEOD Thermal-rider model (first available in JEOD version 1.5).

3.2 Mathematical Formulations

3.2.1 Reduction of Flux by Intervening Body

The flux algorithm is qualitatively described in the section *calculate_flux* (on page 57). The mathematical treatment of flux starts with a trivial inverse-square-law application to obtain a raw flux. A secondary calculation then identifies whether that flux is being eclipsed by an intervening body. This section covers that identification, and some of the easier cases. The intricate calculations to determine the fractional eclipse that occurs in a scenario in which a part of the solar disk is eclipsed are described in the next section (the *Conical Eclipse Calculator* (on page 18)).

The *Process Architecture* (on page 35) contains flow charts (see, in particular, Figures 3.22 and 3.11) that assist in interpreting the interdependence of these methods.

3.2.1.a Introduction to Problem

Suppose that we have a spacecraft orbiting a planet, and that the reference frame in which the position of the spacecraft is known is arbitrarily oriented, but planet-centered.

Suppose further that the planet is being illuminated by a flux with vector $\vec{\phi} = \phi\hat{\phi} = \phi\ [\phi_1, \phi_2, \phi_3]$ in that same reference frame, and that the spacecraft is at a position $[x, y, z]$.

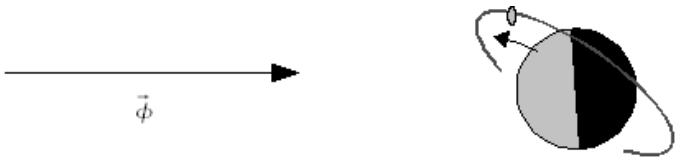


Figure 3.1: Shadowing overview

Define a new coordinate system, $\hat{a}, \hat{b}, \hat{c}$, such that $\hat{a}$ is oriented with the solar flux vector $\vec{\phi}$ for the planet (i.e. from the sun to the planet), and $\hat{b}$ and $\hat{c}$ form a plane perpendicular to the sun.

The origin of this coordinate system is concentric with the inertial system, so that $|\vec{r}_{xyz}| = |\vec{r}_{abc}|$. The new system should be right-handed, so that $\hat{b} \times \hat{c} = \hat{a}$.

The orientation of the b- and c- axes is arbitrary, and there are an infinity of possibilities for such a coordinate system. However, because we have axial symmetry, it is not necessary to define these axes. The position vector can, instead, be expressed in terms of two components – one parallel, and one perpendicular, to the flux vector.

$\vec{r}$ represents the position of the vehicle with respect to the shadow body.

$\vec{d}$ represents the position of the vehicle with respect to the sun.

r, d represent the magnitudes of $\vec{r}, \vec{d}$ respectively.

Then the components of the position vector that are parallel and perpendicular to the flux vector are (respectively):

$r_{\parallel} = \vec{r} \cdot \hat{\phi}$

$r_{\perp} = \sqrt{r^2 - r_{\parallel}^2}$

NOTES

- $r_{\parallel} \leqslant 0$ places the vehicle on the sunward side of the planet. In this case, there can be no eclipse.
- $r_{\parallel} > 0$ places the vehicle on the shadow-side of the planet. In this case, the determination of whether there is an eclipse depends on the value of $r_{\perp}$.
- $r_{\perp} > 0$ always; the positive square root is taken.

3.2.1.b Cylindrical shadowing

The simplest consideration is one in which the planet casts a cylindrical shadow.

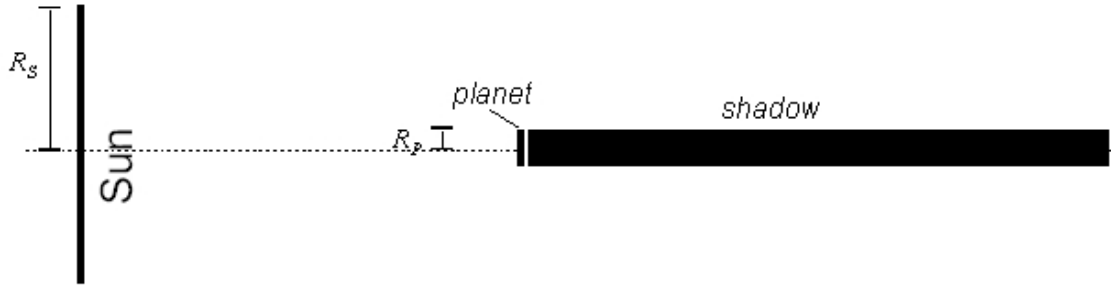


Figure 3.2: Cylindrical shadow geometry

If the spacecraft is in a position such that $r_{\parallel} > 0$ and $r_{\perp} < R_P$, then it will be in 100% shadow. Otherwise, it will be 100% illuminated.

3.2.1.c Conical shadowing

A more complicated scenario is to consider the more realistic conical shadow:

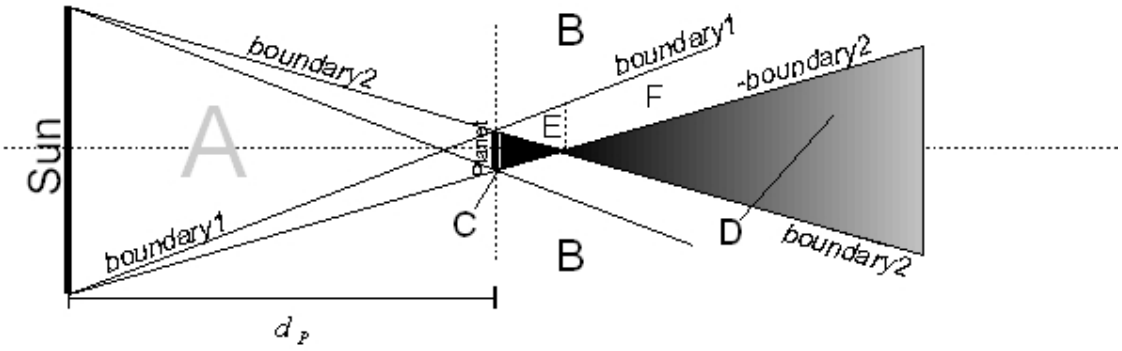
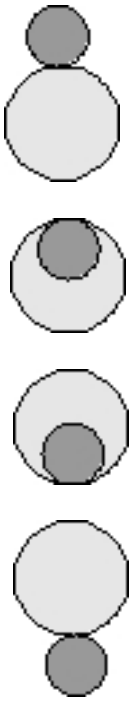


Figure 3.3: Conical shadow geometry

The following sequence of diagrams may help with visualization of the how the relative positions of a planet and the sun might change as a vehicle moves across the eclipse field. For this sequence, consider Figure 3.3 with a vehicle moving from bottom to to through regions B-F-D-F-B.



- -boundary 1 (B-F): the circle of the planet and that of the sun make first contact
- boundary 2 (F-D): the planet disk is completely contained within the solar disk
- -boundary2 (D-F): the planet disk begins to leave the solar disk
- boundary1 (F-B): the two circles separate

The same sequence could also describe a similar path, from top to bottom, moving through regions B-E-C-E-B

- boundary 1 (B-E): the circle of the planet and that of the sun make first contact
- boundary 2 (E-C): the planet disk completely covers the solar disk
- -boundary2 (C-E): the solar disk starts to emerge from behind the planet disk
- -boundary1 (E-B): the two circles separate

The two boundaries can be represented as simple lines, $y = mx + b$. The gradient for the two boundaries is easily evaluated using the distance between the sun and planet, d_P.

- boundary1: $\frac{R_P+R_S}{d_P}$
- boundary2: $\frac{R_P-R_S}{d_P}$

The y-intercept for both boundaries is R_P.

Thus, the two boundaries are defined by the relations:

- boundary1: $r_\perp = \left(\frac{R_P+R_S}{d_P}\right) r_\parallel + R_P$
- boundary2: $r_\perp = \left(\frac{R_P-R_S}{d_P}\right) r_\parallel + R_P$

For the purposes of algebra, let b_1 and b_2 represent the values of $r_\perp$ that would lie on boundary 1 and boundary 2, respectively, at some value of $r_\parallel$.

Region A of Figure 3.3, is characterized by $r_\parallel < 0$. In this region there will never be any shadow and the full disk is counted.

Region B of Figure 3.3 is characterized by $r_\parallel > 0$ and $r_\perp \geqslant b_1$. In this region, the full disk will be counted.

NOTES:

- $r_\perp > b_1 \implies r_\perp d_P \geqslant (R_P + R_S) r_\parallel + R_P d_P$

Region C of Figure 3.3 is characterized by $r_\parallel > 0$ and $r_\perp \leqslant b_2$. In this region, the eclipse is total; none of the disk will be counted, $I = 0$.

NOTES:

- $r_\perp < b_2 \implies r_\perp d_P \leqslant (R_P - R_S) r_\parallel + R_P d_P$
- Because $R_P < R_S$ (typically) and $r_\perp \geqslant 0$, this becomes impossible at large $r_\parallel$, as expected.

Region D of Figure 3.3 is characterized by $r_\parallel > 0$ and $r_\perp \leqslant -b_2$. In this region, the eclipse will be annular, and a function of $r_\parallel$ only (i.e. independent of $r_\perp$).

NOTES:

- $r_\perp \leqslant -b_2 \implies r_\perp d_P \leqslant (R_S - R_P) r_\parallel - R_P d_P$
- For modeling shadowing from a source other than the sun, it could be possible to have $R_P > R_S$. In this case, the left hand side of the inequality is positive and the right-hand-side negative-definite. The inequality cannot be satisfied and region D is not available.

In the diagram below, the white circle represents the solar disk, and the shaded circle the intervening planetary disk.



The planetary angular size is $\frac{2R_P}{r}$ and the solar angular size is $\frac{2R_S}{d}$.

The area of the disk is proportional to the square of the angular size;

hence the planetary disk obscures $\left(\frac{R_P \cdot d}{R_S \cdot r}\right)^2$ of the solar disk,

and the illumination becomes

$$I = 1 - \left(\frac{R_P \cdot d}{R_S \cdot r}\right)^2$$

Region E of Figure 3.3 is characterized by $(r_\parallel > 0)$ and $(r_\perp > b_2)$ and $r_\perp < b_1$ and $\frac{R_P}{r} > \frac{R_S}{d}$. In this region, the eclipse will be partial, transitioning to total. The magnitude of the eclipse effect will be a function of both $r_\parallel$ and $r_\perp$. The resulting illumination will transition from 100% on boundary1 to 0% on boundary2.

NOTES:

- Algorithmically, this set of constraints is acheived when the tests for regions A-D have failed and $R_P \cdot d > R_S \cdot r$

In the diagram below, the white circle represents the solar disk, and the shaded circle the intervening planetary disk.



The mathematical treatment of this transition is described in subsection 3.2.2.

Region F of Figure 3.3 is characterized by $(r_{\parallel} > 0)$ and $(r_{\perp} > b_2)$ and $r_{\perp} < b_1$ and $\frac{R_P}{r} \geqslant \frac{R_S}{d}$. In this region, the eclipse will be partial, transitioning to annular. The magnitude of the eclipse effect will be a function of both $r_{\parallel}$ and $r_{\perp}$. The resulting illumination will transition from 100% on boundary1 to the value determined for region D on (-)boundary2.

NOTES:

- Algorithmically, this set of constraints completes the set. It is achieved when tests for all other regions have failed.

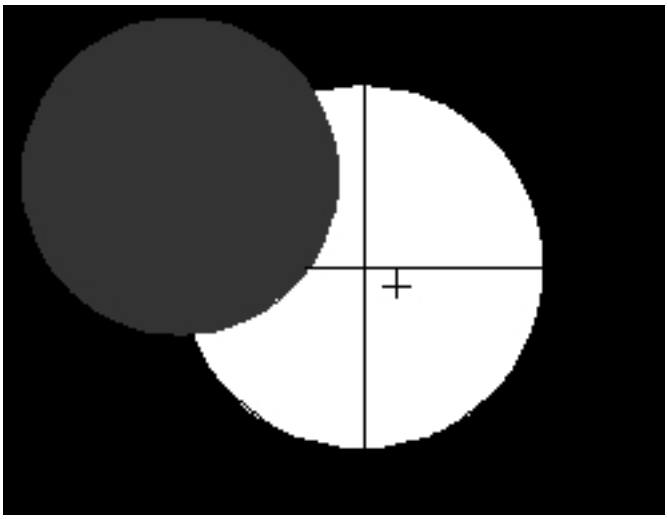


The mathematical treatment of this transition is described in subsection 3.2.2.

3.2.1.d The flux unit-vector

The direction of the flux acting on the spacecraft in full illumination is going to be on a line from the center of the sun reference frame to the center of pressure of the spacecraft (or, in the case of a representation in terms of multiple facets, the desired vectors would be to the center of pressure of each facet). The center of pressure of each facet is defined relative to the structural reference frame in the Surface Model.

When the sun is partially obscured, the origin of the flux vector will be shifted slightly from the center of the sun to the center of radiation, as shown in the diagram below, with the cross representing the center of radiation of the partially obscured solar disk.



If we were to be concerned with the consequential change in direction of the flux vector, it would no longer be sufficient to work in the axially symmetric, two-dimensional system just developed. Instead, a full 3-dimensional treatment would be required, which would add significant computational expense, with minimal impact on the results. The radiation pressure is a relatively small force, and the effect of the deviation of the direction of the flux vector over such small angles is negligible compared to other approximations being made.

The direction of the flux vector is therefore assumed to be on the direction from the center of the sun to some reference point on the spacecraft. Which reference point should be used is not particularly relevant for the purposes of determining this direction; using the wrong reference point will have even less of an effect than using the wrong center of radiation for the source. Consequently, the same flux vector will be used for all of the facets that make up the vehicle.

3.2.2 Conical Eclipse Calculator

This subsection describes the method used for calculating the extent of shadowing for partial spacecraft shadowing cases, such as would occur in in regions E and F of Figure 3.3.

3.2.2.a Definitions

The front body (the eclipsing body) will be given the subscript *1*.

The rear body (the sun) will be given the subscript *2*.

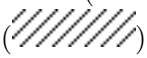
The term "radius", as used in this description, is not always intended to represent the physical radius, but often the angular radius, as observed from the point of interest. Reference will be made to places where the radius of the planet is larger than that of the sun. This does not imply that the planet must be physically larger, just that it occupies a larger solid angle due to its relative proximity. Typically, linear distances are represented by Roman lettering (R, r) and angles by Greek lettering (θ, ψ, ϕ). Ultimately though, we are looking for a fractional area, so both the linear and angular sizes will become moot.

3.2.2.b Introduction

Figure 3.4 shows the area of the shaded subsection of a circle, defined by angle θ and radius R. It can be expressed as

$$A = R^2(\theta - sin\theta cos\theta)$$

For two intersecting circles, the area can be identified by considering it as a sum of two such two circle segments, as shown in Figure 3.5 :

In this diagram, the shaded area on the left () is attributed to the circle on the right (labeled 2), while the area on the right (), is attributed to the circle on the left (labeled 1). The total shaded area is then:

$$A \;=\; A_1 + A_2 \;=\; {R_1}^2(\theta_1 - \cos\theta_1 \sin\theta_1) + {R_2}^2(\theta_2 - \cos\theta_2 \sin\theta_2)$$

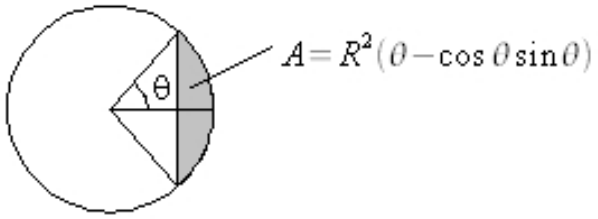


Figure 3.4: The area of the segment of a circle

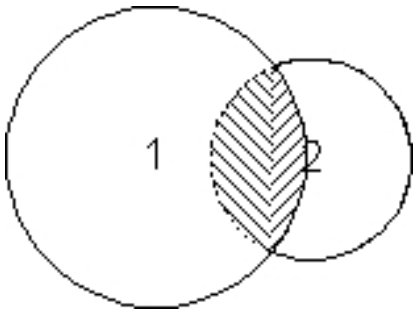


Figure 3.5: The area of overlap of two circles, decomposed into two segments

3.2.2.c Direct Computation Method

The desired output value is the fraction of this area with respect to the area of the solar disk. Without loss of generality, assign subscript 2 to the solar disk and subscript 1 to the planetary disk. Use upper-case R_i to represent the size of the disk, lower-case r_i to represent the distance to the disk, and ψ_i to represent the angular radius of the disk.

$$\psi_i = \frac{R_i}{r_i}$$

Assign variables to the ratio of angular radii and area:

$$\rho = \frac{\psi_1}{\psi_2}$$
$$\alpha = \frac{A}{\pi R_2^2}$$

Thus the fractional area α can be expressed in terms of ρ and θ_i, independent of R_i, r_i, and ψ_i:

$$\alpha = \frac{1}{\pi}\left(\rho^2(\theta_1 - sin\theta_1 cos\theta_1) + \theta_2 - \cos\theta_2 \sin\theta_2\right)$$

To compute the angles θ_i, we start with a triangle with vertices at the two disk centers and one of the points of intersection, as shown in Figure 3.6.

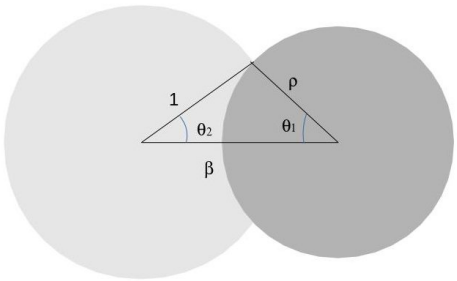


Figure 3.6: The scaled angular sizes of the two disks.

The scaled separation between the disk centers is

$$\beta = cos\theta_2 + \rho cos\theta_1$$

The perpendicular distance from the line joining the centers to the intersection of the two circles is

$$sin\theta_2 = \rho sin\theta_1$$

Consequently, the values of θ_i can be determined:

$$\begin{aligned}
\rho^2 cos^2\theta_1 &= (\beta - cos\theta_2)^2 \\
\rho^2 cos^2\theta_1 &= \rho^2(1 - sin^2\theta_1) = \rho^2 - sin^2\theta_2 = \rho^2 - 1 + cos^2\theta_2 \\
(\beta - cos\theta_2)^2 &= \rho^2 - 1 + cos^2\theta_2 \\
cos\theta_2 &= \frac{\beta^2 + 1 - \rho^2}{2s} \\
cos\theta_1 &= \frac{\beta^2 - 1 + \rho^2}{2s\rho}
\end{aligned}$$

This still requires an expression for the scaled separation "distance", β. The angle ϕ between the two centers can be obtained from the cosine rule:

$$d_P^2 = r_1^2 + r_2^2 - 2r_1 r_2 cos\phi$$

where r_i are the linear distances from the vehicle to the respective disk-centers, and d_P is the linear distance between disk centers. β and ϕ are related by the angular radius of the solar disk, ψ_2.

$$\beta = \frac{\phi}{\psi_2} = \frac{cos^{-1}\left(\frac{r_1^2 + r_2^2 - d^2}{2r_1 r_2}\right)}{\psi_2}$$

Notes:

- This is a cumbersome calculation, requiring inverse-trigonometric functions to obtain β, θ_1 and θ_2 and 2 square-root functions to determine $sin\theta_1$ and $sin\theta_2$.
- The calculation $\theta_i = cos^{-1}(cos\theta_i)$ requires that $-1 \leqslant cos\theta_i \leqslant 1$ but the expressions for $cos\theta_i$ have no intrinsic limit protection on them. However, this requirement is handled within the overall model algorithm because this computation is only reached where $abs(1-\rho) \leqslant \beta \leqslant 1+\rho$; this constraint is equivalent to the necessary constraint on the value of $cos\theta_i$.

An alternative approach provides a very close approximation using a cubic polynomial of the separation distance with coefficients that are themselves quadratic or cubic polynomials of the ratio of the disk radii. This alternative approach improves speed by a factor of approximately 4.

3.2.2.d Alternative Representation

Using a system of ratios, it is straightforward to identify the extent of the eclipse between none and maximum, where maximum occurs when the smaller disk is completely covered by the larger disk. Note - in the case where $R_2 < R_1$, maximum eclipse is total eclipse, whereas when $R_1 < R_2$, maximum eclipse will be when the solar disk completely encompasses the planetary disk.

We can introduce δ, $\delta \in [0, 1]$ to represent the ratio of the current extent of the eclipse to its maximum value.

δ has value *0* when the limbs make first contact, and value *1* at second contact (total occlusion). It is associated with the linear coverage distance $s = 1 + \rho - \beta$ (see Figure 3.7) rather than with the occluded area, and increases linearly with s.

Note that the distance between first contact and second contact is the diameter of the smaller disk. Thus $s \in [0, min(2R_1, 2R_2)]$.

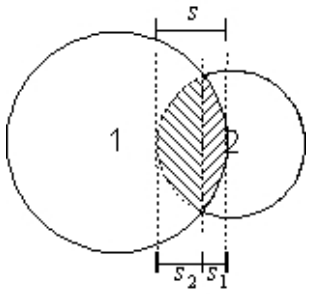


Figure 3.7: Overlap of two circles

Thus,

$$s = \begin{cases} \delta \cdot 2R_2 & R_2 < R_1 \\ \delta \cdot 2R_1 & R_1 < R_2 \end{cases}$$

In the previous section, we developed an expression for the overlap area as a fraction of the area of the solar disk. Suppose, instead, we had developed an expression for the overlap area as a fraction of the smaller disk. Let us label this new fractional area α_- and label the smaller radius as R_-.

Then

$$s = 2\delta \cdot R_{-}$$

Note that the upper bound for α_{-} is independent of the relative sizes of the two disks (unlike the upper bound for α). We always have $0 \leqslant \alpha_{-} \leqslant 1$.

Now, with the issue associated with relative sizes set aside, we can obtain an expression for α_{-} in terms of δ.

Notice that the final desired result is still α, and that must depend on whether the solar disk is smaller or larger than the planetary disk because we are, after all, looking for the fractional covering of the *solar* disk. This dependency can be brought back in after determining a value for α_{-}:

$$\alpha = \begin{cases} \alpha_{-} & \rho \geqslant 1 \\ \rho^2 \alpha_{-} & \rho < 1 \end{cases}$$

3.2.2.e Polynomial Approximation

The quantity α_{-}, for all its complexity, is a surprisingly well behaved function of δ and can be approximated to very high precision with a cubic polynomial in δ

$$\alpha_{-} = a_3\delta^3 + a_2\delta^2 + a_1\delta + a_0$$

Value of δ

Recall that δ is a linear function of the overlap distance, which is measured by $r_{\perp}$. It starts with value 0 on boundary b_1 at first contact and climbs to value 1 at second contact on boundary b_2 (or $-b_2$, depending on the value of $r_{\parallel}$). See Figure 3.3 for an illustration of these two boundaries.

Let b_i represent the value of $r_{\perp}$ on boundary i. Then:

$$\delta = \frac{b_1 - r_{\perp}}{b_1 - |b_2|}$$

Note that the value of b_2 goes negative for $r_{\parallel}$ greater than some critical value, as the planetary disk becomes smaller than the solar disk. To a reasonable approximation, the switch in sign occurs when $\rho = 1$ (this is true where $r_{\perp} = 0$, but not quite true off that axis.)

$$|b_2| \approx \begin{cases} b_2 & \rho \geqslant 1 \\ -b_2 & \rho \leqslant 1 \end{cases}$$

Thus,

$$\delta \approx \begin{cases} \frac{b_1 - r_{\perp}}{b_1 - b_2} & \rho \geqslant 1 \\ \frac{b_1 - r_{\perp}}{b_1 + b_2} & \rho \leqslant 1 \end{cases}$$

The values b_i can be obtained from geometry (see Figure 3.3 for an illustration of these two boundaries) using simple linear expression $y = mx + b$.

$$
\begin{aligned}
b_1 &= m_1 r_{\parallel} + R_P \\
b_2 &= m_2 r_{\parallel} + R_P \\
m_1 &= \frac{R_P + R_S}{d_P} \\
m_2 &= \frac{R_P - R_S}{d_P}
\end{aligned}
$$

Thus,

$$
\delta \approx \begin{cases} \frac{(R_P - r_{\perp})d_P + (R_P + R_S)r_{\parallel}}{2R_S r_{\parallel}} & \rho \geqslant 1 \\ \frac{(R_P - r_{\perp})d_P + (R_P + R_S)r_{\parallel}}{2R_P(d_P + r_{\parallel})} & \rho < 1 \end{cases} \tag{3.1}
$$

Value of the coefficients a_i

The coefficients a_i are themselves functions of the radius ratio ρ' and can be approximated by quadratic or cubic functions of ρ':

$$
\rho' = \frac{R_-}{R_+} = \begin{cases} \frac{1}{\rho} & \rho \geqslant 1 \\ \rho & \rho \leqslant 1 \end{cases}
$$

Curve fitting to the analytic solution produces

$a_3(\rho') = 0.758656\rho'^2 - 0.0637441\rho' - 1.08955$

$a_2(\rho') = -1.2205\rho'^2 - 0.61815\rho' + 1.629209$

$a_1(\rho') = 0.43723\rho'^2 - 0.529210\rho' + 0.473649$

$a_0(\rho') = -0.01242\rho'^3 - 0.0036715\rho'^2 + 0.0150263\rho' - 0.0078259$

The approximation is now complete.

$$
\alpha \approx \begin{cases} \rho^2 \left(a_3(\rho) \cdot \delta^3 + a_2(\rho) \cdot \delta^2 + a_1(\rho) \cdot \delta + a_0(\rho) \right) & \rho < 1 \\ a_3\left(\frac{1}{\rho}\right) \cdot \delta^3 + a_2\left(\frac{1}{\rho}\right) \cdot \delta^2 + a_1\left(\frac{1}{\rho}\right) \cdot \delta + a_0\left(\frac{1}{\rho}\right) & \rho \geqslant 1 \end{cases} \tag{3.2}
$$

Figure 3.8 shows the analytic function and the polynomial approximation for values of

$\rho = 30,\ 2,\ 1,\ 0.75,\ 0.5,\ 0.25,\ 0.1$.

The analytic solution is shown as a solid line, and the approximation as a dotted line for each case. The difference between the two lines is small for all cases.

3.2.2.f Boundary Conditions and Numerical Limitations

This polynomial approximation should meet certain boundary conditions; namely that

$$
\delta = 0 \Rightarrow \alpha = 0
$$

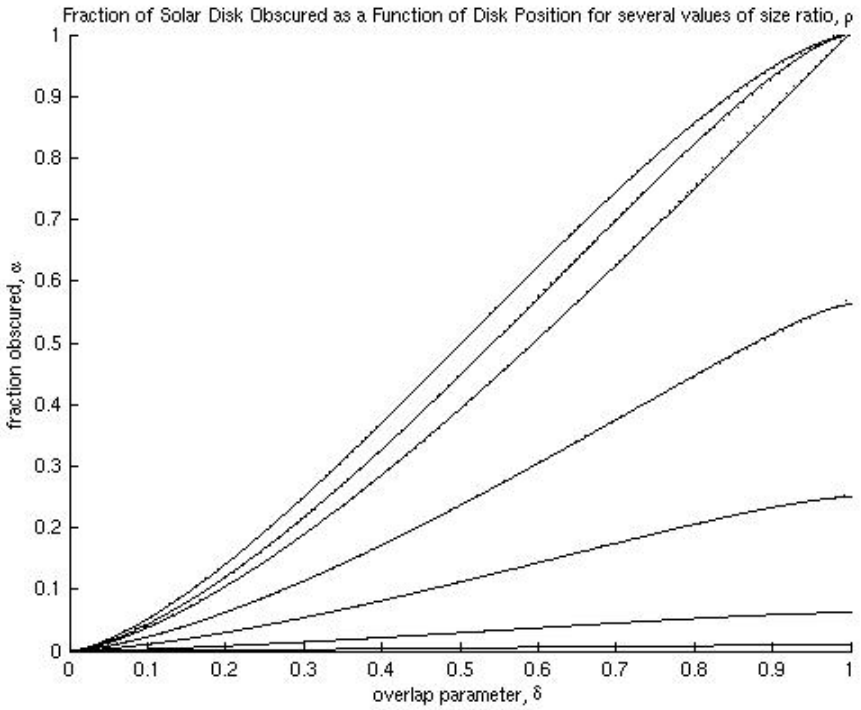


Figure 3.8: Comparison of polynomial approximation to analytical solution for partial eclipse calculation

$$\delta = 1 \Rightarrow \begin{cases} \alpha = \rho^2 & \rho < 1 \\ \alpha = 1 & \rho \geqslant 1 \end{cases}$$

The coefficients given are a result of best-fit curve-fitting to data that intrinsically included the boundary conditions, but the polynomial approximation does not explicitly satisfy those conditions.

For $\delta = 0$, $\alpha \in \left[-8.9 \times 10^{-3}, -2.7 \times 10^{-3}\right]$. For small values of δ , the approximated value of α is less than the true value, with a maximum error of 0.89% when $\rho = 1.0$.

For $\delta = 1$ and $\rho < 1$, $\alpha = \rho^2 \left(-0.01242\rho^3 - 0.0283\rho^2 + 0.04022\rho + 1.0054821\right) \in [1.005, 1.017] \times \rho^2$.

For $\delta = 1$ and $\rho > 1$, $\alpha \in [1.005, 1.017]$, For large values of δ , the approximated value of α is greater than the true value, with a maximum error of 1.7% when $\rho = 0.53$ or $\rho = 1.9$.

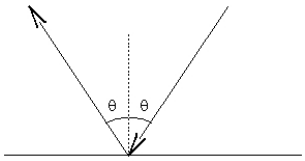
3.2.3 Response to Flux

The interaction of matter and radiation is, to say the least, difficult to model. Advances in computer graphics have produced excellent techniques for very finely detailed, empirically based, rendering of that interaction, but these techniques are typically computationally expensive, and do not lend themselves to efficient simulation of orbital dynamics.

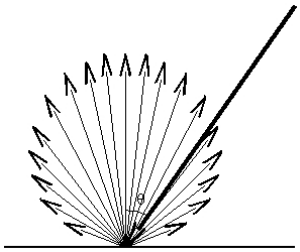
Subsection 3.2.3.a describes the technique used in this model, and subsection 3.2.3.d describes the validation of that technique.

3.2.3.a Treatment of Flux in Calculation of Radiation Force

Specular Reflection: is the type of reflection observed from smooth surfaces, such as mirrors. Incident radiation is reflected at the same wavelength and at the same angle so that a clear image could, in principle, be obtained.

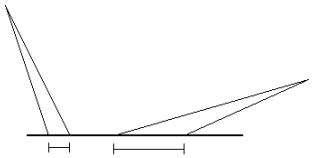


Diffuse Reflection is the type of reflection observed from rough surfaces. With a reflection that is totally diffuse, all information about the direction of the incident radiation is lost, but the radiation is still reflected at the same wavelength as the incident radiation.



Absorption/Emission: *absorption* is a process by which the incident radiation interacts with the surface structure and transfers its energy and momentum to the object. This energy and momentum may be partially or entirely *emitted,* spontaneously, at some later time, but may be in any direction, and at any wavelength. "Some later time" may be instantaneous.

Lambertian reflection is a special type of diffuse reflection, in which the apparent brightness of the reflected light is constant for all viewing angles. It is clear from the accompanying diagram that, when comparing the view through a given solid angle, an observer looking at the surface from an angle close to parallel to the plane (right side of diagram) will see more of the surface than an observer looking at the surface from an angle close to perpendicular to the plane (left side of diagram). If the emission were isotropic, the surface would appear brighter the closer the observer's line of sight came to being parallel with the plane. Instead, an emission profile that varies as $\cos\theta$ factors in the increased surface area factor and produces a uniform brightness for all viewing angles. One of the consequences of Lambertian reflection is that of axial symmetry, i.e. the reflected light is symmetric about the normal to the plane. For the purposes of momentum transfer, the integration over the full range of reflected light yields that Lambertian reflection is equivalent to absorbing the incident light and emitting 2/3 of it along the normal to the plane. Note that this is not true for purposes of calculating energy transfer.



The Lambertian profile can also be used to represent the thermal emission from the surface; then, for an object in thermal equilibrium, the two processes (diffuse reflection and absorption/emission) generate the same force (per involved photon of any given energy).

3.2.3.b Real Reflection

Reflection is a complex process involving the interaction of the radiation with the electromagnetic potentials created by the atomic/molecular structures at and near the surface of the object. The effect of these interactions can be empirically measured for any given surface, and recorded in a Bidirectional Reflectance Distribution Function (BRDF). This function provides information on the relative strength of the reflected radiation per solid angle, based on the solid-angle of incidence. Many surfaces have a high degree of symmetry about the normal axis, simplifying the 4-dimensional BRDF significantly. Even so, assuming axial symmetry and therefore requiring only a 2-dimensional BRDF for each individually oriented exterior surface of a spacecraft is still an unrealistic expectation.

3.2.3.c Overview of Method

Photons interacting with matter are typically scattered, or absorbed. We assume that the plates of the spacecraft are opaque, so that the scattering process can be treated as reflection, with no transmission. This model eliminates the computational overhead associated with an approximated BRDF, and instead relies on a combination of absorption/emission, a Lambertian model of diffuse reflection, and specular reflection. In this approximation, we assume that some fixed fraction of the incident radiation is absorbed, and some fraction reflected. Of the reflected radiation, some fraction is reflected specularly and some diffusely. The extent to which these three processes contribute to the overall interaction is material dependent, and set by the user for each plate.

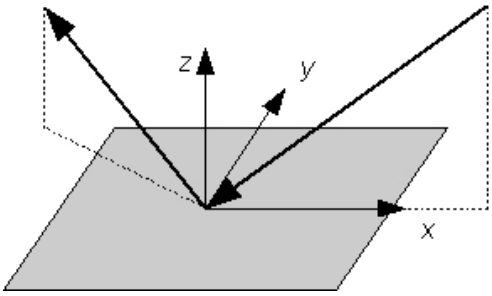
3.2.3.d Validation of Radiation Interaction Approximations

To test the validity of this approximation, a series of axially symmetric BRDFs were established by constructing a surface from pseudo-randomly oriented microscopic plates, and assuming specular reflection or absorption at each microscopic plate. For each surface (of multiple plates), an approximation was attempted that would match the observed force characteristics to those of the new model. For each surface, a reasonable model was identified.

The fundamental consideration is that a photon hits the surface and the surface responds in absorbing that momentum, and a photon leaves the surface, causing a response in reaction. Across a wide range of materials and angles of incidence, changing the relative strengths of the three processes causes differences of only a few percent from one another, and a precise determination of the weighting values is generally not necessary.

3.2.3.e The Microscopic Plate Model

Set up axes such that the surface lies in the x-y plane, with the z-axis defining the surface normal. Radiation is incident along the x-axis, from +x, hitting the surface at the coordinate origin.



As each photon is incident on the surface, it hits a random microscopic plate, and is reflected accordingly. The orientation of each plate is determined by the following algorithm:

1. Rotate the plate about the z-axis by some angle θ_z with uniform distribution on $0 \leqslant \theta_z < \pi$
2. Rotate the plate about the *rotated* x-axis by some angle θ_x with normal distribution with mean 0 and standard deviation determined by the desired "roughness" of the surface.

Define:

$\hat{i}, \hat{j}, \hat{k}$ The unit vectors defined by the coordinate system above

$\hat{p}$ to be the unit vector directed *against* the incident radiation

$\hat{s}$ to be the unit vector associated with the reflected radiation, and x, y, and z by $\hat{s} = x\hat{i} + y\hat{j} + z\hat{k}$

$\hat{n}$ to be the unit vector normal to the plate

ϕ to be the angle between $\hat{k}$ and $\hat{p}$

$\hat{s} = x\hat{i} + y\hat{j} + z\hat{k}$ can be found in terms of θ_x and θ_z by the method following.

The original reference frame, *O,* is rotated to a new reference frame, *O'*, such that the surface normal is oriented along $\hat{k}$' , the local z-axis in *O'*. The axes in *O'* can be expressed in terms of the angles θ_x and θ_z so that

$$\hat{i}' = \cos\theta_z\hat{i} + \sin\theta_z\hat{j}$$

$$\hat{j}' = -\cos\theta_x\sin\theta_z\hat{i} + \cos\theta_x\cos\theta_z\hat{j} + \sin\theta_x\hat{k}$$

$$\hat{k}' = \sin\theta_x\sin\theta_z\hat{i} - \sin\theta_x\cos\theta_z\hat{j} + \cos\theta_x\hat{k}$$

Thus the transformation matrix representing the rotation from *O* to *O'* is

$$\begin{bmatrix} \cos\theta_z & \sin\theta_z & 0 \\ -\cos\theta_x\sin\theta_z & \cos\theta_x\cos\theta_z & \sin\theta_x \\ \sin\theta_x\sin\theta_z & -\sin\theta_x\cos\theta_z & \cos\theta_x \end{bmatrix}$$

Thus, the vector $\vec{p}$ can be transformed into *O'*. In *O'*, the beam is reflected in the local x-y plane at the origin, so $\hat{s}' = \begin{bmatrix} s_1' \\ s_2' \\ s_3' \end{bmatrix} = \begin{bmatrix} -p_1' \\ -p_2' \\ p_3' \end{bmatrix}$

Then this can be transformed back into the frame *O* by the transpose of the rotation matrix to produce the unit vector:

$$\hat{s} = \begin{bmatrix} \sin\phi(2\sin^2\theta_x \sin^2\theta_z - 1) + \cos\phi(2\sin\theta_x \cos\theta_x \sin\theta_z) \\ \sin\phi(-2\sin^2\theta_x \sin\theta_z \cos\theta_z) + \cos\phi(-2\sin\theta_x \cos\theta_x \cos\theta_z) \\ \sin\phi(2\sin\theta_x \cos\theta_x \sin\theta_z) + \cos\phi(\cos^2\theta_x - \sin^2\theta_x) \end{bmatrix}$$

Integrating over all values of θ_x and θ_z, weighted by the respective probability functions

$$p(\theta_z) = \frac{1}{\pi} d\theta_z$$

$$p(\theta_x) = \sqrt{\frac{c}{\pi}} \left(e^{-c{\theta_x}^2} \right) d\theta_x$$

produces an expected value

$$\langle \hat{s} \rangle = \begin{bmatrix} -0.5 \ \sin\phi \ \left(1 + e^{-\frac{1}{c}}\right) \\ 0 \\ \cos\phi \ e^{-\frac{1}{c}} \end{bmatrix} \tag{3.3}$$

The value c in the above expressions is a measure of the "smoothness" of the surface; a small value c tends to a uniform distribution on θ_x, indicative of a rough surface, while a large value of c tends to produce a delta distribution on θ_x (i.e. all parallel to the surface), indicative of a smooth surface.

Note that the average of unit vectors will typically not be a unit vector; this value is not a unit vector.

This value also includes photons reflected "down" into the surface, which should undergo subsequent reflections until either absorbed or reflected out.

Instead of using this value for the calculation of the force due to reflection, the value was numerically simulated by firing 10^6 photons at the surface and tracking each photon until it was either absorbed, or reflected out. For code validation, this process was changed slightly so that all down-reflected photons were ignored, and instead re-calculated as newly incident photons; these results matched the analytic solution to better than 0.5%. The differences observed when the effect of multiple reflections is included is seen mostly on the z-axis, and increases with both the incident angle, ϕ and the "smoothness" factor, c. The results of this investigation can be found in the section *Validation of Flux-Vehicle Interaction-Model Principles* (on page 103).

The incremental force added to the surface by a photon is

$$d\vec{F} = \begin{Bmatrix} \phi_\gamma \left(-\hat{p} - \hat{s}\right) & \textit{reflection} \\ \phi_\gamma \left(-\hat{p} - \frac{2}{3}\hat{k}\right) & \textit{absorption} \end{Bmatrix}$$

and the total force

$$\vec{F} = \sum_{i=1}^{i=N} d\vec{F}_i \tag{3.4}$$

The simplifying model used here, that avoids individual photon mapping, assumes that some fraction α is reflected form the plane of the whole surface, of which some fraction δ is reflected totally

diffusely and $1-\delta$ is reflected purely specularly. The balance $(1-\alpha)$ is absorbed and re-emitted. The re-emission profile is assumed to be Lambertian, as is the diffuse reflection profile.

The unit vectors $\hat{p}$ and $\hat{s}$ have the same meaning as before, with

$\hat{p} = \sin\phi\hat{i} + \cos\phi\hat{k}$

$\hat{s} = -\sin\phi\hat{i} + \cos\phi\hat{k}$

The total force then is

$$\vec{F} = N\phi_\gamma \left(-\hat{p} - \alpha(1-\delta)\hat{s} - \frac{2}{3}(1-\alpha(1-\delta))\hat{k} \right)$$

Note: under the assumption of thermal equilibrium, there is no distinction to be made between the force-expression associated with diffuse reflection and that with absorption/emission. The force equation can therefore be simplified by saying some fraction σ is reflected specularly:

$$\vec{F} = N\phi_\gamma \left(\hat{i}(\sin\phi)(\sigma-1) + \hat{k}\left((\sigma-1)\left(\frac{2}{3} - \cos\phi\right) - 2\cos\phi \right) \right) \tag{3.5}$$

The value σ can now be found as the value that minimizes the error between the two forces (equations 3.4 and 3.5)

The value α can be found from a count of the number of absorbed and reflected photons in the microscopic plate model,

$$\alpha = \frac{n_{reflected}}{n_{photons}}.$$

The value δ can be found from α and σ .

A validation of this model can be found in section *Validation of Flux-Vehicle Interaction-Model Principles* (on page 103).

3.2.4 Response of Temperature and Thermal Emission to Incident Flux

3.2.4.a Temperature Variation

Without a source of incident energy, the object should cool by radiation, according to

$$C\frac{dT}{dt} = -\epsilon A\sigma T^4 \tag{3.6}$$

where C represents the heat capacity, ϵ the emissivity , α the albedo, A the emitting area, σ the Stefan-Boltzmann constant, and T the temperature.

α and ϵ are surface dependent values, and must be included as part of the plate model.

The analytic solution to this expression is

$$T(t) = \frac{T_0}{\sqrt[3]{1 + 3\left(\frac{\epsilon\sigma A}{C}\right)\ (T_0)^3\ t}} \tag{3.7}$$

with $T(0) = T_0$

The simulation is tested against this analytic result in section 5.1.5 on page 90.

There is no closed-form analytic solution for the case of an illuminated plate.

For the temperature response under illumination, we first consider the variation of the force due to thermal emission, described in the next subsection.

3.2.4.b Variation of Force due to Thermal Emission

The force due to emission can be calculated from the magnitude of the momentum carried away with the emission in the direction of that emission, which is given by

$$\frac{1}{c}\frac{dE}{dt},$$

Thus,

$$\vec{F} = \frac{2}{3}\frac{1}{c}\frac{dE}{dt}(-\hat{n}) \tag{3.8}$$

with c representing the speed of light, E the energy, and $\hat{n}$ the normal to the plate surface.

Due to symmetry, the total vector momentum emitted will be in the direction of the normal vector. Because the emission is not unidirectional, there will be components of the emission with opposing momentum vectors oriented in the plane of the surface; consequently, the magnitude of the *net* momentum emission will not be equal to the ratio of the *total* energy emission to the speed of light. Instead, we have to assume an emission profile; in this model, we assume a Lambertian profile for which the necessary factor to account for the difference is 2/3 (see subsection 3.2.3.a for description of the Lambertian profile).

The force on the plate is, of course, in the opposite direction to the emission.

Then, continuing the assumption that the plate is not illuminated, the force exerted on the plate by its emission is

$$F = \frac{2\epsilon\ \sigma\ A\ {T_0}^4}{3c}\left(1 + \frac{3\epsilon\ \sigma\ A}{C}\ (T_0)^3 t\right)^{-\frac{4}{3}} \tag{3.9}$$

It should be recognized that the values of t for the force and for the temperature are necessarily different due to the discretization of time in the simulation. The analytic expressions are valid for any particular time, t, and this is sufficient for the temperature. However, the force will be converted to acceleration, then integrated to provide velocity and position increments. Using the value of the force at either the beginning or end of a timestep, for the entire integration leads to errors; a more accurate representation would be to use some midpoint value. Instead, the force is

calculated at time t for the timestep from t to $t+\Delta t$. A closer approximation for the calculated force would be:

$$F(t \to t+\Delta t) \approx \frac{2\epsilon\ \sigma\ A\ {T_0}^4}{3c}\left(1+\frac{3\epsilon\ \sigma\ A}{C}\ (T_0)^3\left(t+\frac{\Delta t}{2}\right)\right)^{-\frac{4}{3}} \tag{3.10}$$

The simulation handles this problem by first integrating the temperature over the timestep using a Runge-Kutte 4th order algorithm:

The intermediate temperature deltas, $I_1, I_2, I_3, and I_4$ are calculated by

$I_1 = \frac{\Delta t}{C}\left(\Pi - \epsilon A\sigma {T_i}^4\right)$

$I_2 = \frac{\Delta t}{C}\left(\Pi - \epsilon A\sigma\left(T_i + I_1/2\right)^4\right)$

$I_3 = \frac{\Delta t}{C}\left(\Pi - \epsilon A\sigma\left(T_i + I_2/2\right)^4\right)$

$I_4 = \frac{\Delta t}{C}\left(\Pi - \epsilon A\sigma\left(T_i + I_3\right)^4\right)$

and averaged to give

$T_{i+1} = T_i + \left(\frac{I_1+2I_2+2I_3+I_4}{6}\right)$

where

- Δt is the timestep;
- σ is the Stefan-Boltzmann constant; and
- C is the heat capacity,
- Π is the power absorption rate from all sources,
- ϵ is the emissivity,
- A is the area, and
- T_i is the starting temperature of the plate.

Then, using an energy balance,

$C\Delta T = \left(\Pi - \xi\right)\Delta t$

the rate at which energy is emitted, ξ, is calculated based on the temperature change and the energy absorbed during that timestep.

This emitted energy is converted into a momentum, and multiplied by the 2/3 that results from the integration over the Lambertian emission profile to give a time-averaged force, valid over the entire timestep.

$$\langle\vec{F}\rangle = \frac{2}{3c}\left(\frac{C\Delta T}{\Delta t} - \Pi\right)\hat{n} \tag{3.11}$$

Consequently, the routine *thermal_integrator* calculates the value corresponding to the energy balance over the timestep that is in turn used in the calculation of the emission force by routine *radiation_pressure.*

3.2.5 Coefficient of Reflection

When using the single isotropic, isothermal, spherical, default surface, there is the option to use a Coefficient of Reflection rather than the combination of albedo and diffuse parameters. This section describes how that coefficient is used in the more general framework.

The concept of the Coefficient of Reflection is that the vehicle can be modeled as a single, perfectly absorbing, flat plate, with some *effective area, A'* that is always oriented perpendicular to the flux vector (i.e. its normal vector and the flux vector are anti-parallel). To obtain a constant area requires that the vehicle be modeled as a sphere, so that its orientation cannot affect its projected area.

The physical dimensions of the vehicle can then be modeled with a single parameter, the radius, R. The cross-sectional area that the vehicle projects perpendicular to the flux vector is then πR^2.

In general, $A' \neq \pi R^2$. The difference is due to the difference between the reflective characteristics of the real surface and the assumed perfect absorption of the simple surface. That difference is modeled as the Coefficient of Reflection, C_R:

$$A' = C_R \pi R^2$$

Just as with the regular surface, developed in the Surface Model, the simple surface can be considered to interact with the radiation field through absorption, specular reflection, and diffuse reflection.

3.2.5.a Absorption

The projected area absorbs everything that is incident upon it; the projected area and effective area are identical. $C_R = 1.0$

3.2.5.b Specular Reflection

Figure 3.9 shows a representation of a sphere, illuminated from the right, with three light rays added showing the paths due to specular reflection.

One ray is reflected stright back, one is reflected at an angle close to perpendicular to its incident path, and another is deflected very little from its incident path. For a photon hitting the surface at a point that subtends some angle θ from the flux vector, the reflected ray will be at angle 2θ. The momentum change of the photon (defining the x-axis as being anti-parallel to the flux vector) is then

$$\Delta\vec{p} = p(\hat{i} + \cos 2\theta \hat{i} + \sin 2\theta \hat{j})$$

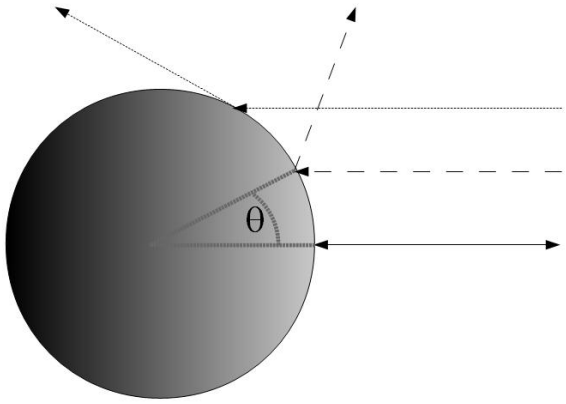


Figure 3.9: Light Rays Incident on a Specular Spherical Surface

where p is the magnitude of the momentum of the photon.

Assuming that the sphere is uniformly illuminated (at least across the illuminated side) leads to the argument that, by symmetry, the forces off the x-axis will cancel to zero.

To obtain the force along the x-axis, consider dividing the sphere into concentric rings, centered about $\theta = 0$. For a small range, $d\theta$, there will be a ring of elemental area $(2\pi R \sin\theta)(Rd\theta)$. This elemental area will project, perpendicular to the flux vector, an area $(2\pi R^2 \sin\theta d\theta)\cos\theta$.

Therefore, the fraction of the projected disk resulting from the sphere at some angle θ is

$$\delta(\theta) = 2\sin\theta\cos\theta d\theta = \sin 2\theta d\theta$$

This is the fraction of the total flux that is incident at any particular angle θ.

The force due to specular reflection is then

$$\vec{F} = \vec{\phi}\pi R^2 \int_0^{\pi/2} (1 + \cos 2\theta) sin2\theta d\theta$$

where $\vec{\phi}$ is the momentum flux vector ($Ncdotm^{-2}$).

The somewhat surprising result of this integration is that

$$\vec{F} = -\vec{\phi}\pi R^2$$

i.e., the specular sphere behaves exactly the same as a perfectly absorbing sphere, and therefore has $C_R = 1.0$.

While this may not be expected, it is logical with further consideration. The photons incident very close to the edge of the sphere will continue almost unaffected, resulting in minimal force on the sphere. Conversely, the photons incident near the center will be reflected back on the same path,

resulting in twice the force that they would produce if absorbed. Those incident with $\theta = 45^o$ will exert exactly the same force (along the x-axis) on the sphere as they would exert if absorbed. Therefore, the photons hitting inside 45 degrees exert additional force, those outside 45 degrees exert less force, with nice symmetry. Integrating the projected area for $0 \leqslant \theta < 45^o$ and for $45 < \theta \leqslant 90^o$ yields the same value; the deficit resulting from those photons outside the 45 degree circle balances with the excess from those inside the circle.

3.2.5.c Diffuse Reflection

The diffuse reflection is handled using a Lambertian profile and the same mathematical methods as in the previous section.

$$\vec{F} = \vec{\phi}\pi R^2 \int_0^{\pi/2} \left(1 + \frac{2}{3}\cos\theta\right) sin2\theta d\theta = \frac{13}{9}\vec{\phi}\pi R^2$$

i.e., the diffuse sphere produces a larger force than would a specular or absorbing sphere of the same size, and has $C_R = \frac{13}{9}$.

3.2.5.d Composite Surface

Combining the three processes together yields

$$C_R = (1-\alpha) + \alpha(1-\delta) + \frac{13}{9}\alpha\delta = 1 + \frac{4}{9}\alpha\delta \in [1.0, 1.444] \tag{3.12}$$

where α is the albedo and δ is the diffuse parameter.

Note that there is a one-to-many relation between the values of C_R and the values of α, δ.

The code continues to use α and δ; when a user specifies a coefficient, it is converted into

$\alpha = \frac{9}{4}(C_R - 1)$ and $\delta = 1$

3.3 Detailed Design

This section is divided into 3 parts:

- A process flow-through description, or process architecture, of the sequence in which the various functions are called, and the interaction between the objects.
- An alphabetized list of the objects that comprise the Radiation Pressure Model with reference to the parent class where appropriate, and description of the functions contained within the object.
- A description of the external data files used by the objects.

Further, the *Reference Manual* [1] contains a structural overview of the Radiation Pressure Model.

3.3.1 Process Architecture

This section describes the flow from function to function. The operation of each function is described under its object in the section *Functional Design* (on page 52).

3.3.1.a Flow charts of the Radiation Pressure Model architecture

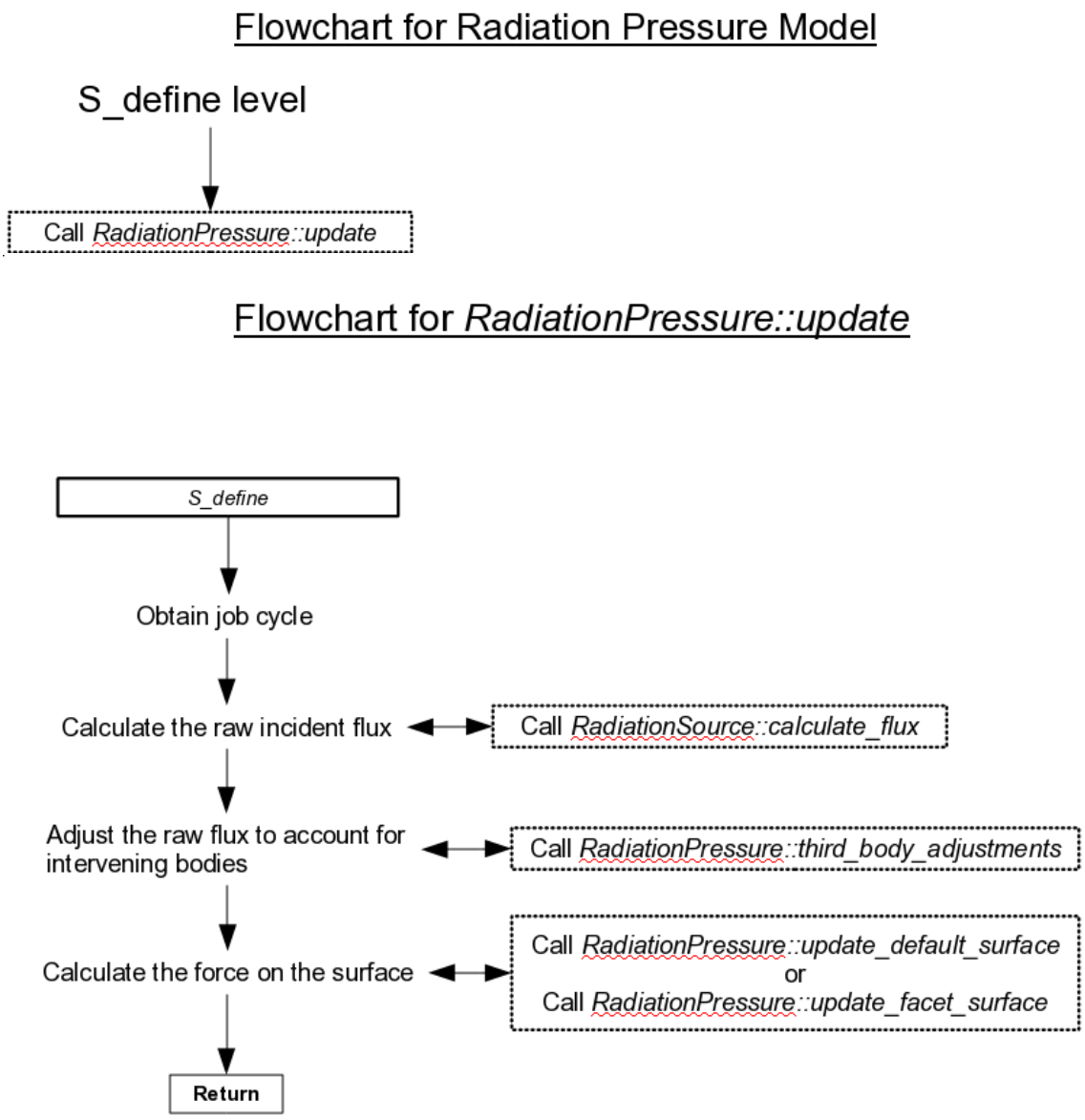


Figure 3.10: Flowcharts for Radiation Pressure Model

Links:
RadiationSource::calculate_flux (on page 40)
RadiationPressure::third_body_adjustments (on the next page)
RadiationPressure::update_default_surface (on page 38)
RadiationPressure::update_facet_surface (on page 39)

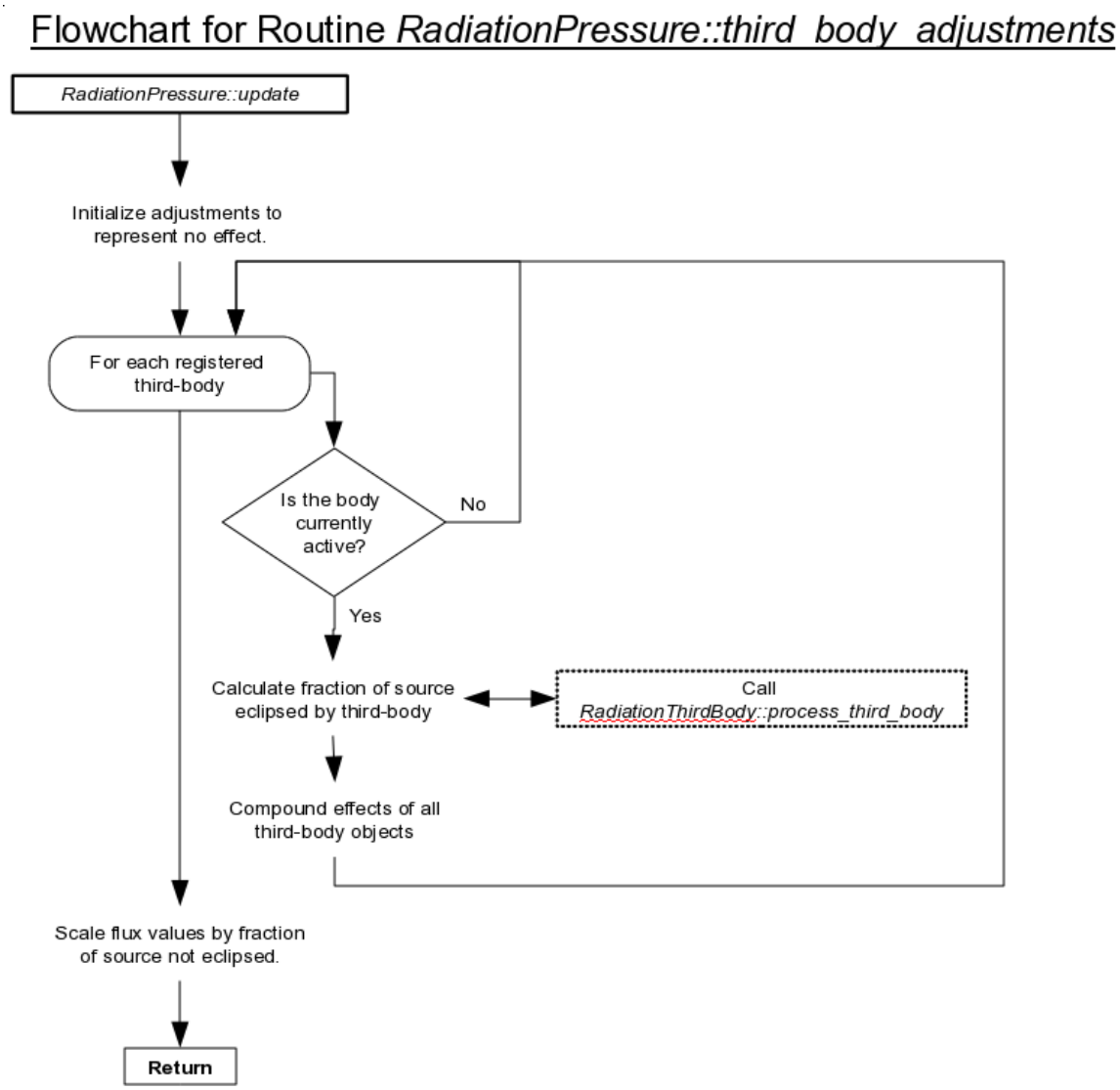


Figure 3.11: Flowcharts for Radiation Pressure Model (continued)

Links:
RadiationPressure::update (on the facing page)
RadiationThirdBody::process_third_body (on page 49)

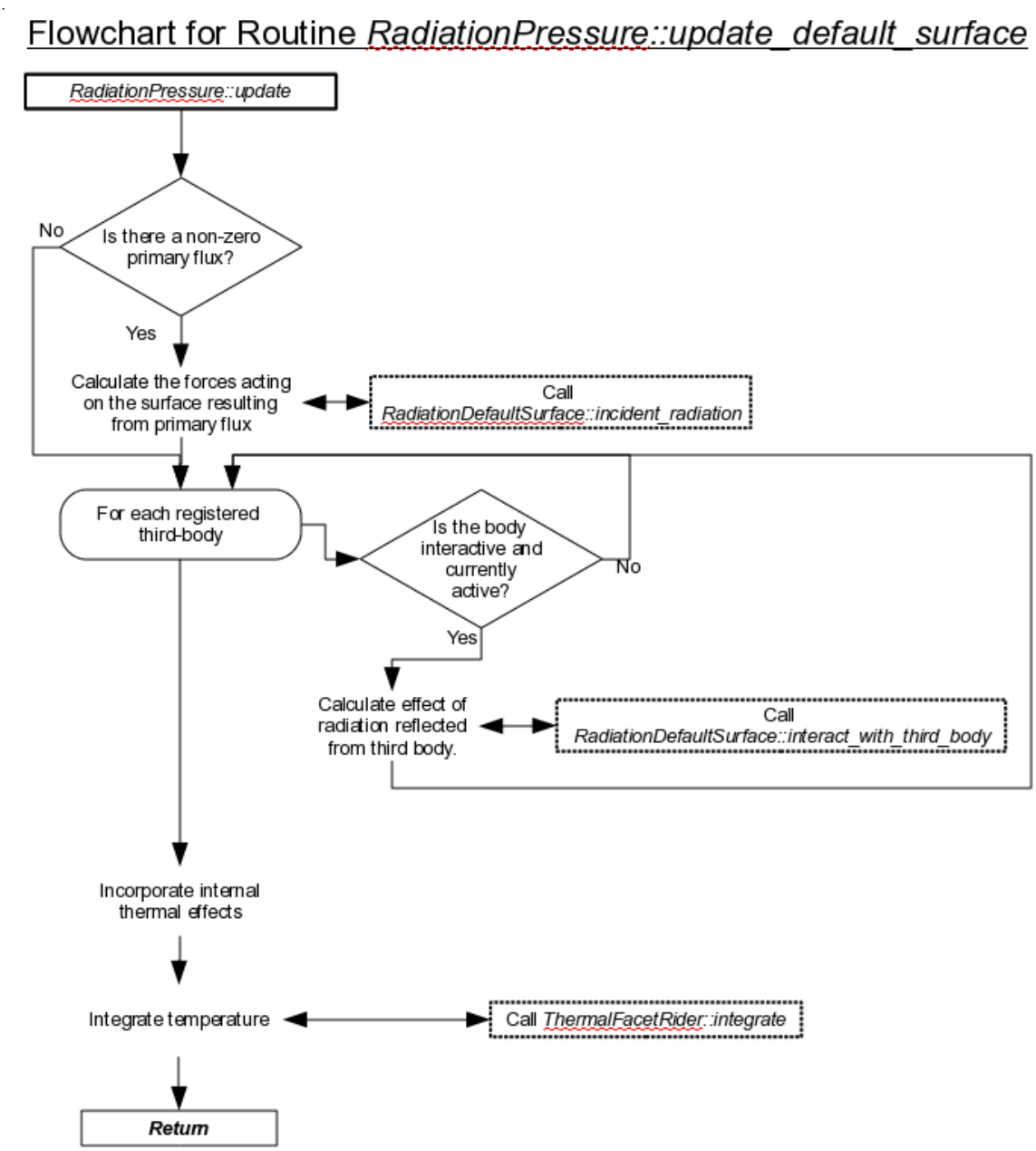


Figure 3.12: Flowcharts for class RadiationPressure (continued)

Links:
RadiationPressure::update (on page 36)
RadiationDefaultSurface::incident_radiation (on page 44)
RadiationDefaultSurface::interact_with_third_body (on page 45)
ThermalFacetRider::integrate (on page 51)

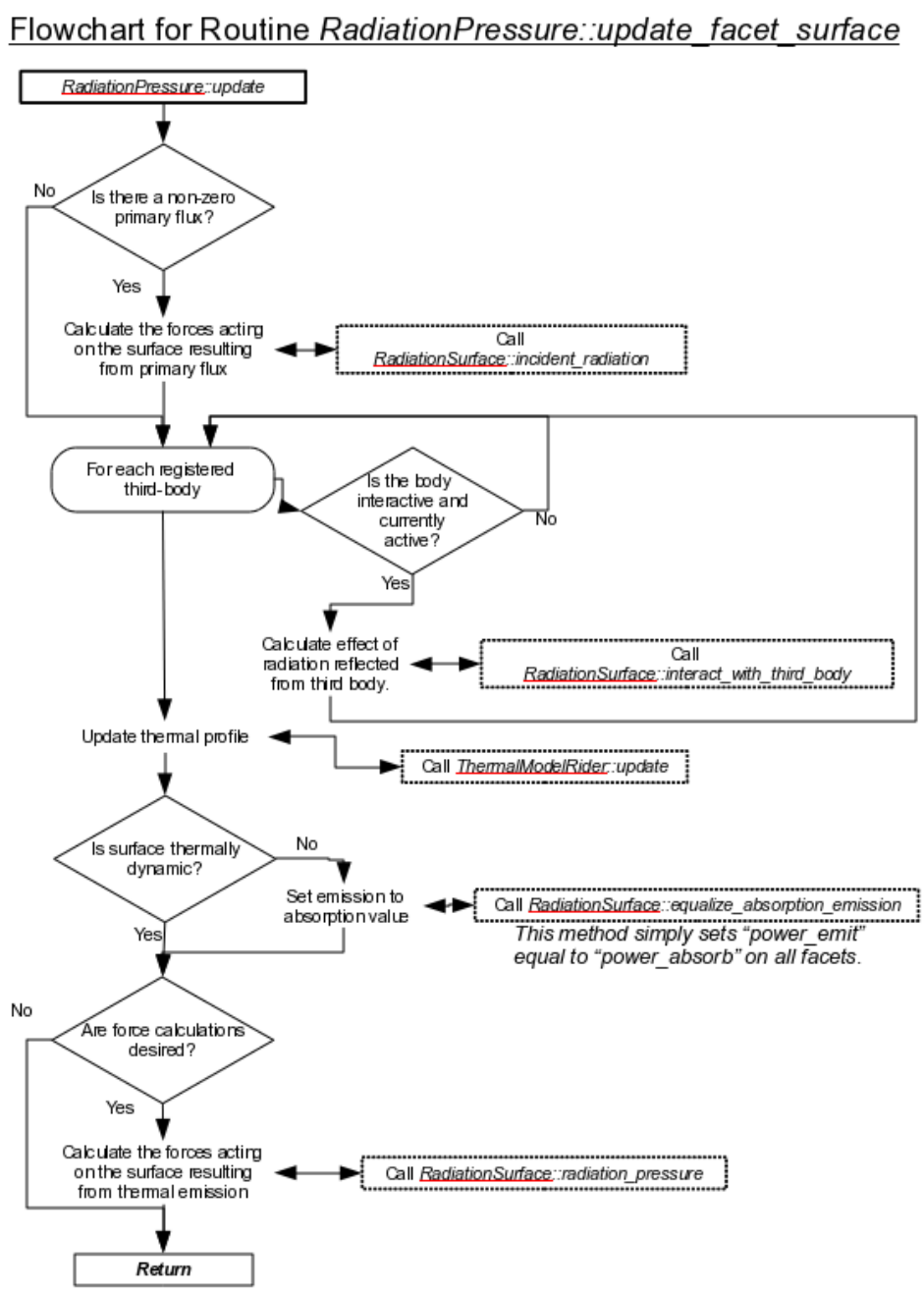


Figure 3.13: Flowcharts for class RadiationPressure (continued)

Links:
RadiationPressure::update (on page 36)
RadiationSurface::incident_radiation (on page 41)
RadiationSurface::interact_with_third_body (on page 41)
ThermalModelRider::update (on page 50)
RadiationSurface::radiation_pressure (on page 42)

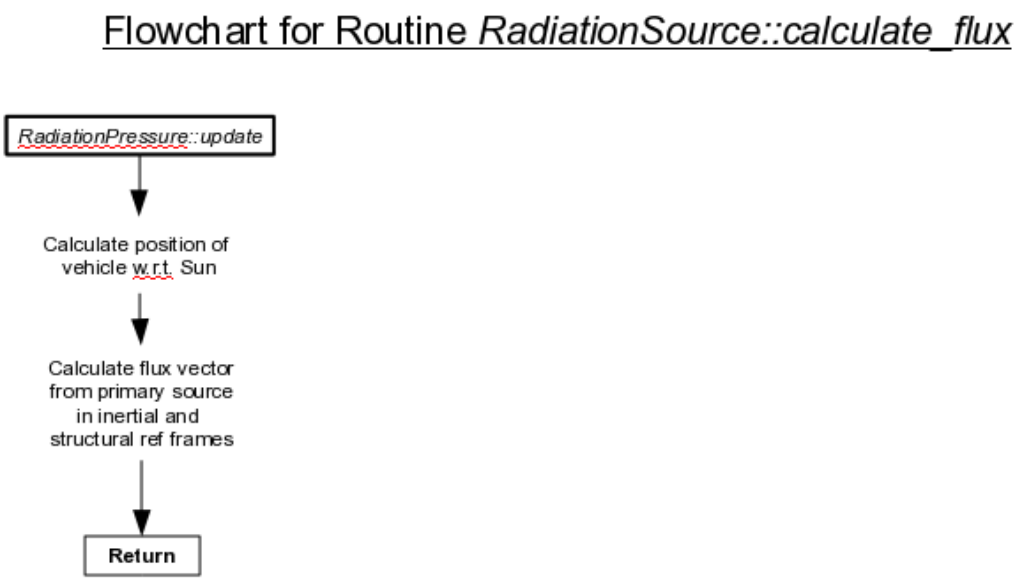


Figure 3.14: Flowchart for class RadiationSource

Links:
RadiationPressure::update (on page 36)

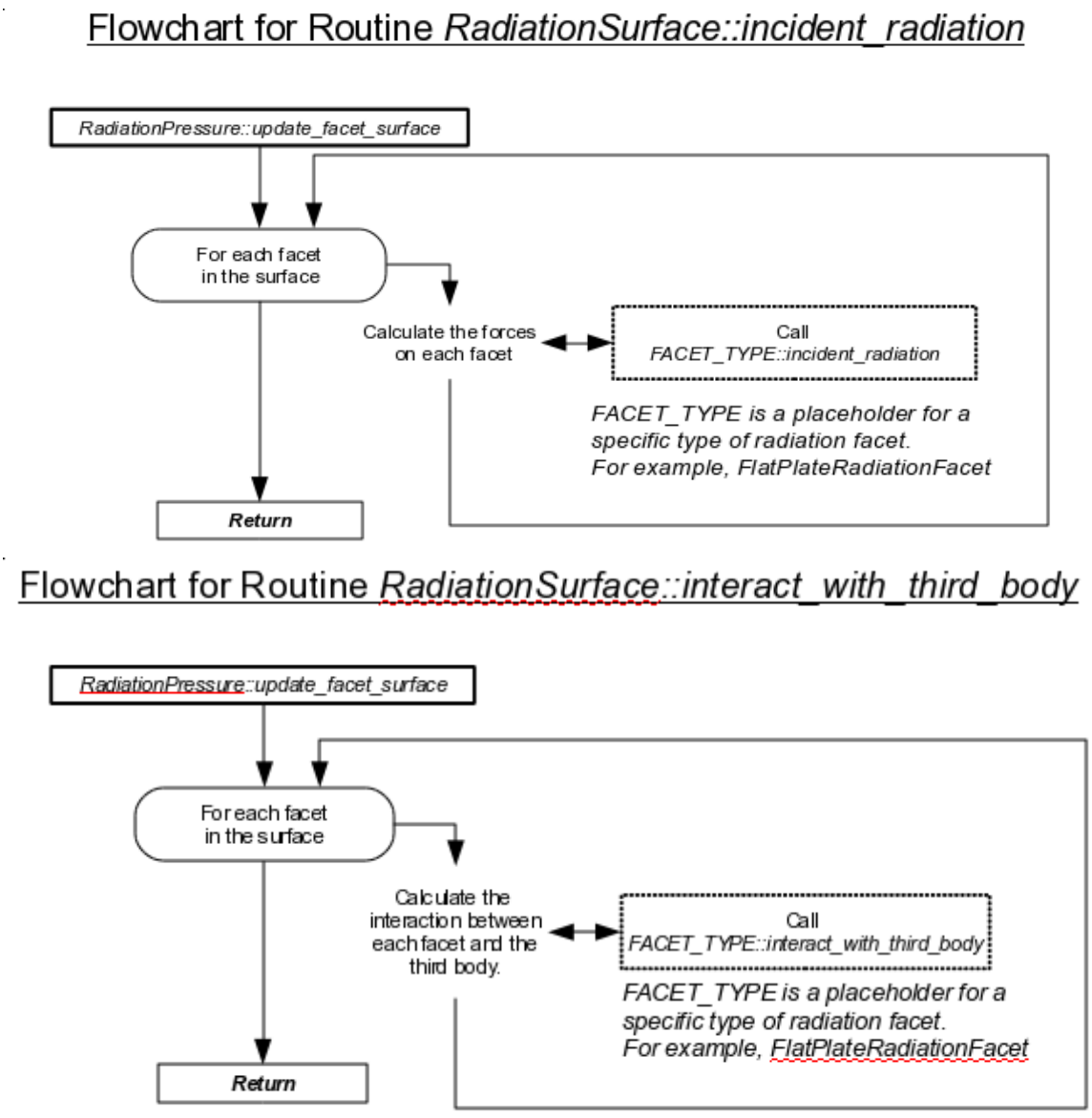


Figure 3.15: Flowcharts for class RadiationSurface

Links:
RadiationPressure::update_facet_surface (on page 39)
FlatPlateRadiationFacet::incident_radiation (on page 46)
FlatPlateRadiationFacet::interact_with_third_body (on page 47)

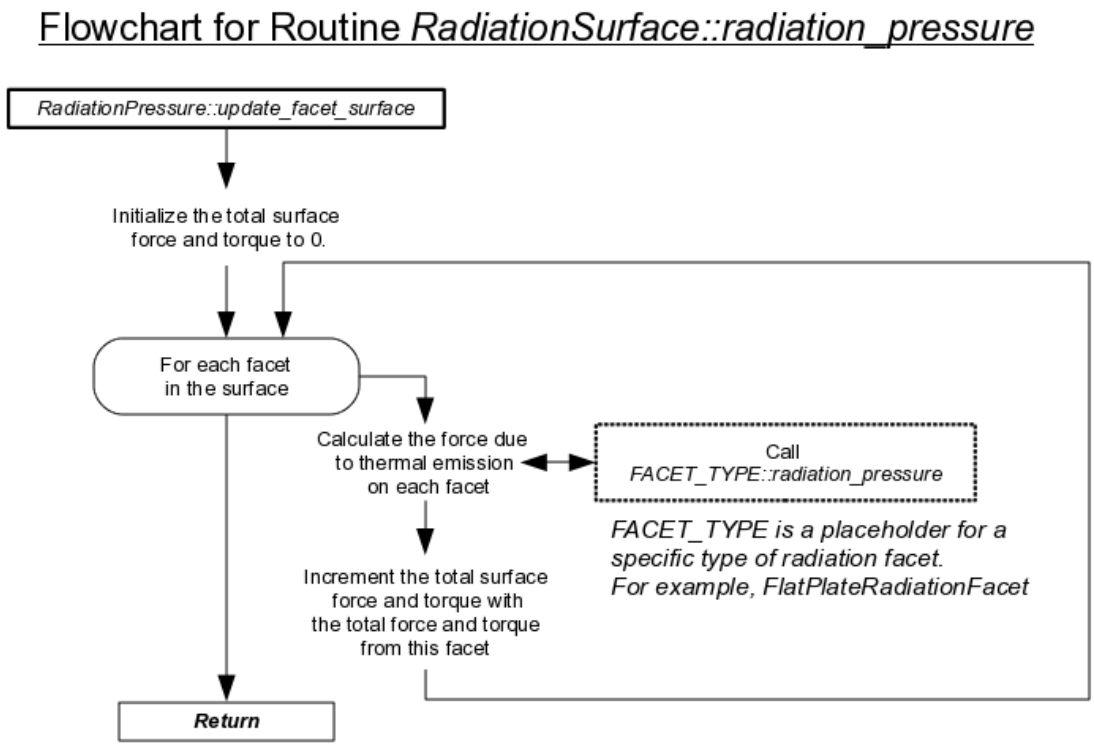


Figure 3.16: Flowcharts for class RadiationSurface (continued)

Links:
RadiationPressure::update_facet_surface (on page 39)
FlatPlateRadiationFacet::radiation_pressure (on page 46)

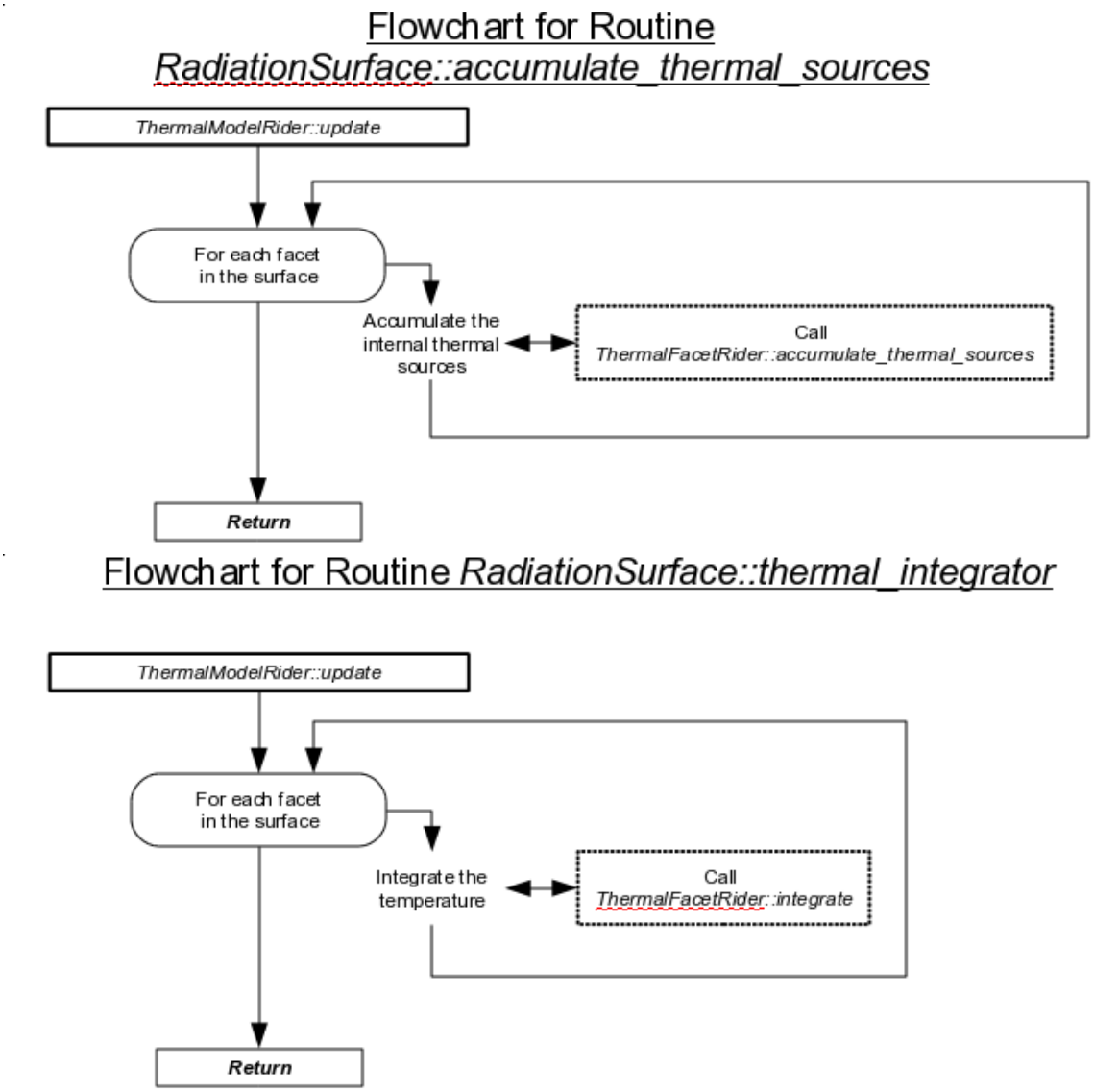


Figure 3.17: Flowcharts for class RadiationSurface (continued)

Links:
ThermalModelRider::update (on page 36)
ThermalFacetRider::accumulate_thermal_sources (on page 51)
ThermalFacetRider::integrate (on page 51)

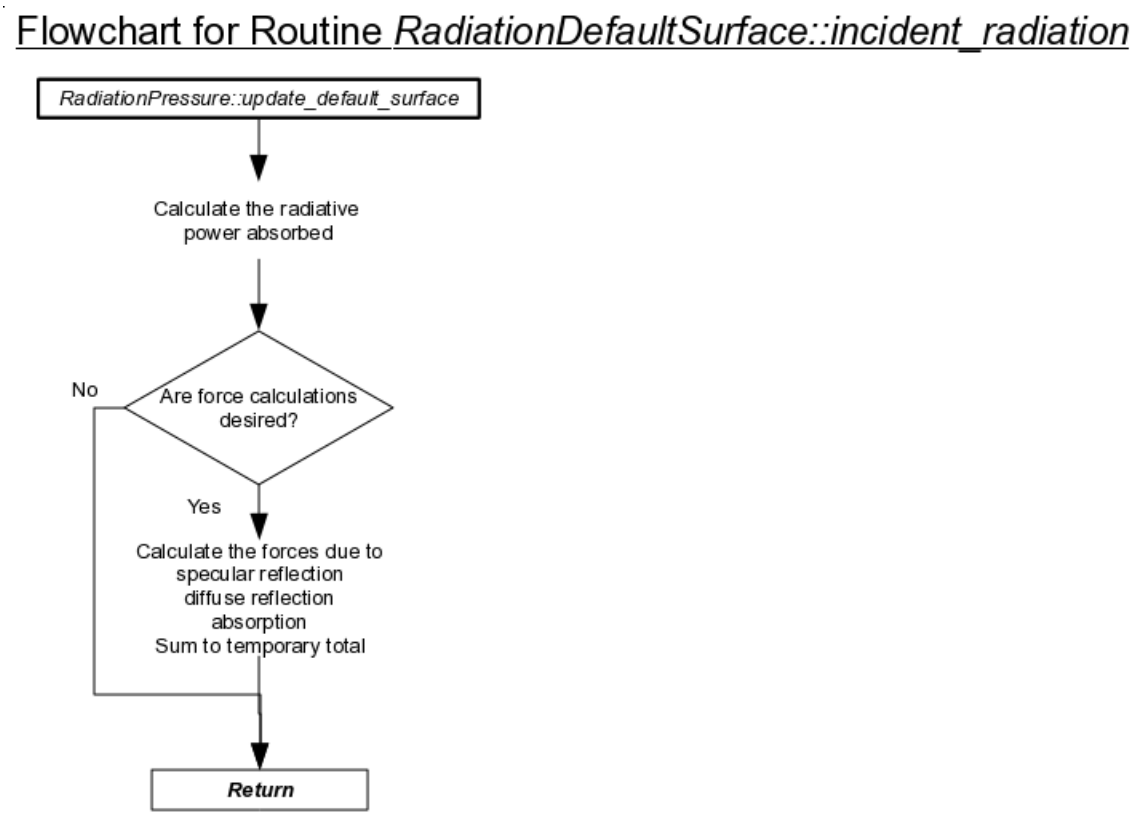


Figure 3.18: Flowchart for class RadiationDefaultSurface

Links:
RadiationPressure::update_default_surface (on page 38)

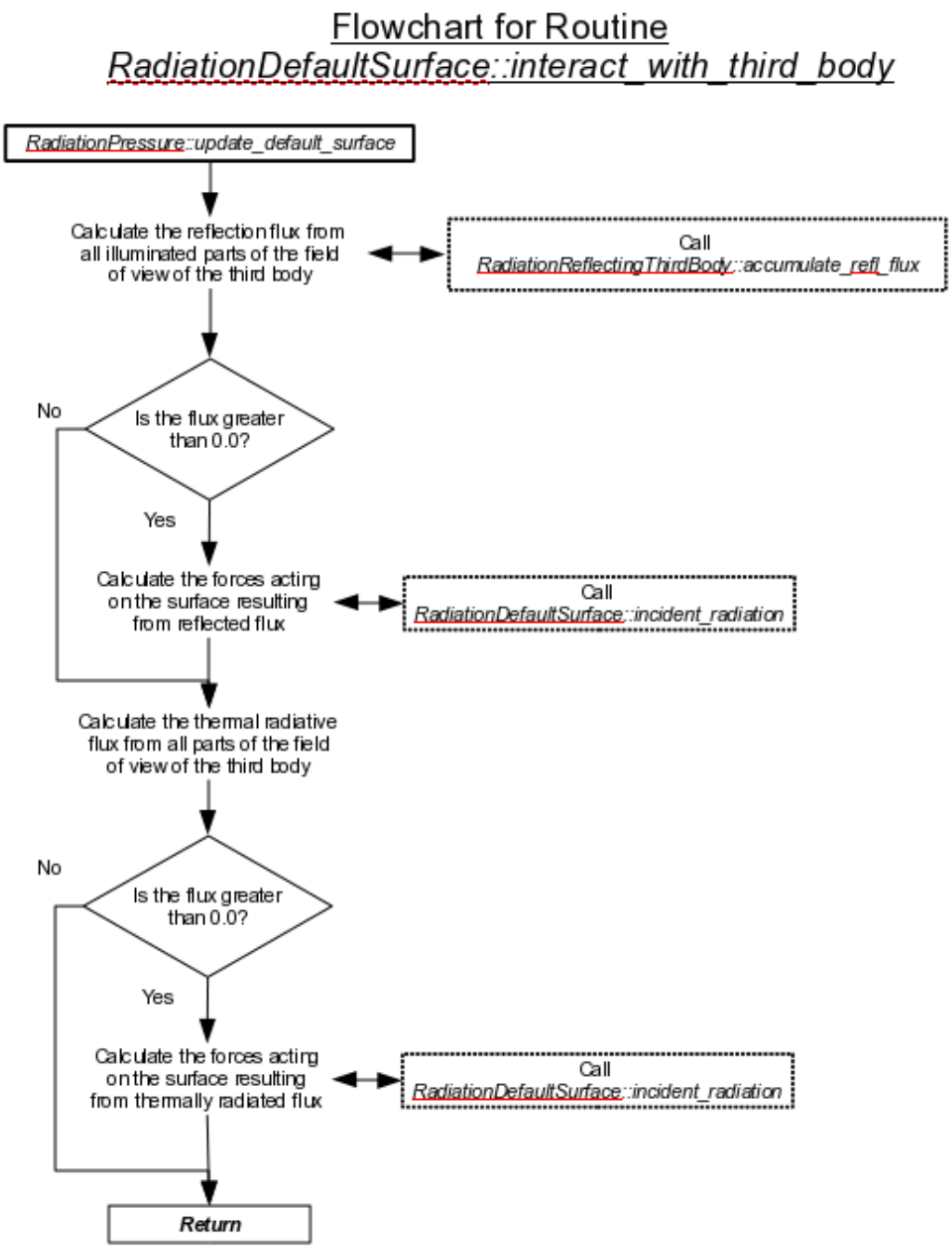
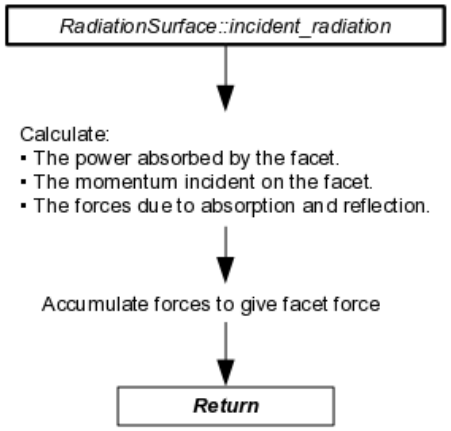


Figure 3.19: Flowchart for class RadiationDefaultSurface (continued)

Links:
RadiationPressure::update_default_surface (on page 38)
RadiationReflectingThirdBody::accumulate_refl_flux(not implemented)
RadiationDefaultSurface::incident_radiation (on the facing page)
RadiationReflectingThirdBody::accumulate_rad_flux(not implemented)

Flowchart for Routine *FlatPlateRadiationFacet::incident_radiation*



Flowchart for Routine *FlatPlateRadiationFacet::radiation_pressure*

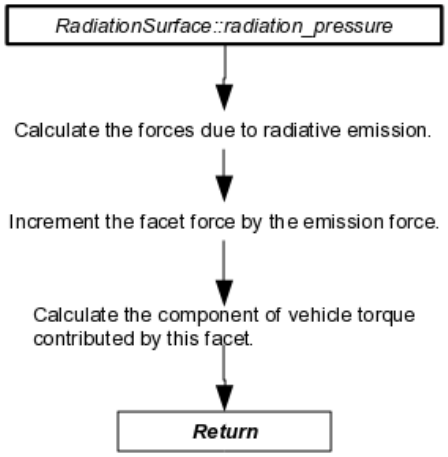


Figure 3.20: Flowcharts for class FlatPlateRadiationFacet

Links:
RadiationSurface::incident_radiation (on page 41)
RadiationSurface::radiation_pressure (on page 42)

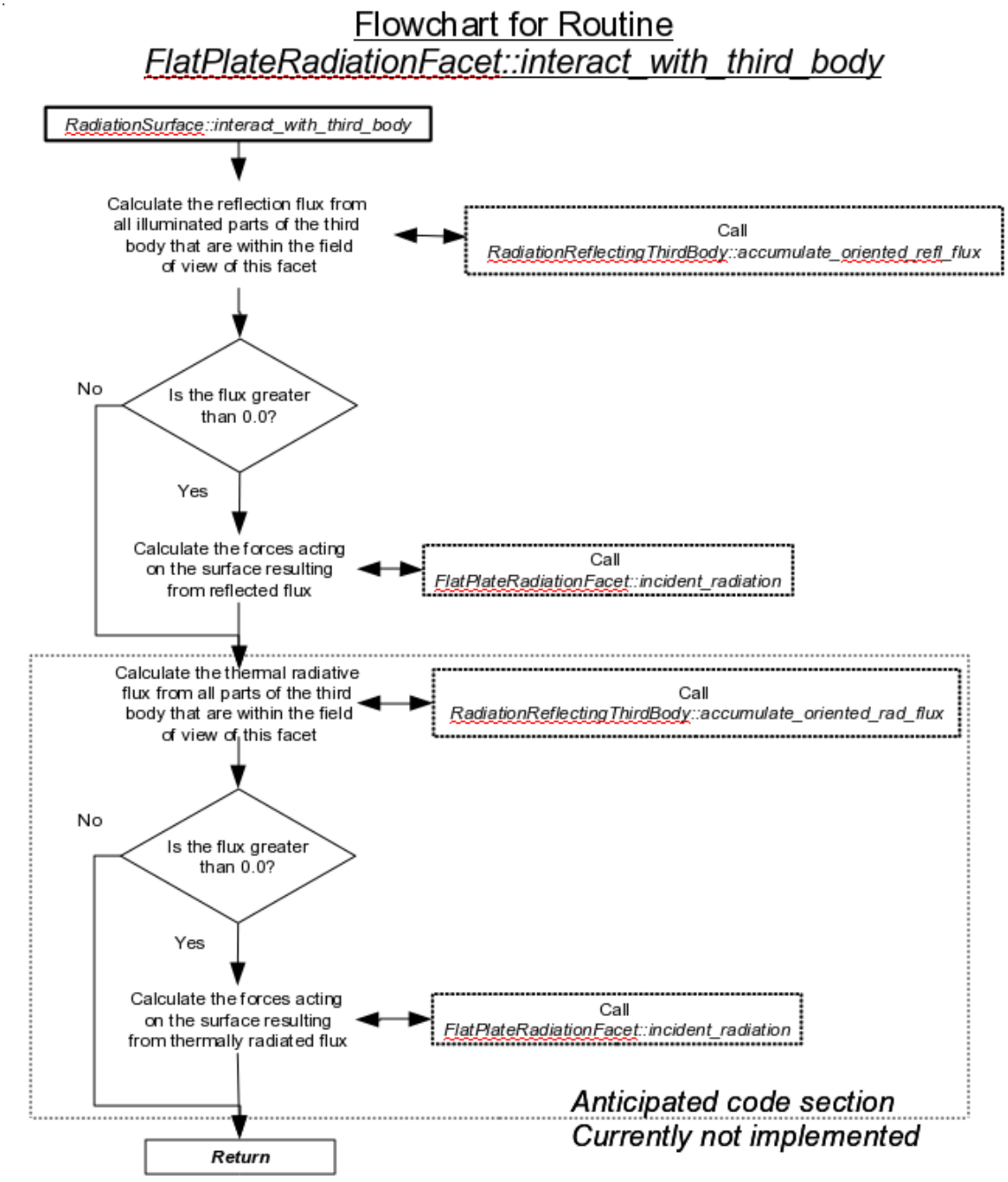


Figure 3.21: Flowcharts for class FlatPlateRadiationFacet (continued)

Links:
RadiationSurface::interact_with_third_body (on page 41)
RadiationReflectingThirdBody::accumulate_refl_flux(not implemented)
FlatPlateRadiationFacet::incident_radiation (on the facing page)
RadiationReflectingThirdBody::accumulate_rad_flux(not implemented)

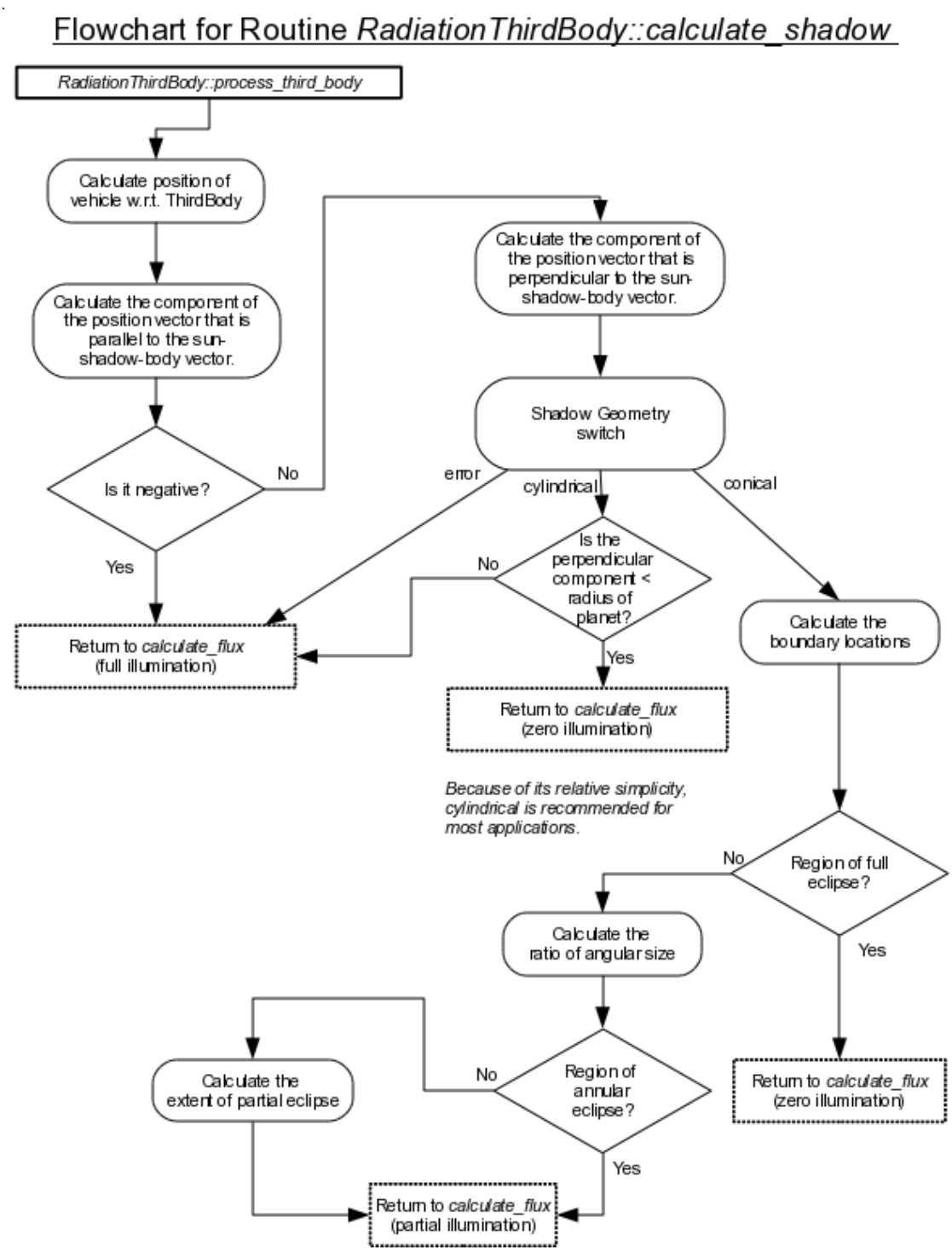


Figure 3.22: Flowchart for class RadiationThirdBody

Links:
RadiationSource::calculate_flux (on page 40)

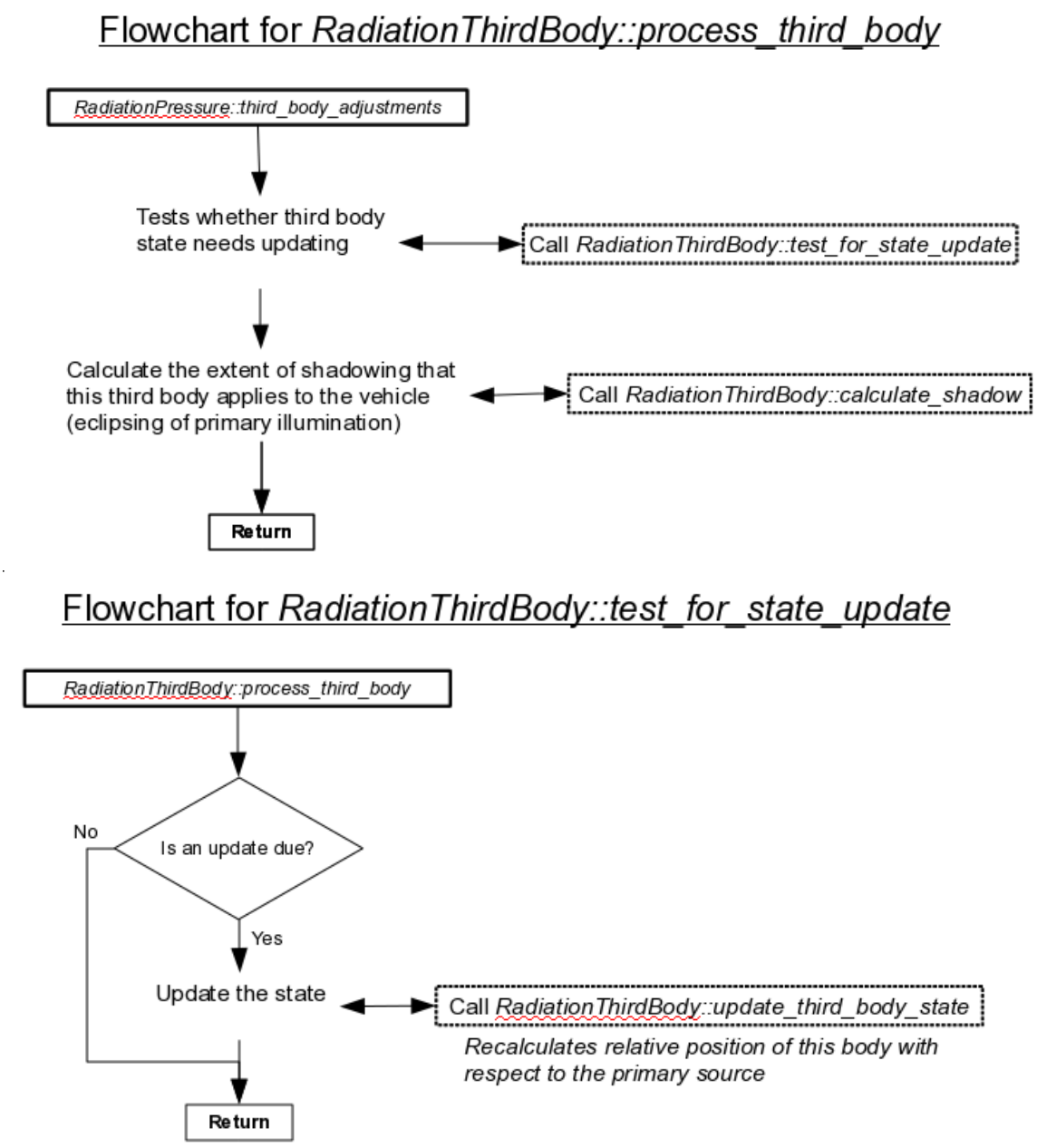


Figure 3.23: Flowchart for class RadiationThirdBody (continued)

Links:
RadiationPressure::third_body_adjustments (on page 37)
RadiationThirdBody::test_for_state_update (on this page)
RadiationThirdBody::calculate_shadow (on the facing page)
RadiationThirdBody::process_third_body (on this page)

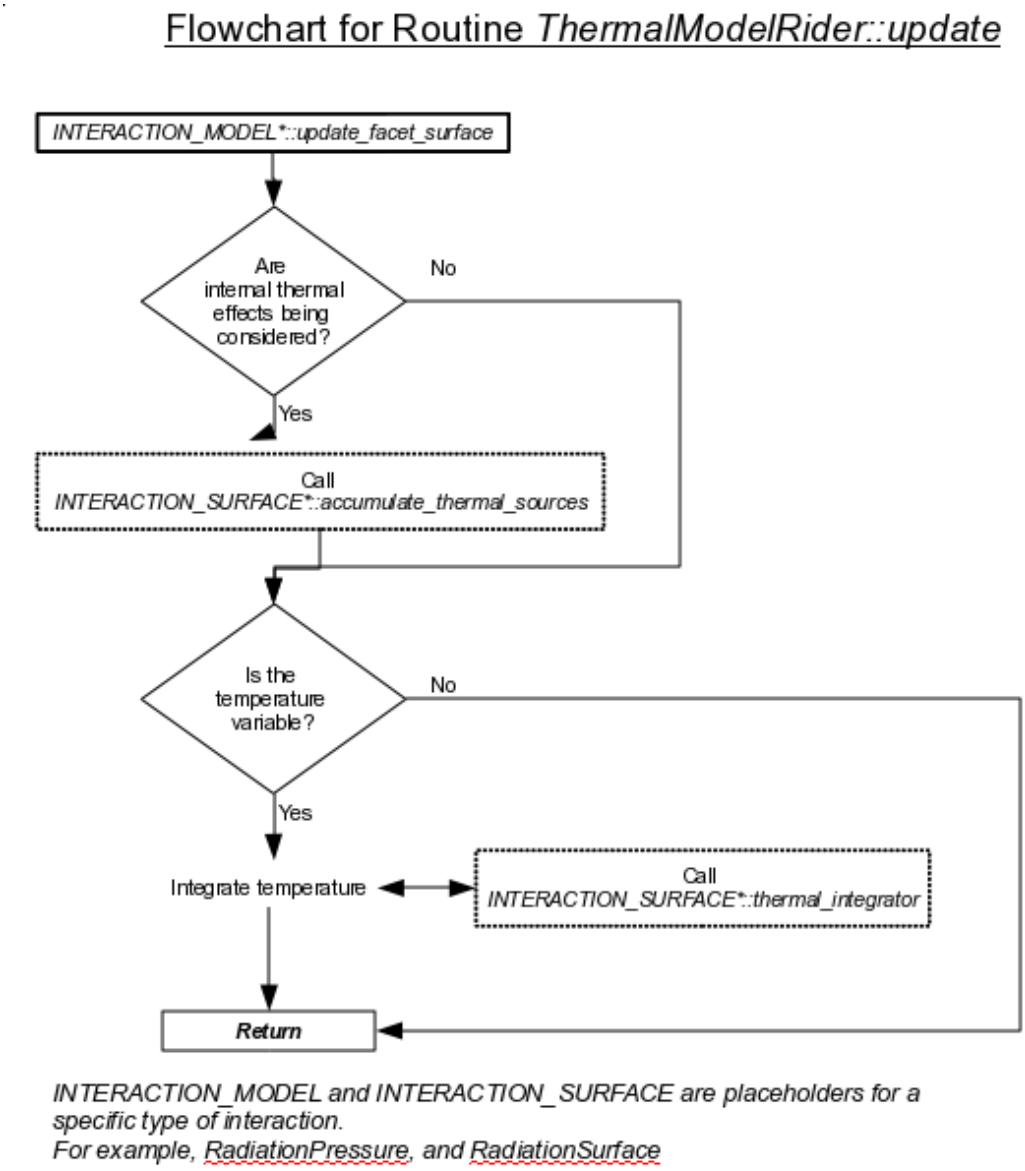


Figure 3.24: Flowchart for class ThermalModelRider

Links:
RadiationPressure::update_facet_surface (on page 39)
RadiationSurface::accumulate_thermal_sources (on page 43)
RadiationSurface::thermal_integrator (on page 43)

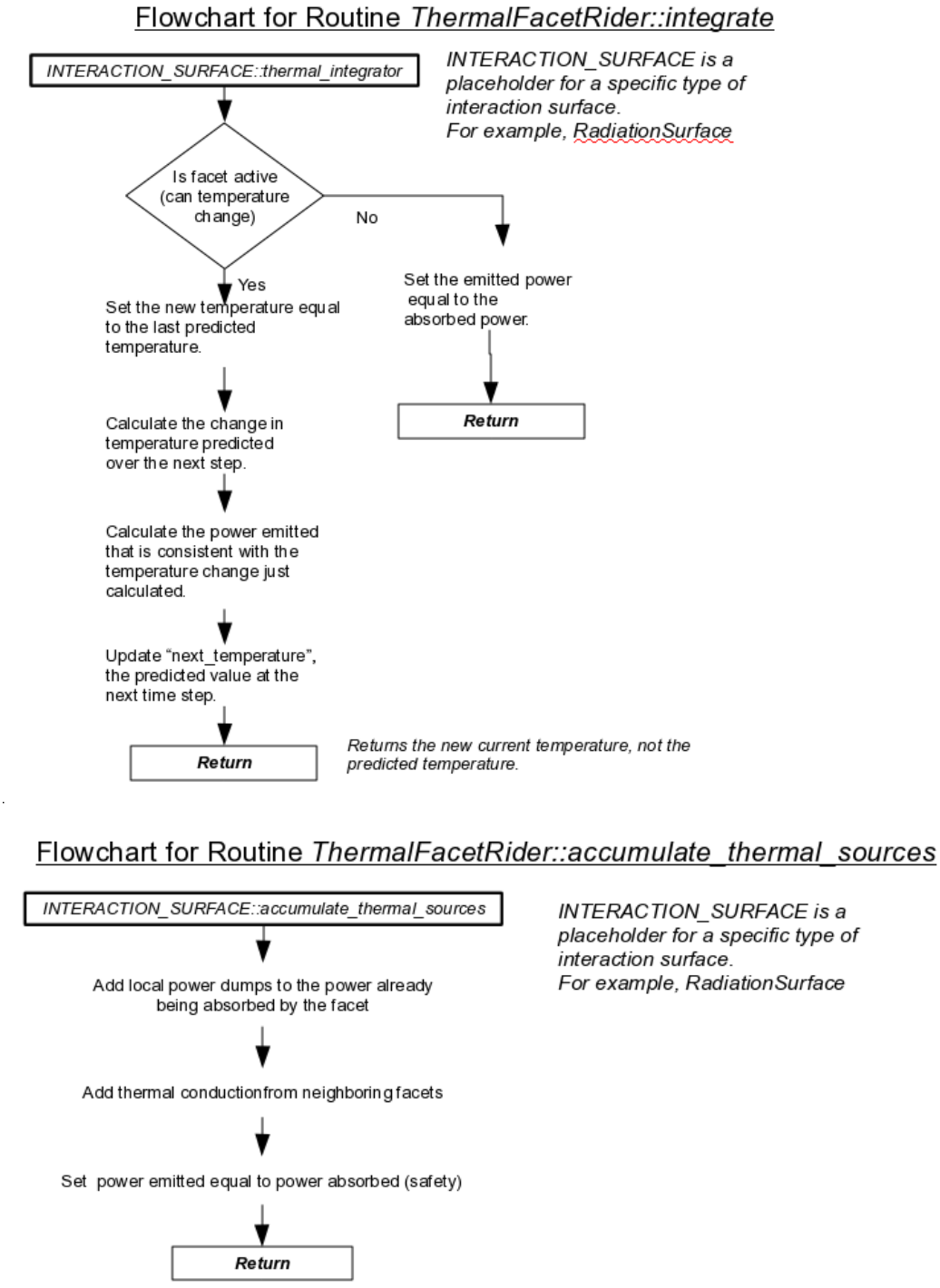


Figure 3.25: Flowcharts for class ThermalFacetRider

Links:
RadiationSurface::thermal_integrator (on page 43)
RadiationSurface::accumulate_thermal_sources (on page 43)

3.3.2 Functional Design

See the Reference Manual [1] for a summary of member data and member methods for all classes. This section describes the functional operation of the methods in each class.

Methods are ordered alphabetically within the class in which they reside; classes are ordered alphabetically. Methods that are inherited from a parent class are described in that parent class.

Throughout this description, some words are used in both a generic sense, and as particular instances of objects within the code, e.g. "facet". Generally, an attempt has been made to capitalize those instances that imply a specific case of an object, and leave the generic term in lower case. As a pedantic example: a surface comprises a number of facets, and each Interaction Surface comprises a number of Interaction Facets, each of which is associated with a Facet; each Facet contains the details of each facet.

1. **Class: FlatPlateRadiationFacet** **Inheritance:** RadiationFacet

   (a) **incident_radiation**

   Calculates the energy incident on the facet (*areaxflux_e*), and from that the rate at which energy is being absorbed.

   If the model is being called only to manage the thermal aspect, it is complete. Otherwise, it continues to calculate the momentum incident on the facet (*areaxflux)*, and from that the forces due to absorption, diffuse reflection, and specular reflection. The values calculated are added to the preexisting values (which are initialized to zero at every time-step) to allow this same method to be used for multiple sources.

   (b) **initialize**

   Assigns a pointer from the thermal facet rider to here, calls the thermal facet rider initialization routine, and calculates the vector from the center of rotation (center of mass, identified as *center_grav* in the model) to the center of pressure of the facet.

   (c) **define_facet**

   Copies the normal vector and the facet location into the appropriate variables.

   (d) **interact_with_third_body**

   This method is used to add in the effects of reflective flux coming from another body (referred to as a third-body). It is called from the *RadiationSurface::interact_with_third_body* method, which in turn is called from the *RadiationPressure::update_facet_surface* method; from there, it receives a pointer to a RadiationThirdBody. Since the call is restricted to when the RadiationThirdBody is flagged as *is_interactive*, this pointer must actually be to a RadiationReflectingThirdBody (a derivative class of RadiationThirdBody). Note also that this method is called *for each plate* for each instance of an *interactive* RadiationThirdBody.

   The RadiationReflectingThirdBody is not released as a supported component of JEOD, but the developed (currently undergoing testing) concept is for the third-body to keep track of what elements of its surface are both illuminated and visible to the vehicle. This method makes a call to the third-body *accumulate_oriented_refl_flux* method to accumulate the incident flux from that part of its illuminated visible (i.e. visible to the vehicle) surface that would be visible to this specific facet. The determination of which

parts of the third-body surface are visible to this facet (as opposed to the vehicle in general) is made by considering the orientation of the facet, identified by its *normal*. This new value for flux is then passed into the *incident_radiation* method, where its effect is added to that from the primary source flux.

A secondary aspect of this method is to accumulate the thermal radiative flux from the third-body. This method is still undergoing development. Conceptually, it is more simple than the reflective flux, since the entire body is radiating, so there is no "illuminated" field to evaluate, just the field of view from the facet. However, the wavelength-profile of that radiation is substantially different than that for the visible radiation previously considered for the primary and reflected fluxes. Consequently, most materials are likely to have a different albedo for this infrared radiation, and the albedo plays a very large role in the *incident_radiation* method. However, the primary purpose of this calculation is to model the thermal contribution of this radiation because the flux is typically very weak, so typically has only a negligible dynamic effect. Indeed, in order for the dynamic effect to be adequate to cause a significant shift in the trajectory, the vehicle would have to be sufficiently close to the body as to make aerodynamic forces so dominant that the dynamic effect from radiative flux would be lost in the noise of the aerodynamics. Consequently, a temporary fix to this problem is provided by adjusting the flux to accommodate the change in albedo when calculating the absorbed power, and turning off the force calculations (the force calculations use albedo in a way that is inconsistent with the manipulation that permits its use for absorption calculations).

(e) **radiation_pressure**

This method is called after all sources have been considered. It uses the recorded value of emitted power to derive the force due to thermal emission. The four forces – absorption; specular and diffuse reflection; and emission – are summed together to provide the total facet force, *force*. Torque is calculated by considering the total force for the facet.

(f) **initialize_runtime_values**

This method simply zeros out the absorbed power, and the forces due to absorption and reflection at the start of each time step. This allows for the incremental calculation of these elements.

2. **Class: FlatPlateRadiationFactory** **Inheritance:** InteractionFacetFactory (Surface Model)

(a) **create_facet**

This method takes the defined Facet, and generates a Radiation Facet from it. Each Facet has a set of parameters (*FacetParams*) associated with it. First, a check is made that the set of parameters is actually a set of Radiation Parameters, and that the Facet is actually a Flat Plate Thermal Facet (i.e., a flat plate with a Thermal Rider). If either test fails, the code cannot proceed.

Each Flat Plate Radiation Facet can see the base-class (Facet) data; that pointer is set. Then two initialization routines are called, one to initialize the radiation aspects of the facet, and one to initialize the flat plate aspects.

(b) **is_correct_factory**

This method simply returns a true or false depending on whether the factory is being passed an appropriate facet type.

3. **Class: RadiationDefaultSurface** **Inheritance:** Base-class

   (a) **initialize**

   Runs several tests to ensure that the initialization data is self-consistent and valid, and calls the thermal facet rider initialization routine.

   (b) **incident_radiation**

   Calculates the energy incident on the facet (*areaxflux_e*), and from that the rate at which energy is being absorbed.

   If the model is being called only to manage the thermal aspect, it is complete. Otherwise, it continues to calculate the momentum incident on the facet (*areaxflux*), and from that the forces due to absorption, diffuse reflection, and specular reflection. The three forces are accumulated into one value, and incremented to the value *force* (this is initialized to zero at each timestep; incrementing allows the same method to be used for multiple sources).

   (c) **interact_with_third_body**

   This method follows the same outline as that presented for the *FlatPlateRadiationFacet* (on page 52), with the exceptions that:

   - this method is called from the *RadiationPressure::update_default_surface* method, not the *RadiationPressure::update_facet_surface* method.
   - The calls are to *accumulate_refl_flux* and *accumulate_rad_flux*; these are slightly different since the default surface has no orientation dependency.

   Otherwise, the issues and restrictons are identical.

4. **Class: RadiationFacet** **Inheritance:** InteractionFacet (Surface Model)

   (a) **initialize**

   Pure virtual function, must be defined in subclasses

   (b) **radiation_pressure**

   Pure virtual function, must be defined in subclasses

   (c) **incident_radiation**

   Pure virtual function, must be defined in subclasses

   (d) **define_facet_core**

   Extracts the data from the facet parameter set, and assigns it to the a ppropriate variables in the facet data structure. This includes data residing in the Thermal Facet Rider.

5. **Class: RadiationMessages** **Inheritance:** Base-class

   This is the message handler for the Radiation Pressure Model. It has no methods.

6. **Class: RadiationParams** **Inheritance:** FacetParams (Surface Model)

   This class simply contains member data used to describe the radiative properties of each facet in the Radiation Surface. It has no methods.

7. **Class: RadiationPressure** **Inheritance:** Base-class

(a) **initialize**

There are two instances of the initialize function, differentiated by the arguments they take. One is used when the desired surface is the simple default surface, and the other when the desired surface is a full Surface Model.

Both methods behave essentially the same:

i. The dynamics manager pointer is set.

ii. To provide compatibility with older versions (where the RadiationThirdBody objects were recorded in the RadiationSource object), any ThirdBody objects stored in the RadiationSource object are copied over to the RadiationPressure object, along with the shadow geometry (which used to be common to all RadiationThirdBody objects).

iii. The environment is initialized with a call to initialize_environment.

iv. The surface is initialized, with a call to the appropriate method.

(b) **initialize_environment**

First, the primary source is initialized, then assigned to all RadiationThirdBody objects. Each RadiationThirdBody object is then initialized, which includes instructions to the Dynamics Manager to subscribe to the appropriate reference frame to ensure that the frame is updated as needed.

(c) **update**

This is the top-level method in the Radiation Pressure Model.

The time-step is obtained from the simulation engine suing the interface method *get_job_cycle* and the Time Model's *scale_factor*.

Next, a call is made to calculate the incident flux from the primary source; this is immediately followed up with a call to identify the modifications to that flux that the RadiationThirdBody objects provide.

Finally, with the net flux available, a call is made to update the surface.

(d) **third_body_adjustments**

This method calls each RadiationThirdBody object's *process_third_body* method, which returns the fraction of the primary source that the RadiationThirdBody is eclipsing. These values are multiplied together, using the assumption that no eclipses overlap. The final value is used to modify the incident flux, previously calculated.

(e) **update_default_surface**

This method is a simplified version of the method *update_facet_surface* (on this page). It processes the same calculations to identify the incident flux. Then because there is only one facet, the only thermal update to carry out is the potential for dumping thermal energy from some internal power source. The temperature is then integrated if necessary. Because the surface is isothermal, there is no net thermal emission force, and because the surface is spherical, there is no torque.

(f) **update_facet_surface**

This method processes the following steps:

i. Set to zero the absorbed power, and the forces due to absorption, and specular and diffuse reflection on each facet with a call to the *RadiationSurface::initialize_runtime_values* method. The power and force components are incremented during the processing of the Radiation Pressure Model, so must start at zero.

ii. A call is made to *RadiationSurface::incident_radiation* to process the forces resulting from absorption and reflection of light direct from the primary source (Sun). The flux from the primary source has already been calculated.

iii. For each registered RadiationThirdBody that is also interactive (i.e. is a RadiationReflectingThirdBody) and is active (i.e. typically this means that it is in the vicinity of the vehicle), the interactions with the flux reflected and/or radiated from that body are calculated (see *interact_with_third_body* (on the next page)).

iv. The thermal rider is updated. The primary purpose of this call is to integrate the temperature; this may seem unnecessary if the user does not wish to maintain an evolutionary profile of the temperature. However, this method also contains the methods to add thermal energy from within the vehicle. Then, if the temperature is dynamic, it is integrated and the emitted power obtained. If the temperature is to be held fixed, the method *equalize_absorption_emission* is called to set the emitted power equal to the absorbed power, but after internal energy sources have been added to the surface.

v. With the emitted power now available, the force due to thermal emission can be calculated if forces are desired (see *radiation_pressure* (on page 58). With the emission force calculated, all forces are known; the surface accumulates all forces and corresponding torques from all facets into an overall surface force. Those data are copied into the model force and torque.

(g) **add_third_body**

The RadiationPressure class keeps a record of all RadiationThirdBody objects that could influence the raw flux between the primary source (Sun) and target (the vehicle with which the particular instance of the RadiationPressure model is associated). The primary role of this method is to verify that RadiationThirdBody objects have been initialized correctly, and if so, to add them to the list (vector) of RadiationThirdBody objects. It includes verification that the object exists, is named with a unique name (within this model), and has not previously been added to any other RadiationPressure model (because the RadiationThirdBody can see the model's RadiationSource data, and RadiationSource contains data about the source-vehicle position, each RadiationThirdBody object can be associated with only one vehicle, and consequently, only one RadiationPressure model).

The actual registration involves a test of casting to a RadiationReflectingThirdBody. This class is not implemented in this release of JEOD, but provides the foundation for future addition of an albedo model.

Although this method is typically called during simulation setup (pre-initialization), it can be called at any time in the simulation. Hence, if the model has already been initialized, it is necessary to initialize this third-body (otherwise, the initialization will be performed as an integral aspect of the model initialization).

Finally, if any third-body objects are flagged as being active, the model flag must be set accordingly.

(h) **set_third_body_active**

RadiationThirdBody objects may be listed with the RadiationPressure model, but inactivated. This means that they will not be considered when calculating the effect of the

third-bodies. This method changes the state of a RadiationThirdBody from inactive to active; the RadiationThirdBody is identified by name.

(i) **set_third_body_inactive**
See *set_third_body_active* immediately above. This method works in reverse.

(j) **find_third_body**
This simple method identifies the location in the vector of third-body pointers of a particular instance, identified by name.

(k) **set_calculate_forces**
This method is used to set the *calculate_forces* flag. Note that the model force and torque will not be calcualted if this is set to false. Consequently, it is essential that these values be set to zero with the flag being turned off, lest the last value be retained and included for all time.

8. **Class: RadiationSource** **Inheritance:** Base-class

(a) **initialize**
Identifies the source by name (most often "Sun"), and ensures that the Dynamics Manager maintains its position information.

(b) **calculate_flux**
The incident flux is governed by an inverse-square law applied to the illuminating source over the distance between source and vehicle, with additional factoring to account for intervening bodies that could cause shadowing or eclipses.
The distance from source to vehicle is determined, and the raw flux determined as a vector in the inertial and the vehicle structural reference frames, and as a magnitude.

9. **Class: RadiationSurface** **Inheritance:** Interaction Surface (Surface Model)

A Radiation Surface is a surface derived from the Surface Model. A Radiation Default Surface is another class entirely, and is used for simple, crude application of radiation pressure theory. The methods presented in this class are only applied to a Radiation Surface, and never to a Radiation Default Surface.

(a) **initialize**
Contains a sequence of calls and checks to initialize each of the facets that comprise the surface, and verify the suitability of the data in each facet.

(b) **allocate_array**
Allocates the memory for the Radiation Facets.

(c) **allocate_interaction_facet**
Calls the appropriate factory to generate Radiation Facets from the base Facets, and checks that the generation was consistent.

(d) **incident_radiation**
Calls the Radiation Facet method *incident_radiation* on each Radiation Facet in turn.

(e) **interact_with_third_body**
Calls the Radiation Facet method *interact_with_third_body* on each Radiation Facet in turn. See, for example, *FlatPlateRadiationFacet::interact_with_third_body* (on page 52).

(f) **accumulate_thermal_sources**

Calls the Thermal Facet Rider method *accumulate_thermal_sources* on each Radiation Facet in turn.

(g) **thermal_integrator**

Calls the Thermal Facet Rider method *integrate* on each Radiation Facet in turn.

(h) **equalize_absorption_emission**

Sets the Thermal Facet Rider value *power_emit* equal to the Thermal Facet Rider value *power_absorb* for each Radiation Facet in turn.

(i) **radiation_pressure**

Calls the Radiation Facet method *radiation_pressure* on each Radiation Facet in turn. Sums the forces and torques from each facet into a force and torque for the surface as a whole. Note that this method is only called from the main model function call if the flag *calculate_forces* is set.

(j) **initialize_runtime_values**

Calls the Radiation Facet method *initialize_runtime_values* on each Radiation Facet in turn.

10. **Class: RadiationSurfaceFactory** **Inheritance:** InteractionSurfaceFactory (Surface Model)

(a) **add_facet_params**

Checks that the parameter list is a Radiation Parameter list, and calls the generic Interaction Surface Factory method *add_facet_params*.

11. **Class: RadiationThirdBody** **Inheritance:** Base Class

A Third Body is an object that is neither the vehicle nor the primary illumination source (usually Sun). It can shadow the vehicle, and potentially reflect light onto the vehicle from the primary illumination source.

(a) **initialize**

Identifies the Dynamics Manager's memory allocation for the physical object that this third-body is representing, and makes sure that the Dynamics Manager will maintain up-to-date information about its state.

Calculates some radii comparisons that are used in the conical shadowing calculation.

(b) **process_third_body**

This is the primary executable for handling the ThirdBody shadowing effects. In many applications, it is not necessary to update the position and orientation of the third-body as rapidly as that of the vehicle. Particularly in the case where a third-body presents a complex albedo map for reflected radiation, that update can be unnecessarily time-consuming. The first task is to query whether the state of the third body requires an update, with a call to *test_for_state_update* (note, this method will perform the update if it is necessary). Next, the shadow that the body casts is evaluated based on its most recent state.

(c) **calculate_shadow**

See Figure 3.22 for a flow chart of how this method is processed, and sections *Reduction of Flux by Intervening Body* (on page 12) and *Conical Eclipse Calculator* (on page 18) for a complete mathematical treatment of this algorithm.

In summary of the method, the following steps are taken:

i. The position vector of the Third Body with respect to the primary source is calculated (*source_to_third*), and decomposed into magnitude and unit vector.

ii. The position vector of the vehicle with respect to the Third Body is calculated (*third_to_cg*).

iii. The component of *third_to_cg* that is parallel to *source_to_third* is calculated (*r_par*). If $r_par < 0$, the vehicle is "in front" of the Third Body and cannot be shadowed by it.

iv. The component of *third_to_cg* that is perpendicular to *source_to_third* is calculated (*r_perp*).

v. The user has two options for the geometry of the shadow:

- A *cylindrical* shadow blocks out the cross section of the Third Body parallel to *source_to_third*, extending to infinity.
- A *conical* shadow treats the lines extending from the limbs of the illuminating body to the limbs of the Third Body to determine whether all, part, or none, of the illuminating disk is obscured. This method is described in section *Conical Eclipse Calculator* (on page 18).

vi. Full or zero obscuration are easy to handle, but the situation in which part of the disk is obscured is more challenging. To avoid multiple inverse trigonometric functions, a description has been developed utilizing a 3rd order polynomial, with 2nd or 3rd order polynomial coefficients.

(d) **test_for_state_update**

If sufficient time has elapsed since the last time the state was updated (as determined by a user-defined value, *update_interval*), update the state, and record the time.

(e) **update_third_body_state**

Generates the position of the third body with respect to the primary source. For a generic RadiationThirdBody, orientation is not necessary.

(f) **convert_shadow_from_int**

This method is used to ensure compatibility between the old paradigms in which the RadiationThirdBody objects are contained within the RadiationSource, and the new paradigm in which they are stand-alone objects registered with the RadiationPressure model. The shadow geometry enumeration is located differently for these two paradigms; this method moves the older implementation to the new enumeration.

(g) **accumulate_refl_flux**

For a generic RadiationThirdBody object, there is no reflection flux from its surface. This method is provided to allow for extension of an albedo model to a RadiationDefaultSurface, but for this class, it returns 0.0.

(h) **accumulate_rad_flux**

For a generic RadiationThirdBody object, there is no radiative flux from its surface. This method is provided to allow for extension of a thermal radiation model to a RadiationDefaultSurface, but for this class, it returns 0.0.

(i) **accumulate_oriented_refl_flux**

For a generic RadiationThirdBody object, there is no reflection flux from its surface. This method is provided to allow for extension of an albedo model to a RadiationSurface, but for this class, it returns 0.0.

(j) **accumulate_oriented_refl_flux**

For a generic RadiationThirdBody object, there is no radiative flux from its surface. This method is provided to allow for extension of a thermal radiation model to a RadiationSurface, but for this class, it returns 0.0.

(k) **is_interactive**

This is used to distinguish between those third body objects that can generate flux vectors (through radiative emission or reflection) from thise that cannot. This generic form of a RadiationThirdBody cannot. Consequently, this method returns *false*.

(l) **get_added_to_model**

The *added_to_model* flag gets set when an instance of a RadiationThirdBody gets added to a RadiationPressure model. It is essential that a RadiationThirdBody object can only be in one RadiationPressure model. It is possible to have multiple RadiationThirdBody objects that represent the same object (e.g. for a 2-vehicle simulation around Earth, there may be two RadiationThirdBody objects that represent Earth, once for each vehicle's RadiationPressure model), but the same object must not be used twice. The *added_to_model* protects against that.

(m) **set_added_to_model**

See *get_added_to_model* immediately above.

12. **Class: RadiationReflectingThirdBody** **Inheritance:** RadiationThirdBody

This derived class is left as a stub to allow for a third-body object to contain an albedo model and provide a reflective and radiative flux to the vehicle. It is not currently implemented.

3.4 Inventory

All Radiation Pressure Model files are located in the directory `${JEOD_HOME}/models/interactions/radiation_pressure`. Relative to this directory,

- Header and source files are located in the model `include` and `src` subdirectories. Table 3.1 lists the configuration-managed files in these directories.
- Data files are located in the model `data` subdirectory. See table **??** for a listing of the configuration-managed files in this directory.
- Documentation files are located in the model `docs` subdirectory. See table 3.2 for a listing of the configuration-managed files in this directory.
- Verification files are located in the model `verif` subdirectory. See table 3.3 for a listing of the configuration-managed files in this directory.

Table 3.1: Source Files

File Name
include/flat_plate_radiation_facet.hh
include/flat_plate_radiation_factory.hh
include/radiation_base_facet.hh
include/radiation_default_surface.hh
include/radiation_facet.hh
include/radiation_messages.hh
include/radiation_params.hh
include/radiation_pressure.hh
include/radiation_source.hh
include/radiation_surface.hh
include/radiation_surface_factory.hh
include/radiation_third_body.hh
src/cmake_file_list.cmake
src/flat_plate_radiation_facet.cc
src/flat_plate_radiation_factory.cc
src/radiation_base_facet.cc
src/radiation_default_surface.cc
src/radiation_facet.cc
src/radiation_messages.cc
src/radiation_pressure.cc
src/radiation_pressure__default_surface.cc
src/radiation_pressure__surface_model.cc

Continued on next page

Table 3.1: Source Files (continued from previous page)

File Name
src/radiation_source.cc
src/radiation_surface.cc
src/radiation_surface_factory.cc
src/radiation_third_body.cc

Table 3.2: Documentation Files

File Name
docs/radiation_pressure.pdf
docs/refman.pdf
docs/tex/RadiationPressure.bib
docs/tex/RadiationPressureAbstract.tex
docs/tex/RadiationPressureIntro_change_history.tex
docs/tex/RadiationPressureIntroduction.tex
docs/tex/RadiationPressureReqt.tex
docs/tex/RadiationPressureReqt_trace.tex
docs/tex/RadiationPressureSpec_Concept.tex
docs/tex/RadiationPressureSpec_Detail.tex
docs/tex/RadiationPressureSpec_Detail_Flow.tex
docs/tex/RadiationPressureSpec_Inventory.tex
docs/tex/RadiationPressureSpec_Math.tex
docs/tex/RadiationPressureUser_Analysis.tex
docs/tex/RadiationPressureUser_Extension.tex
docs/tex/RadiationPressureUser_Integration.tex
docs/tex/RadiationPressureVV_Metrics.tex
docs/tex/RadiationPressureVV_Valid.tex
docs/tex/RadiationPressureVV_Verif.tex
docs/tex/makefile
docs/tex/model_name.mk
docs/tex/radiation_pressure.sty
docs/tex/radiation_pressure.tex
docs/tex/wrapfig.sty
docs/tex/figs/LAGEOS.jpg
docs/tex/figs/LAGEOS2.jpg
docs/tex/figs/blank.jpg

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/figs/Fe_int/force_analytic.jpg
docs/tex/figs/Fe_int/force_emission10plate.jpg
docs/tex/figs/Fe_int/temperature-time.jpg
docs/tex/figs/Fe_int/temperature_analytic.jpg
docs/tex/figs/Fsda/forces_parallel.jpg
docs/tex/figs/Fsda/forces_perpendicular.jpg
docs/tex/figs/Plate_mod/Fx_ref_dif_alb_high.jpg
docs/tex/figs/Plate_mod/Fx_ref_dif_alb_low_.jpg
docs/tex/figs/Plate_mod/Fx_ref_dif_alb_mid_.jpg
docs/tex/figs/Plate_mod/Fx_ref_spc_alb_high.jpg
docs/tex/figs/Plate_mod/Fx_ref_spc_alb_low_.jpg
docs/tex/figs/Plate_mod/Fx_ref_spc_alb_mid_.jpg
docs/tex/figs/Plate_mod/Fy_ref_dif_alb_high.jpg
docs/tex/figs/Plate_mod/Fy_ref_dif_alb_low_.jpg
docs/tex/figs/Plate_mod/Fy_ref_dif_alb_mid_.jpg
docs/tex/figs/Plate_mod/Fy_ref_spc_alb_high.jpg
docs/tex/figs/Plate_mod/Fy_ref_spc_alb_low_.jpg
docs/tex/figs/Plate_mod/Fy_ref_spc_alb_mid_.jpg
docs/tex/figs/Plate_mod/I_alb_high_HC_high_ref_dif.jpg
docs/tex/figs/Plate_mod/I_alb_high_HC_high_ref_spc.jpg
docs/tex/figs/Plate_mod/I_alb_high_HC_low__ref_dif.jpg
docs/tex/figs/Plate_mod/I_alb_high_HC_low__ref_spc.jpg
docs/tex/figs/Plate_mod/I_alb_high_HC_mid__ref_dif.jpg
docs/tex/figs/Plate_mod/I_alb_high_HC_mid__ref_spc.jpg
docs/tex/figs/Plate_mod/I_alb_low__HC_high_ref_dif.jpg
docs/tex/figs/Plate_mod/I_alb_low__HC_high_ref_spc.jpg
docs/tex/figs/Plate_mod/I_alb_low__HC_low__ref_dif.jpg
docs/tex/figs/Plate_mod/I_alb_low__HC_low__ref_spc.jpg
docs/tex/figs/Plate_mod/I_alb_low__HC_mid__ref_dif.jpg
docs/tex/figs/Plate_mod/I_alb_low__HC_mid__ref_spc.jpg
docs/tex/figs/Plate_mod/I_alb_mid__HC_high_ref_dif.jpg
docs/tex/figs/Plate_mod/I_alb_mid__HC_high_ref_spc.jpg
docs/tex/figs/Plate_mod/I_alb_mid__HC_low__ref_dif.jpg
docs/tex/figs/Plate_mod/I_alb_mid__HC_low__ref_spc.jpg

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/figs/Plate_mod/I_alb_mid__HC_mid__ref_dif.jpg
docs/tex/figs/Plate_mod/I_alb_mid__HC_mid__ref_spc.jpg
docs/tex/figs/Plate_mod/I_alb_mid__HC_mid_ref_spc2.jpg
docs/tex/figs/Plate_mod/sim4forces.jpg
docs/tex/figs/cyl_cub_comp/cyl_cub_fig1.jpg
docs/tex/figs/cyl_cub_comp/cyl_cub_fig2.jpg
docs/tex/figs/def_surf/specular.jpg
docs/tex/figs/def_surf/specular.odg
docs/tex/figs/eclipse/annular_eclipse_layout.jpg
docs/tex/figs/eclipse/drawings1_te.jpg
docs/tex/figs/eclipse/drawings2_te.jpg
docs/tex/figs/eclipse/flux-radial_distance.jpg
docs/tex/figs/eclipse/force_partial_eclipse.jpg
docs/tex/figs/flowchart/flow_FP_incident_radiation.png
docs/tex/figs/flowchart/flow_FP_interact_with_third_body.png
docs/tex/figs/flowchart/flow_FP_radiation_pressure.png
docs/tex/figs/flowchart/flow_accumulate_thermal_sources.png
docs/tex/figs/flowchart/flow_calculate_flux.png
docs/tex/figs/flowchart/flow_calculate_shadow.png
docs/tex/figs/flowchart/flow_chart.odg
docs/tex/figs/flowchart/flow_default_incident_radiation.png
docs/tex/figs/flowchart/flow_default_interact_with_third_body.png
docs/tex/figs/flowchart/flow_facet_accumulate_thermal_sources.png
docs/tex/figs/flowchart/flow_facet_thermal_integrate.png
docs/tex/figs/flowchart/flow_incident_radiation.png
docs/tex/figs/flowchart/flow_interact_with_third_body.png
docs/tex/figs/flowchart/flow_process_third_body.png
docs/tex/figs/flowchart/flow_radiation_pressure.png
docs/tex/figs/flowchart/flow_test_for_state_update.png
docs/tex/figs/flowchart/flow_thermal_integrator.png
docs/tex/figs/flowchart/flow_thermal_update.png
docs/tex/figs/flowchart/flow_third_body_adjustments.png
docs/tex/figs/flowchart/flow_top_level.png
docs/tex/figs/flowchart/flow_update.png

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/figs/flowchart/flow_update_default_surface.png
docs/tex/figs/flowchart/flow_update_facet_surface.png
docs/tex/figs/isoth_app/iso_app_figs1.jpg
docs/tex/figs/isoth_app/iso_app_figs2.jpg
docs/tex/figs/isoth_app/iso_app_figs3.jpg
docs/tex/figs/isoth_app/iso_app_figs4.jpg
docs/tex/figs/isoth_app/iso_app_figs5.jpg
docs/tex/figs/isoth_app/iso_app_figs6.jpg
docs/tex/figs/isoth_app/iso_app_figs7.jpg
docs/tex/figs/isoth_app/iso_app_figs8.jpg
docs/tex/figs/orbital/orbit_delp.jpg
docs/tex/figs/orbital/orbit_delv.jpg
docs/tex/figs/orbital/orbitfluxshort.jpg
docs/tex/figs/orbital/radiation_effect.jpg
docs/tex/figs/orbital/solarflux.jpg
docs/tex/figs/orbital/solarflux1.jpg
docs/tex/figs/sda/1_sda.jpg
docs/tex/figs/sda/2_sda.jpg
docs/tex/figs/sda/3_sda.jpg
docs/tex/figs/sda/4_sda.jpg
docs/tex/figs/sda/F__x_diff__gloss.jpg
docs/tex/figs/sda/F__x_diff__matte.jpg
docs/tex/figs/sda/F__x_diff__rough.jpg
docs/tex/figs/sda/F__x_diff_smooth.jpg
docs/tex/figs/sda/F__y_diff__gloss.jpg
docs/tex/figs/sda/F__y_diff__matte.jpg
docs/tex/figs/sda/F__y_diff__rough.jpg
docs/tex/figs/sda/F__y_diff_smooth.jpg
docs/tex/figs/sda/F__z_diff__gloss.jpg
docs/tex/figs/sda/F__z_diff__matte.jpg
docs/tex/figs/sda/F__z_diff__rough.jpg
docs/tex/figs/sda/F__z_diff_smooth.jpg
docs/tex/figs/sda/F_error_albedo__low_surf__gloss.jpg
docs/tex/figs/sda/F_error_albedo__low_surf__matte.jpg

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/figs/sda/F_error_albedo__low_surf__rough.jpg
docs/tex/figs/sda/F_error_albedo__low_surf_smooth.jpg
docs/tex/figs/sda/F_error_albedo__med_surf__gloss.jpg
docs/tex/figs/sda/F_error_albedo__med_surf__matte.jpg
docs/tex/figs/sda/F_error_albedo__med_surf__rough.jpg
docs/tex/figs/sda/F_error_albedo__med_surf_smooth.jpg
docs/tex/figs/sda/F_error_albedo_high_surf__gloss.jpg
docs/tex/figs/sda/F_error_albedo_high_surf__matte.jpg
docs/tex/figs/sda/F_error_albedo_high_surf__rough.jpg
docs/tex/figs/sda/F_error_albedo_high_surf_smooth.jpg
docs/tex/figs/sda/F_x_error_albedo__low_surf__gloss.jpg
docs/tex/figs/sda/F_x_error_albedo__low_surf__matte.jpg
docs/tex/figs/sda/F_x_error_albedo__low_surf__rough.jpg
docs/tex/figs/sda/F_x_error_albedo__low_surf_smooth.jpg
docs/tex/figs/sda/F_x_error_albedo__med_surf__gloss.jpg
docs/tex/figs/sda/F_x_error_albedo__med_surf__matte.jpg
docs/tex/figs/sda/F_x_error_albedo__med_surf__rough.jpg
docs/tex/figs/sda/F_x_error_albedo__med_surf_smooth.jpg
docs/tex/figs/sda/F_x_error_albedo_high_surf__gloss.jpg
docs/tex/figs/sda/F_x_error_albedo_high_surf__matte.jpg
docs/tex/figs/sda/F_x_error_albedo_high_surf__rough.jpg
docs/tex/figs/sda/F_x_error_albedo_high_surf_smooth.jpg
docs/tex/figs/sda/F_z_error_albedo__low_surf__gloss.jpg
docs/tex/figs/sda/F_z_error_albedo__low_surf__matte.jpg
docs/tex/figs/sda/F_z_error_albedo__low_surf__rough.jpg
docs/tex/figs/sda/F_z_error_albedo__low_surf_smooth.jpg
docs/tex/figs/sda/F_z_error_albedo__med_surf__gloss.jpg
docs/tex/figs/sda/F_z_error_albedo__med_surf__matte.jpg
docs/tex/figs/sda/F_z_error_albedo__med_surf__rough.jpg
docs/tex/figs/sda/F_z_error_albedo__med_surf_smooth.jpg
docs/tex/figs/sda/F_z_error_albedo_high_surf__gloss.jpg
docs/tex/figs/sda/F_z_error_albedo_high_surf__matte.jpg
docs/tex/figs/sda/F_z_error_albedo_high_surf__rough.jpg
docs/tex/figs/sda/F_z_error_albedo_high_surf_smooth.jpg

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/figs/sda/reflection1_sda.jpg
docs/tex/figs/sda/reflection2_sda.jpg
docs/tex/figs/sda/reflection3_sda.jpg
docs/tex/figs/sda/reflection4_sda.jpg
docs/tex/figs/sda/reflection5_sda.jpg
docs/tex/figs/sda/s_x_smooth.jpg
docs/tex/figs/sda/s_y_smooth.jpg
docs/tex/figs/sda/s_z_smooth.jpg
docs/tex/figs/sda/unit_vec_diff__x__gloss.jpg
docs/tex/figs/sda/unit_vec_diff__x__matte.jpg
docs/tex/figs/sda/unit_vec_diff__x__rough.jpg
docs/tex/figs/sda/unit_vec_diff__x_smooth.jpg
docs/tex/figs/sda/unit_vec_diff__y__gloss.jpg
docs/tex/figs/sda/unit_vec_diff__y__matte.jpg
docs/tex/figs/sda/unit_vec_diff__y__rough.jpg
docs/tex/figs/sda/unit_vec_diff__y_smooth.jpg
docs/tex/figs/sda/unit_vec_diff__z__gloss.jpg
docs/tex/figs/sda/unit_vec_diff__z__matte.jpg
docs/tex/figs/sda/unit_vec_diff__z__rough.jpg
docs/tex/figs/sda/unit_vec_diff__z_smooth.jpg
docs/tex/figs/shadow/shadow_calculator_fig1.jpg
docs/tex/figs/shadow/shadow_calculator_fig2.jpg
docs/tex/figs/shadow/shadow_calculator_fig3.jpg
docs/tex/figs/shadow/shadow_calculator_fig4.jpg
docs/tex/figs/shadow/shadow_calculator_fig5.jpg
docs/tex/figs/shadow/shadow_calculator_fig6.jpg
docs/tex/figs/shadow/shadow_calculator_fig7.jpg
docs/tex/figs/shadow/shadow_seeker_fig1.jpg
docs/tex/figs/shadow/shadow_seeker_fig10.jpg
docs/tex/figs/shadow/shadow_seeker_fig11.jpg
docs/tex/figs/shadow/shadow_seeker_fig2.jpg
docs/tex/figs/shadow/shadow_seeker_fig3.jpg
docs/tex/figs/shadow/shadow_seeker_fig4.jpg
docs/tex/figs/shadow/shadow_seeker_fig5.jpg

Continued on next page

Table 3.2: Documentation Files (continued from previous page)

File Name
docs/tex/figs/shadow/shadow_seeker_fig6.jpg
docs/tex/figs/shadow/shadow_seeker_fig7.jpg
docs/tex/figs/shadow/shadow_seeker_fig8.jpg
docs/tex/figs/shadow/shadow_seeker_fig9.jpg

Table 3.3: Verification Files

File Name
verif/SIM_1_BASIC/S_define
verif/SIM_1_BASIC/S_overrides.mk
verif/SIM_1_BASIC/DP_Product/DP_validation.xml
verif/SIM_1_BASIC/Log_data/log_radiation_rec.py
verif/SIM_1_BASIC/Modified_data/radiation_surface.py
verif/SIM_1_BASIC/Modified_data/third_bodies.py
verif/SIM_1_BASIC/SET_test/RUN_basic/input.py
verif/SIM_1_BASIC/SET_test/RUN_basic_cr/input.py
verif/SIM_1_BASIC/SET_test/RUN_basic_dynbody/input.py
verif/SIM_2A_SHADOW_CALC/S_define
verif/SIM_2A_SHADOW_CALC/S_overrides.mk
verif/SIM_2A_SHADOW_CALC/DP_Product/DP_validation.xml
verif/SIM_2A_SHADOW_CALC/Log_data/log_radiation_rec.py
verif/SIM_2A_SHADOW_CALC/Modified_data/date_and_time.py
verif/SIM_2A_SHADOW_CALC/Modified_data/third_bodies.py
verif/SIM_2A_SHADOW_CALC/SET_test/RUN_annular_eclipse/ input.py
verif/SIM_2A_SHADOW_CALC/SET_test/RUN_shadow_cooling/ input.py
verif/SIM_2A_SHADOW_CALC/SET_test/RUN_transverse_shadow/ input.py
verif/SIM_2_SHADOW_CALC/S_define
verif/SIM_2_SHADOW_CALC/S_overrides.mk
verif/SIM_2_SHADOW_CALC/DP_Product/DP_validation.xml
verif/SIM_2_SHADOW_CALC/Log_data/log_radiation_rec.py
verif/SIM_2_SHADOW_CALC/Log_data/log_radiation_rec_v2.py
verif/SIM_2_SHADOW_CALC/Modified_data/date_and_time.py

Continued on next page

Table 3.3: Verification Files (continued from previous page)

File Name
verif/SIM_2_SHADOW_CALC/Modified_data/radiation_surface.py
verif/SIM_2_SHADOW_CALC/Modified_data/radiation_surface_a.py
verif/SIM_2_SHADOW_CALC/Modified_data/radiation_surface_v2.py
verif/SIM_2_SHADOW_CALC/Modified_data/third_bodies.py
verif/SIM_2_SHADOW_CALC/SET_test/RUN_annular_eclipse/input.py
verif/SIM_2_SHADOW_CALC/SET_test/RUN_shadow_cooling/input.py
verif/SIM_2_SHADOW_CALC/SET_test/RUN_ten_plates/input.py
verif/SIM_2_SHADOW_CALC/SET_test/RUN_transverse_shadow/ input.py
verif/SIM_3_ORBIT/S_define
verif/SIM_3_ORBIT/S_overrides.mk
verif/SIM_3_ORBIT/DP_Product/DP_validation.xml
verif/SIM_3_ORBIT/Log_data/log_orbit_rec.py
verif/SIM_3_ORBIT/Modified_data/date_and_time.py
verif/SIM_3_ORBIT/Modified_data/radiation_surface.py
verif/SIM_3_ORBIT/Modified_data/third_bodies.py
verif/SIM_3_ORBIT/Modified_data/vehicle_baseline.py
verif/SIM_3_ORBIT/SET_test/RUN_baseline/input.py
verif/SIM_3_ORBIT/SET_test/RUN_radiation/input.py
verif/SIM_3_ORBIT_1st_ORDER/S_define
verif/SIM_3_ORBIT_1st_ORDER/S_overrides.mk
verif/SIM_3_ORBIT_1st_ORDER/DP_Product/DP_validation.xml
verif/SIM_3_ORBIT_1st_ORDER/Log_data/log_orbit_rec.py
verif/SIM_3_ORBIT_1st_ORDER/Modified_data/date_and_time.py
verif/SIM_3_ORBIT_1st_ORDER/Modified_data/radiation_surface.py
verif/SIM_3_ORBIT_1st_ORDER/Modified_data/third_bodies.py
verif/SIM_3_ORBIT_1st_ORDER/Modified_data/vehicle_baseline.py
verif/SIM_3_ORBIT_1st_ORDER/SET_test/RUN_baseline/input.py
verif/SIM_3_ORBIT_1st_ORDER/SET_test/RUN_radiation/input.py
verif/SIM_4_DEFAULT/S_define
verif/SIM_4_DEFAULT/S_overrides.mk
verif/SIM_4_DEFAULT/DP_Product/DP_validation.xml
verif/SIM_4_DEFAULT/Log_data/log_radiation_rec.py
verif/SIM_4_DEFAULT/Modified_data/date_and_time.py
verif/SIM_4_DEFAULT/Modified_data/radiation_surface.py

Continued on next page

Table 3.3: Verification Files (continued from previous page)

File Name
verif/SIM_4_DEFAULT/Modified_data/third_bodies.py
verif/SIM_4_DEFAULT/Modified_data/vehicle_baseline.py
verif/SIM_4_DEFAULT/SET_test/RUN_compare/input.py
verif/S_modules/radiation.sm
verif/S_modules/radiation_1st_order.sm
verif/S_modules/radiation_simple.sm
verif/S_modules/radiation_simple_no_decl.sm
verif/include/radiation_data_recorder.hh
verif/src/radiation_data_recorder.cc
verif/unit_tests/CMakeLists.txt
verif/unit_tests/flat_plate_radiation_facet_ut.cc
verif/unit_tests/flat_plate_radiation_factory_ut.cc
verif/unit_tests/makefile
verif/unit_tests/radiation_base_facet_ut.cc
verif/unit_tests/radiation_default_surface_ut.cc
verif/unit_tests/radiation_facet_ut.cc
verif/unit_tests/radiation_pressure__default_surface_ut.cc
verif/unit_tests/radiation_pressure__surface_model_ut.cc
verif/unit_tests/radiation_pressure_ut.cc
verif/unit_tests/radiation_source_ut.cc
verif/unit_tests/radiation_surface_factory_ut.cc
verif/unit_tests/radiation_surface_ut.cc
verif/unit_tests/radiation_third_body_ut.cc

Chapter 4

User Guide

The Analysis section of the User Guide is intended primarily for users of preexisting simulations. It contains:

- a description of how to modify Radiation Pressure Model variables after the simulation has compiled, including an in-depth discussion of the input file,
- an overview of how to interpret (but not edit) the S_define file,
- a sample of some of the typical variables that may be logged.

The Integration section is intended for simulation developers. It describes the necessary configuration of the Radiation Pressure Model within an S_define file, and the creation of standard run directories. The latter component assumes a thorough understanding of the preceding Analysis section. Where applicable, the user may be directed to selected portions of Product Specification (Chapter 3).

The Extension section is intended primarily for developers needing to extend the capability of the Radiation Pressure Model. Such users should have a thorough understanding of how the model is used in the preceding Integration section, and of the model specification (described in Chapter 3).

4.1 Analysis

4.1.1 Identifying the Radiation Pressure Model

If the radiation pressure model is included in your simulation, there must be an instance declared of the class *RadiationPressure*. There must also be a Radiation Surface through which the radiation field can interact with the vehicle. Searching the S_define for "RadiationPressure" is a good strategy for determining the presence of a Radiation Pressure model in the simulation. Once identified, it is also necessary to identify which type of surface is being used; there are two fundamentally different types of surface, the general surface (the S_define will contain an instance of a RadiationSurface) and the model-specific, default surface (the S_define will contain an instance of a RadiationDefaultSurface).

The default surface is defined by relatively few characteristics, so is more easily created, managed, and manipulated. However, it is constrained to represent the effect of a spherical shell around the vehicle, so has limited applicability. Because the default surface is so easily defined, the definition often resides in an input file.

The general surface uses the Surface Model to define the surface (see the Surface Model [5] documentation for details on how this functions). The basic premise of the surface model is that the actual vehicular surface can be represented as a combination of geometric surfaces, or facets (typically, these are flat plates). The number of facets used to describe the surface depends on the complexity of the surface and the desired fidelity of the approximation. Because the general surface can be significantly more complex, it is typically defined in a data file that is included into the input file.

4.1.2 Editing the Radiation Pressure Model

The model parameters and the individual parameters for the surface as defined can easily be edited. It is also possible to replace one surface with another mid-simulation (for example, to increase the desired fidelity during particularly sensitive phases of the mission), although this requires knowledge of how to implement a surface, and is beyond the scope of the Analyst. Details can be found in the Integration section of the User's Guide.

The description of the control is divided into model-wide control, surface element control, and illumination control.

4.1.2.a Model control

The most important flag is the *active* flag; this turns the radiation pressure model on or off. This value defaults to true.

The following settings apply to the model as a whole:

- *calculate_forces*, default = true.
  The Radiation Pressure Model can be used to model surface temperature variations and forces resulting from the interaction of the vehicle surface with the radiation environment. Turning this off restricts the scope to temperature only.

4.1.2.b RadiationThirdBody Control

The frequency with which the RadiationThirdBody state is updated can be manipulated. This is more important for RadiationReflectingThirdBody items than for generic RadiationThirdBody. The value *update_interval* specifies the minimum time interval between updates. The state will be updated at the call following passage of this time. If this value is set to 0.0 (default), the state will be updated continuously.

4.1.2.c Default Surface Control

The following values affect the behavior if the default surface:

- Cross-sectional Area (*cx_area*, default = 0.0).

- Surface Area (*surface_area*, default = 0.0).
  The area of the surface can be defined in one of two ways – either as a cross-sectional, or a surface area. Since the default surface models a sphere, these differ by a factor of 4. The user should not attempt to define both.

- Albedo (*albedo*, default = -1.0).
  This is the fraction of light incident on the surface that is immediately reflected back away. This value runs from 0.0 (matte black) to 1.0 (mirrored).

  Either the radiation coefficient, or the combination of albedo and diffuse parameter, must be specified for a default surface. The negative default settings on these values ensure that one *and only one* valid entry is made.

- Diffuse parameter (*diffuse*, default = -1.0).
  This is the fraction of reflected light that is reflected diffusely; reflection can often be classified into specular or diffuse reflection, specular reflection is the type of reflection from a mirror, whereas diffuse reflection is the type of reflection off a matte surface. A rough matte surface would have a diffuse parameter close to 1, while a highly polished surface would typically have a diffuse parameter closer to 0.

  Either the radiation coefficient, or the combination of albedo and diffuse parameter, must be specified for a default surface. The negative default settings on these values ensure that one *and only one* valid entry is made.

- Emissivity (*thermal.emissivity*, default = 0.0).
  The emissivity measures how close to Planck-like Black Body radiation the thermal emission actually is. This value is typically highly dependent on albedo, with the sum of the two terms approximately equal to 1.0.

- Heat Capacity (*thermal.heat_capacity*, default = 0.0).
  The heat capacity must be defined if the surface is thermally dynamic. It is quite unreasonable to set some arbitrary number as a default setting, so 0.0 was assigned. However, that means that the temperature of the surface can change without bound for an infinitesimal imbalance in the energy received and emitted (the vehicle requires 0.0 energy to change its temperature by each degree). If this number is not at all known, the only safe option is to make the surface thermally static by turning off the active flag.

- Internal Sources (*thermal.thermal_power_dump*), default = 0.0).
  If the vehicle has some source of internal thermal energy generation (e.g. some sort of reactor), this value would be set equal to the power input to the vehicle surface from that source.

- Radiation Coefficient (*rad_coeff*, default = -1.0).
  The radiation coefficient (C_R) provides an alternative method for specifying the reflection

parameters, using only one value instead of albedo (α) and diffuse (δ). Numerically,

$$C_R = 1 + \frac{4}{9}\alpha\delta \in [1.000, 1.444]$$

Either the radiation coefficient, or the combination of albedo and diffuse parameter, must be specified for a default surface. The negative default settings on these values ensures that one *and only one* valid entry is made.

- Temperature (*temperature*, default = 0.0).
  The extent to which radiation is absorbed and emitted has an effect on temperature, and vice versa. Since the radiation model includes pressure from thermal radiative emission, it is important that the temperature is known to some level of accuracy.

- Thermally Dynamic (*thermal.active*, default = true).
  Integration of the temperature is time-consuming and cumbersome. If the vehicle is in a situation in which the surface temperature does not change appreciably, this value can be turned off to circumvent the integration. In that case, the temperature of the vehicle will be constant in time.

4.1.2.d General Surface Control

Each facet in the surface is assigned a set of physical and material properties. The physical properties typically vary from facet to facet, but many facets may share the same material properties if they are fabricated of the same material.

4.1.2.d.i Physical Properties

The following values are assigned to each facet in the surface.

- Position (*position*, 3-vector).
  The relative positions of the facets are necessary to generate torques. If the surface is a fixed and immovable monolith, these values are typically expressed in the structural reference frame.

- Orientation (*normal*, 3-vector, unit vector).
  As the vehicle position and attitude changes, facets will tend to rotate in and out of sunlight. The orientation allows the correct angle to the flux vector to be calculated. This should be expressed in the same reference frame as the position.

- Area (*surface_area*).

- Temperature (*temperature*).
  See Default Surface for explanation.

- Thermally Dynamic (*thermal.active*).
  See Default Surface for explanation.

- Internal Sources (*thermal.thermal_power_dump*.
  See Default Surface for explanation.

4.1.2.d.ii Material Properties

Each facet contains an element, *param_name*, that identifies the material from which it is made. Each set of parameters contains the following information:

- Name (*name*).
  This matches the value of *param_name* in the facet.

- Albedo (*albedo*).
  See Default Surface for explanation.

- Diffuse Parameter(*diffuse*).
  See Default Surface for explanation.

- Emissivity (*thermal.emissivity*).
  See Default Surface for explanation.

- Heat Capacity (*thermal.heat_capacity_per_area*).
  See Default Surface for explanation of the importance of heat capacity. Because this is a material property, and heat capacity depends on the size of the facet, this value is defined as heat capacity per unit area, and internally converted into the heat capacity.

4.1.2.e Illumination Control

The radiation source is assumed to be that of the sun. This can be changed, by changing the values *source.name, source.luminosity, source.radius*; however, it is recommended that these be left at the solar default settings.

The following controls are more typically changed:

- *third_bodies[ii]*− >*shadow_geometry*, default = RadiationSource::Conical.
  This can take a value either Cylindrical (or Cyl) or Conical (or Con), and allows the user to switch between the true conical shadowing of an intervening body, and the simpler cylindrical shadowing. It is not necessary to treat all third-body objects alike.

- *third_bodies[ii]*− >*active*, default = true.
  If the user wishes to include the effects of an intervening body, the method *set_third_body_active* must be run (and *set_third_body_inactive* will inactivate it again). These methods will control the general flag, *third_bodies_active*. The flag is user-controllable, and can be manually set, but should not be (unless the user really knows what he or she is doing).

- *third_bodies_active*, default = false.
  If the *third_bodies_active* flag is set, then any registered and active third-bodies (e.g., Earth) will be processed for affect on the raw flux. IMPORTANT NOTE – there is no reason to change this flag manually. If the third bodies are activated/inactivated using the provided methods, then this flag is automatically controlled (on when any number of third bodies is active, off when all third-bodies are inactive).

4.1.3 Output Data

The primary output data of the Radiation Pressure Model are the values:

- *force*
- *torque*

Also available are the forces due to each interaction process across the surface.

- *rad_surface.F_absorption*
- *rad_surface.F_specular*
- *rad_surface.F_diffuse*
- *rad_surface.F_emission*
- *rad_surface.force*
- *rad_surface.torque*

IMPORTANT NOTE – the model *force* and *torque* are more reliable than the surface versions (*rad_surface.force* and *rad_surface.torque*). In most applications, they are identical. The distinction comes only when an active model is turned off. The model values will be reset to zero, but the surface values will retain their old value. The model values should be used preferentially over the surface values.

The same data is calculated and can be extracted for each individual facet, along with the temperature of the facet. These individual facet data cannot be directly recorded in a regular Trick simulation, but are indirectly available. An example of how to record these data is shown in *verif/SIM_2_SHADOW_CALC/Log_data_radiation_rec.d*, using the additional object *radiation.data_rec*, an instance of RadiationDataRecorder. This functionality requires an additional function call to *radiation.data_rec.record_data* from the S_define, and can be found in SIM_2_SHADOW_CALC, SIM_3_ORBIT, and SIM_4_DEFAULT. The additional functionality and class are defined in the verification section of the model, and are made available to the user, but are not intended to be an integral part of the Radiation Pressure Model.

4.2 Integration

Including the Radiation Pressure Model is a straightforward process, the most complicated part of which is in generating the Radiation Surface. For working examples, the integrator is strongly encouraged to view the simulations in the Radiation Pressure Model verification.

Note that a Radiation Pressure Model is vehicle-specific. In multi-vehicle simulations, it is necessary to have one Radiation Pressure Model per vehicle (or, at least, per vehicle for which radiation pressure is important).

4.2.1 Generating the S_define

The following steps provide a consistent method for integrating Radiation Pressure into the simulation:

1. Declare a simulation object, called *radiation* or similar, in the S_define or equivalent file.
2. Within the simulation object, declare instances of the following classes:
   (a) RadiationPressure.
   For Trick07 simulations, the preferred implementation of RadiationThirdBody objects (a third-body is one that is neither the vehicle nor the source of illumination, e.g., the sun, but can still contribute to the effect of radiation pressure) is as an array located in the RadiationSource object. In this case, this array must be populated with information on the primary source of illumination, and on any third-bodies. In the JEOD v5.4 release, there are two data files (at *models/interactions/radiation_pressure/data* that can be used as templates; one (*sun.d*) recognizes Sun only, the other (*sun_earth_moon.d*) recognizes Sun as the primary source of illumination, and Earth and Moon as third-bodies.
   For Trick10 simulations, the preferred method of implementing RadiationThirdBody objects is to instantiate them as stand-alone objects, and register them with the RadiationPressure model. This method is generally preferred, and the registration is implemented behind-the-scenes even in Trick07 simulations. In this case, there is no default data to add to the radiation pressure model, but instead the data is entered at the input level. See, for example, *verif/SIM_1_BASIC_T10/Modified_data/third_bodies.py* which sets up Earth and Moon as third-bodies much the same as *sun_earth_moon.d* did in the older paradigm.
   (b) Either one of:
   - RadiationSurface or
   - RadiationDefaultSurface

   A simulation object should not contain both of these, nor should it contain multiple examples of either one, unless the intent is to use those sequentially. One instance of RadiationPressure can use only one surface at a time. Users wishing to compare the effect of different surfaces should use multiple simulation objects, each with its own instance of RadiationPressure.
   (c) If the RadiationSurface option is used, also instantiate the following classes / class pointers:

i. RadiationSurfaceFactory
ii. SurfaceModel (located at *utils/surface_model*)
iii. Facet **
An array of pointers to the facets (*Facets* are at *utils/surface_model*).
iv. <*facet-type*> *
An array of facets of type <*facet-type*>, for example, *FlatPlateThermal* * These are probably also at *utils/surface_model*, unless the Surface Model has been extended to provide other facet-types that have been stored elsewhere.
v. An unsigned integer (unassigned value, not necessary for Trick10)
vi. RadiationParams *
vii. FacetParams * .

Note that none of these are initialized with data; the data are included from the input file as necessary.

(d) RadiationDataRecorder *(optional)*.
Use this if it is also desirable to monitor the interaction at the facet level, rather than the model level. Note that this is only available if using a RadiationSurface and not used at all if implementing a RadiationDefaultSurface.

3. Also within the simulation object, schedule the following jobs:

- Pre-initialization.
If the Surface Model is being used, the surface must be created before initialization. In Trick10, this can be done from the python input file. In Trick07, this requires the advance declaration of the jobs that will be called from the input files. This is a potentially confusing area for integration, because these jobs appear as environment class jobs, but are assigned a zero call rate to ensure that they never get called from the S_define. Instead, they are called from the input file where the data are entered, but the methods must still be declared in the S_define in order to be called from the input file. If the Default Surface is being used, or if the simulation engine can handle direct calls from the input-level files (e.g. Trick10, but not Trick07), then these calls should not be included. In the following example the variables being referred to map to the following list of previously defined items:
  - *radiation* is the simulation object
  - *facet_ptr* is the instance of *Facet ***
  - *integer* is the unsigned integer
  - *facet_params* is the instance of *FacetParams **
  - *surface* is the instance of SurfaceModel
  - *rad_surface* is the instance of the radiation surface, be that a RadiationSurface or a RadiationDefaultSurface

```
(0.0, environment) utils/surface_model:
radiation.surface.add_facets(
        In Facet**      new_facets     = radiation.facet_ptr,
        In unsigned int num_new_facets = radiation.integer);
```

```
(0.0, environment) interactions/radiation_pressure:
radiation.surface_factory.add_facet_params(
       In FacetParams*  to_add         = radiation.facet_params);
```

- Initialization
  (a) If the Surface Model is being used, the Radiation Surface must be created from the Surface.

```
(initialization) interactions/radiation_pressure:
radiation.surface_factory.create_surface(
    In SurfaceModel        * surface       =  &radiation.surface,
    Out InteractionSurface * inter_surface =  &radiation.rad_surface);
```

  (b) Whether the Surface Model or the Default Surface is being used, the Radiation Pressure Model must then be initialized. The initialization call differs slightly for the two cases:
    – Surface Model

```
(initialization) interactions/radiation_pressure:
radiation.rad_pressure.initialize(
       In  DynManager       & dyn_mgr        = mngr.dyn_manager,
       In  RadiationSurface * surf_ptr       = &radiation.rad_surface,
       In  double  center_grav[3] =
                        vehicle.dyn_body.mass.composite_properties.position);
```

    – Default Surface

```
(initialization) interactions/radiation_pressure:
radiation.rad_pressure.initialize(
       In  DynManager              & dyn_mgr  = mngr.dyn_manager,
       In  RadiationDefaultSurface * surf_ptr = &radiation.rad_surface);
```

- Scheduled jobs
  (a) Update values

```
(DYNAMICS, scheduled)  interactions/radiation_pressure:
radiation_simple.rad_pressure.update(
       In  RefFrame & veh_struc_frame = vehicle.dyn_body.structure,
       In  double     center_grav[3]  =
                              vehicle.dyn_body.mass.composite_properties.position,
       In  double     scale_factor    = time.manager.dyn_time.scale_factor);
```

  (b) Record data (*optional*).
  Used only with a Surface Model, and only if facet-specific data must be recorded.

```
(DYNAMICS, scheduled)  interactions/radiation_pressure/verif:
radiation.data_rec.record_data (
     In  RadiationSurface * surface_ptr = &radiation.rad_surface);
```

4.2.2 Generating the Input File

The input definition for the Radiation Pressure Model can be divided into two areas: the model-specific parameters, and the surface-specific parameters.

These parameters are all discussed in the Analysis section of the User's Guide. If any parameters are to be set to non-default values, that should be done here.

If the default surface is being used, that could be defined directly in the input file (see the verification simulations released with JEOD v5.4 for examples) or in a separate Modified data file. If the general Surface Model is being used to define the surface, that is typically done in a Modified data file. Again, see the the verification simulations for examples of this Modified data file. There are significant differences for the recommended structure for defining the surface between Trick07 and Trick10. The two methods compare nicely, but are not immediately compatible.

There are two steps necssary to defining RadiationFacet objects. The first is to define the "physical" characteristics of the facets, then to define the surface, or "chemical" properties.

The essential components are as follows (Trick07):

- A memory allocation for each of the facets.

```
#define NUM_FACETS 6
FlatPlateThermal ** fpt_facets;
fpt_facets = alloc (NUM_FACETS);
```

- Creation of the correct number of facets.

```
for (int ii_facet=0; ii_facet<NUM_FACETS; ii_facet++) {
  fpt_facets[ii_facet] = new FlatPlateThermal[1];
}
```

- Allocation of data to each facet, including the name of a parameter set.

```
fpt_facets[0]->position[0] {M} = 1.0 , 0.0, 0.0 ;
fpt_facets[0]->normal[0] = 1.0 , 0.0 , 0.0 ;
fpt_facets[0]->area {M2} = 4.0;
fpt_facets[0]->temperature = 270.0;
fpt_facets[0]->param_name = "radiation_test_material";
fpt_facets[0]->thermal.active = true;
fpt_facets[0]->thermal.thermal_power_dump = 0.0;
fpt_facets[0]->name = "+x";

etc.
```

- Assignment of the facet pointer defined in the S_define to the address of the facets just created in the Modified Data file.

```
radiation.facet_ptr = fpt_facets;
```

- Assignment of the integer defined in the S_define to be equal to the number of facets just created.

```
radiation.integer = NUM_FACETS;
```

- A call to the method *radiation.surface.add_facets* to populate the simulation surface with the data just defined.

  ```
  call radiation.radiation.surface.add_facets(radiation.facet_ptr);
  ```

The corresponding Trick10 input might look like:

- Memory allocation and data assignment for each of the facets.

  ```
  fpt = trick.FlatPlateThermal()
  fpt.thisown = 0
  fpt.position  = trick.attach_units( "m",[ 2.0 , 0.0, 0.0 ])
  fpt.normal  = [ 1.0 , 0.0 , 0.0 ]
  fpt.area  = trick.attach_units( "M2",2.0)
  fpt.temperature = 270.0
  fpt.param_name = "radiation_test_material"
  fpt.thermal.active = True
  fpt.thermal.thermal_power_dump = 0.0
  ```

- A call to the method *radiation.surface.add_facet* to populate the simulation surface with the data just defined.

  ```
  radiation.surface.add_facet(fpt);
  ```

  Notice the omission of the "call" and of the redundant "radiation" at the front of the command. Notice also that fpt is local, and that each facet is added one at a time rather than as a pointer to the array of facets. Consequently the command is the singular *add_facet* rather than the plural *add_facets*.

The surface parameters must also be defined, for Trick07 simulations:

- For each parameter set named in the facet definition, the creation of an instance of a material parameter set.

  ```
  RadiationParams * params;
  params = new RadiationParams;
  ```

- For each parameter set, the assignment of the name and material properties of that set.

  ```
  params->name =  "radiation_test_material";
  params->albedo = 0.5;
  params->diffuse = 0.5;
  params->thermal.emissivity = 0.5;
  params->thermal.heat_capacity_per_area = 50.0;
  ```

- Assignment of the radiation parameter array from the S_define to the address of the new parameters just defined.

```
radiation.facet_params = params;
```

- A call to the method *radiation.surface_factory.add_facet_params* to populate the array of parameter sets in the simulation with the data just defined.

```
call radiation.radiation.surface_factory.add_facet_params(
                                                    radiation.facet_params);
```

For Trick10 simulations the structure is once again similar, but incompatible:

```
params = trick.RadiationParams()
params.thisown = 0

params.name =  "radiation_test_material"
params.albedo = 0.0
params.diffuse = 0.5
params.thermal.emissivity = 1.0
params.thermal.heat_capacity_per_area = 600.0

radiation.surface_factory.add_facet_params(params)
```

4.3 Extension

The Radiation Pressure Model represents a complete method for calculating the effect of incident radiation on a surface. There are several limitations and potential areas for extending the capability of this model:

1. Albedo Pressure modeling
   In JEOD v5.4 only direct radiation pressure from the primary source is calculated. However, the opportunity exists to add a model to represent the effect of scattered light reflected from a third body, for example, Earth. The code architecture was structured such that it would be possible to add additional sources at a later time. This project is mostly completed, but has not been adequately validated for inclusion into JEOD v5.4. Contact the model author for more information on the upcoming release of this extension.

2. Occultation
   As the model is currently implemented, the primary third-body objects are planets. However, the model has been extended to make DynBody objects another option for third-bodies, thereby providing the capability of vehicle-vehicle occultations. The DynBody functionality has not been adequately validated, but it is available in JEOD v5.4for users to experiment with.

3. Self-shadowing
   In JEOD v5.4 it is loosely assumed that surfaces are universally convex, so that a surface cannot create self-shadowing. However, by extending the surface through attached MassBody objects, and considering those MassBody objects in the third-bodies effects, the effect of self-shadowing can be implemented in much the same way as vehicle-vehicle occulations.

4. Improved thermal modeling
   The thermal modeling is restricted to direct illumination sources, and some internal source, but no capability exists to allow for facet-facet conduction. This could be particularly important for large, thin structures, which would have two large surfaces with significant thermal connectivity. An important example would include solar panels, with one side exposed to full illumination, and one exposed to deep space; without some thermal conductivity between those surfaces there would eventually be a very large temperature differential between the two sides. This can be crudely modeled by applying a positive thermal source to one face, and a negative source (sink) to the other, but under the current system, this would be independent of temperature differential.

Chapter 5

Verification and Validation

5.1 Verification

5.1.1 Top-level Requirement

Inspection RadiationPressure_1: Top-level Inspection

This document, the code, and associated files, have been inspected, and together satisfy requirement RadiationPressure_1.

5.1.2 Inspection of Modeling Requirements

Inspection RadiationPressure_2: State Encapsulation

The model includes the capability to represent the relative state of the vehicle, as demonstrated in section 5.1.6 on page 97, thereby satisfying requirement RadiationPressure_2

Inspection RadiationPressure_3: Albedo Pressure

The foundation for the albedo pressure capability is provided in *RadiationSource::calculate_flux*, utilizing the concept of third body interactions, handled by class *RadiationThirdBody*. The method *RadiationThirdBody::calculate_reflection_flux* is not fully developed, but the output of this method can then be used in the appropriate *incident_radiation* method, which will increment, rather than overwrite, the calculated forces to ensure that multiple sources can be accumulated. This satisfies requirement RadiationPressure_5

Inspection RadiationPressure_4: Force and Torque

The data structures are populated by current force and torque values, as required. The legitimacy of those values is demonstrated in sections *Verification of Forces* (on the facing page) and *Verification*

of Flux (on page 97). This satisfies requirement RadiationPressure_6

5.1.3 Verification of Surface Representations

Test RadiationPressure_1: Verification of two surface models

Purpose:
To demonstrate that the Radiation Pressure Model can interface with the Surface Model, and also operate separately from the Surface Model, using its own default surface representation.

Requirements:
Satisfactory conclusion of this test satisfies requirement RadiationPressure_3

Procedure:
A simulation was developed containing two independent radiation pressure models, one using the Surface Model, and the other one using the default surface model. Both models were associated with the same vehicle, so the states were identical. The vehicle was not propagated.

Predictions:
Both surfaces should be initialized and a force calculated for each. Since both surfaces have the same cross-sectional area, the forces should be comparable.

Results:
Results are as expected.

5.1.4 Verification of Force Calculations

The intent of this section is to show that the forces due to absorption, and specular and diffuse reflection are calculated correctly.

Test RadiationPressure_2: Verification that the forces due to absorption, and specular and diffuse reflection are calculated correctly.

Purpose:
To test whether the calculations of the forces due to radiation absorption, and due to reflection through both specular and diffuse processes, are calculated appropriately. While this particular test is applied to a surface comprising flat plates, other tests (RadiationPressure_1, and those in section 5.2.3 on page 118) demonstrate the similarity between the flat plate and the default surface representations.

Requirements:
Satisfactory conclusion of this test, along with the tests RadiationPressure_3 and RadiationPressure_4 satisfies requirement RadiationPressure_7

Procedure:
The simulation used in this test is available at *SIM_2_SHADOW_CALC/RUN_ten_plates*. A

vehicle was built comprising 10 plates oriented at a range of angles with respect to the incident radiation, all illuminated, all of the time. The forces were calculated for each plate with 50% of the incident light being absorbed, and 25% reflected through each of the diffuse and specular processes.

Predictions:

1. Variation with time.
   The plates were held fixed with respect to the constant incident radiation vector; there should be no variation with time.
2. Component of force parallel to the flux vector.
   - Calculation of force due to absorption.
     The only variable on which the force could depend is the angle, θ, between the incident radiation and the plate normal. This angle affects the projected area of the plate, and consequently should affect the force as the cosine of the angle.

     $$F = F_0 \cos\theta \tag{5.1}$$

     where F_0 is the force when $\theta = 0$.
   - Calculation of force due to specular reflection.
     There are two components contributing to the force – the incident and reflected light. The incident magnitude will vary, as with the absorption, with the cosine of the angle. The reflected magnitude will vary with the projected area ($\cos\theta$), and with the cosine of the double angle to account for the angle of reflection. In this simulation, there is half as much light reflected specularly as absorbed, so the force looks like

     $$F = \frac{F_0}{2}\cos\theta + \frac{F_0}{2}\cos\theta\cos 2\theta$$

     which simplifies to

     $$F = F_0 \cos^3\theta \tag{5.2}$$

     Here, F_0 maintains the same value defined for absorption.
   - Calculation of force due to diffuse reflection.
     Similarly to the specular reflection, there will be 2 components, but the angle of "reflection" will be along the normal, with value modified by the Lambert value of 2/3.

     $$F = \frac{F_0}{2}\cos\theta\left(1 + \frac{2}{3}\cos\theta\right) \tag{5.3}$$
3. Component of force perpendicular to flux vector
   - Calculation of force due to absorption.
     The component of force due to absorption perpendicular to the flux vector should be zero.
   - Calculation of force due to specular reflection.
     The magnitude depends on the projected area again, but only has contribution from

the reflected component (the incident component is parallel to the flux vector, by definition).

$$F = \frac{F_0}{2} \cos\theta \sin 2\theta \tag{5.4}$$

- Calculation of force due to diffuse reflection.
  Similarly, the magnitude depends on the projected area and the contribution from the reflected component only.

$$F = \frac{F_0}{2} \cos\theta \left(\frac{2}{3} \sin\theta \right) = \frac{F_0}{6} \sin 2\theta \tag{5.5}$$

Results:

1. As expected, there was no variation with time.
2. Figure 5.1 shows the data for the parallel component of the force for each of the three processes. All 3 variables match their analytical predictions.
3. Figure 5.2 shows the data for the perpendicular component of the force for each of the three processes. All 3 variables match their analytical predictions.

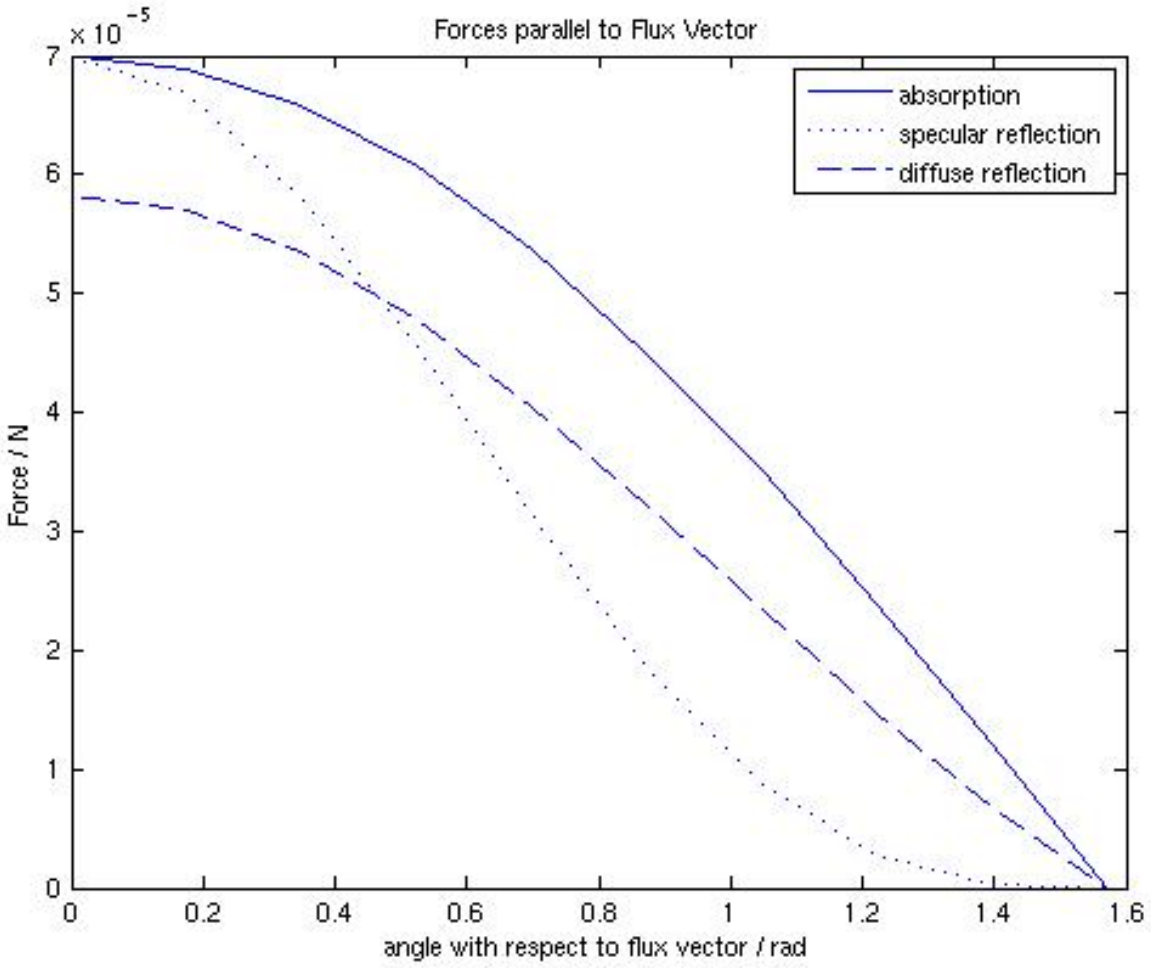


Figure 5.1: The components of the absorption, specular, and diffuse forces parallel to the flux vector as a function of the angle between the flux vector and the surface normal

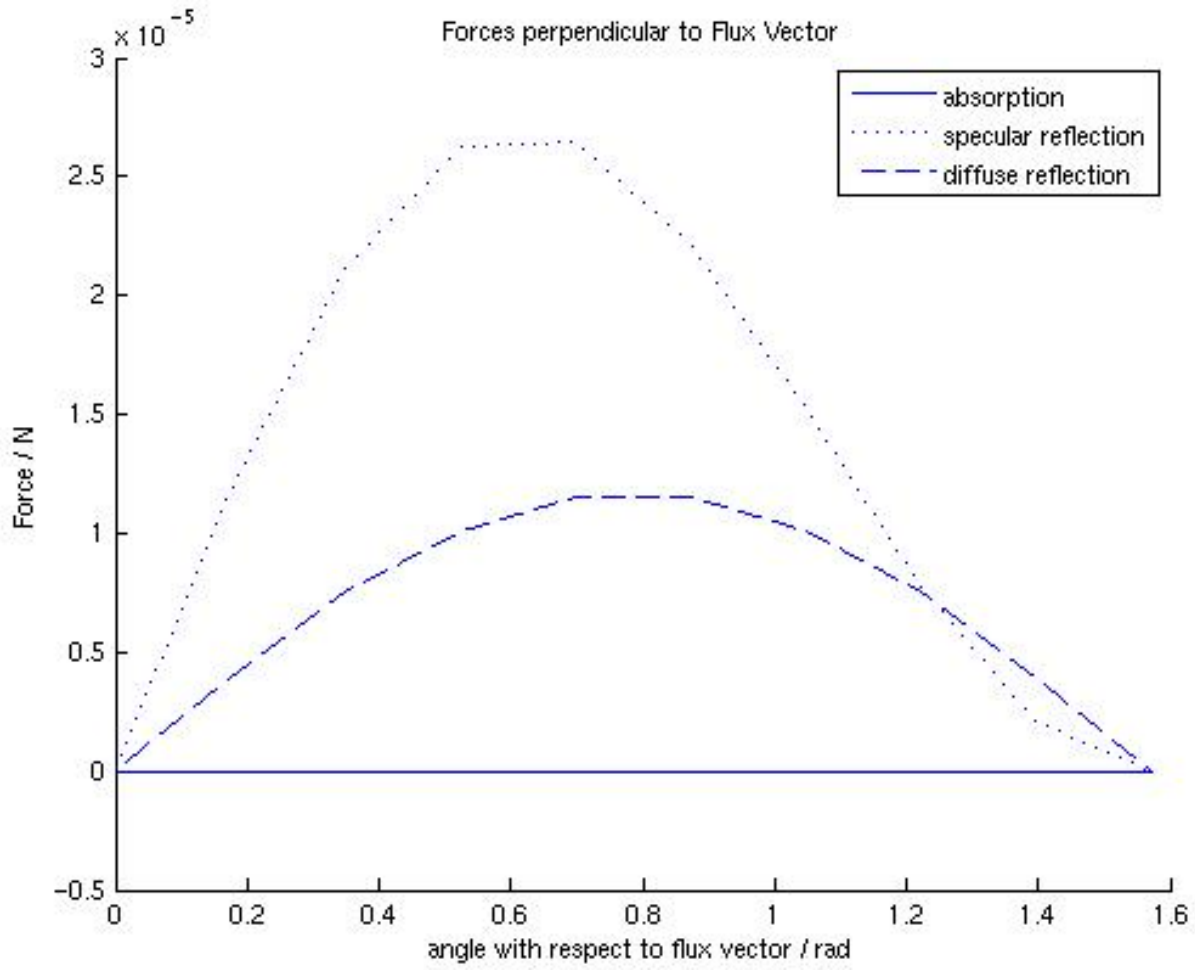


Figure 5.2: The components of the absorption, specular, and diffuse forces perpendicular to the flux vector as a function of the angle between the flux vector and the surface normal

5.1.5 Verification of the Temperature Integration and Thermal Emission Procedures

This section is divided into two parts, one comparing the case of the vehicle without illumination — for which an analytic solution exists — and the second comparing the response of surfaces with different characteristics in the presence of illumination.

5.1.5.a Verification of Thermal Processes without Illumination

Test RadiationPressure_3: Thermal Processes - no illumination

Purpose:

This test is to ensure that the Runge-Kutte plate-temperature integration process is functioning normally in the case of a vehicle cooling in the absence of incident radiation.

Requirements:

Satisfactory conclusion of this test, along with test RadiationPressure_4 satisfies requirements RadiationPressure_8 and RadiationPressure_4.

Satisfactory conclusion of this test, along with the tests RadiationPressure_4 and RadiationPressure_2 satisfies requirement RadiationPressure_7.

Procedure:

The simulations used in this test are available at *SIM_2_SHADOW_CALC/RUN_shadow_cooling* (for the regular Surface Model surface) and at *SIM_2A_SHADOW_CALC/RUN_shadow_cooling* (for the default surface). For the Surface Model, a number of plates were monitored in a flux-free environment to observe their temperature variation with time, and the effect of that temperature variation on the variation of the emissive force with time. Each plate had slightly different characteristics, allowing for the dependency on emissivity, plate area, and heat capacity to be investigated. For the second simulation, a default surface was placed in the same environment. In the absence of incident flux, an analytic solution exists for the temperature and the force, and this can be compared to the numerical solution.

Results:

1. Variation with time of force
   As expected, the absorption, diffuse, and specular forces were all zero for the duration of this simulation. The emission forces matched well with analytic predictions (see Figure 5.3, and had the correct orientation.
2. Variation with time of temperature
   Simulation data matched very well to analytic solution. On any scale longer than 3 timesteps, the graphs of analytical values and simulation values were indistinguishable, with a difference less than 0.05%, and very much smaller than the observed changes in temperature. See Figure 5.4.
3. Variation with emissivity of the temporal evolution of temperature
   As the emissivity increases, the rate of change of temperature also increases, as expected. For ϵ=0, no temperature change was observed.

4. Variation with starting temperature of the temporal evolution of temperature
   With a lower starting temperature, a plate maintains a lower temperature throughout the simulation, but the temperature decreases more slowly. Temperature∼time plots have identical shapes; shifting the curves along the x-axis to match temperature at some time results in matching temperatures for all (valid) time.
5. Variation with plate area of the temporal evolution of temperature
   A plate with a smaller surface area, but identical heat capacity shows a more gradual temperature drop. There is no observable difference between plates for which the heat capacity is decreased proportionally with the area (e.g. smaller plate of the same material).
6. Variation with heat capacity of the temporal evolution of temperature
   A plate with a smaller heat capacity, but identical area shows a more rapid temperature drop. There is no observable difference between plates for which the area is decreased proportionally with the heat capacity (e.g. smaller plate of the same material).

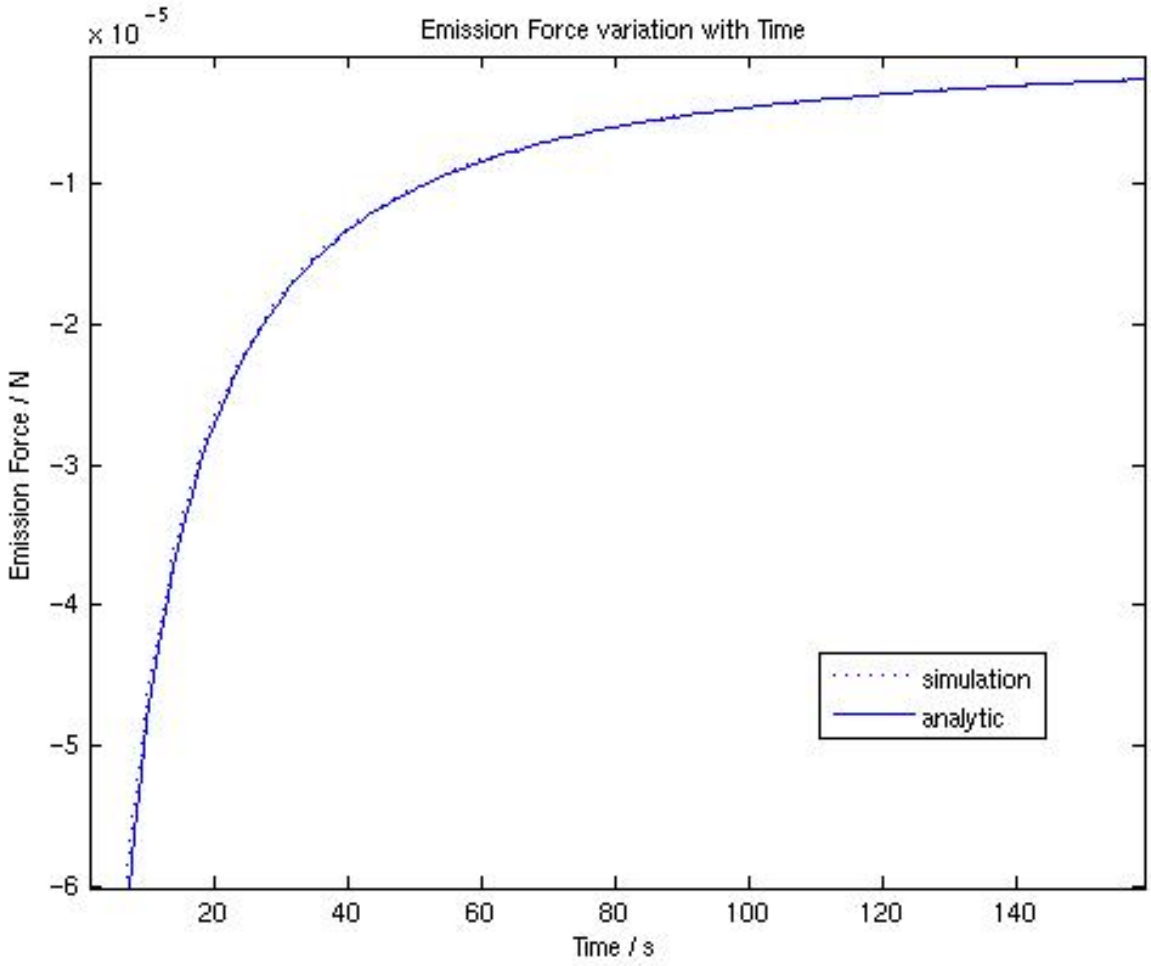


Figure 5.3: Variation with time of force, showing the simulation data and analytic solution overlaying.

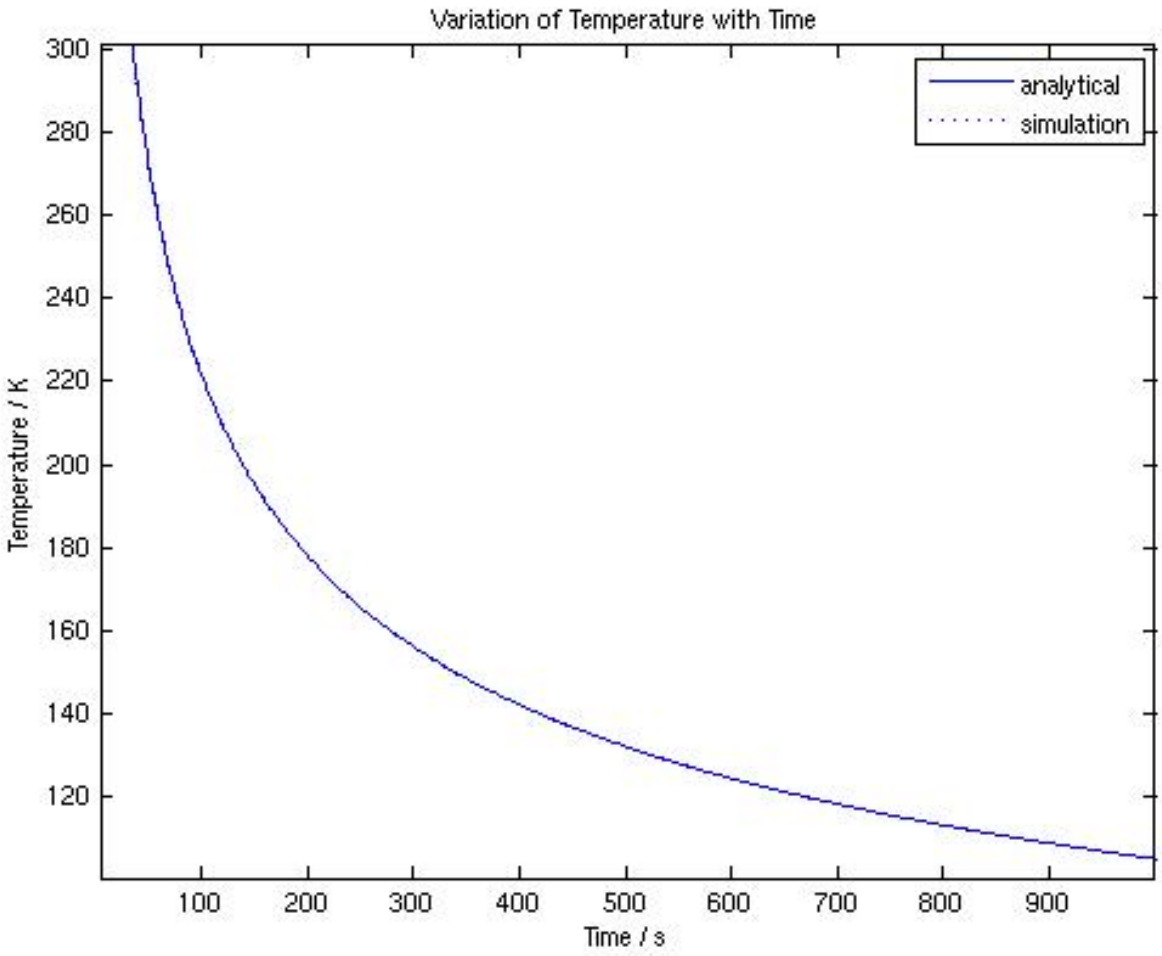


Figure 5.4: Variation with time of temperature, showing the simulation data and analytic solution overlaying.

5.1.5.b Verification of Thermal Processes with Illumination

Test RadiationPressure_4: Thermal Processes - with illumination

Purpose:

To verify that the temperature integration and thermal emission processes are functioning as expected when the vehicle is illuminated.

Requirements:

Satisfactory conclusion of this test, along with test RadiationPressure_3 satisfies requirements RadiationPressure_8 and RadiationPressure_4.

Satisfactory conclusion of this test, along with the tests RadiationPressure_3 and RadiationPressure_2 satisfies requirement RadiationPressure_7.

Procedure:

The simulation used in this test is available at *SIM_2_SHADOW_CALC/RUN_ten_plates*. Ten identical plates were oriented at different angles to the flux vector, with the angle between the normal and the flux vector varying from 0 to 90 degrees. All plates were started with a preliminary temperature of 270 K, and the simulation was allowed to run until all but one plate had approached their equilibrium temperature. The force from each plate was monitored throughout the simulation.

Results:

The equilibrium temperature is calculated from the energy balance equation

$$\phi(A\cos\theta)(1-\alpha) = \epsilon\sigma A{T_{eq}}^4 \tag{5.6}$$

where

- ϕ is the radiative flux,
- θ is the angle between the surface normal and the incident radiation vector,
- α is the albedo of the surface,
- ϵ is the emissivity of the surface,
- σ is the Stefan-Boltzmann constant,
- A is the plate area, and
- T_{eq} is the equilibrium temperature.

Using values of $\alpha = 0.5$, $\epsilon = 0.5$, $\phi = 1400\ W\ m^{-2}$, and with θ varying from 0 to 90 degrees in 10 degree intervals, the equilibrium temperatures of the 10 plates are:

396 K, 395 K, 390 K, 382 K, 371 K, 355 K, 333 K, 303 K, 256 K, and 0 K.

Figure 5.5 shows the variation with time of the temperature of the 10 plates; the 9 plates that had closely approached equilibrium temperatures had final temperatures all within 1% of their analytical equilibrium temperature.

The final plate, with an equilibrium temperature of 0 K has a temperature variation that is decreasing at a rate consistent with expectations.

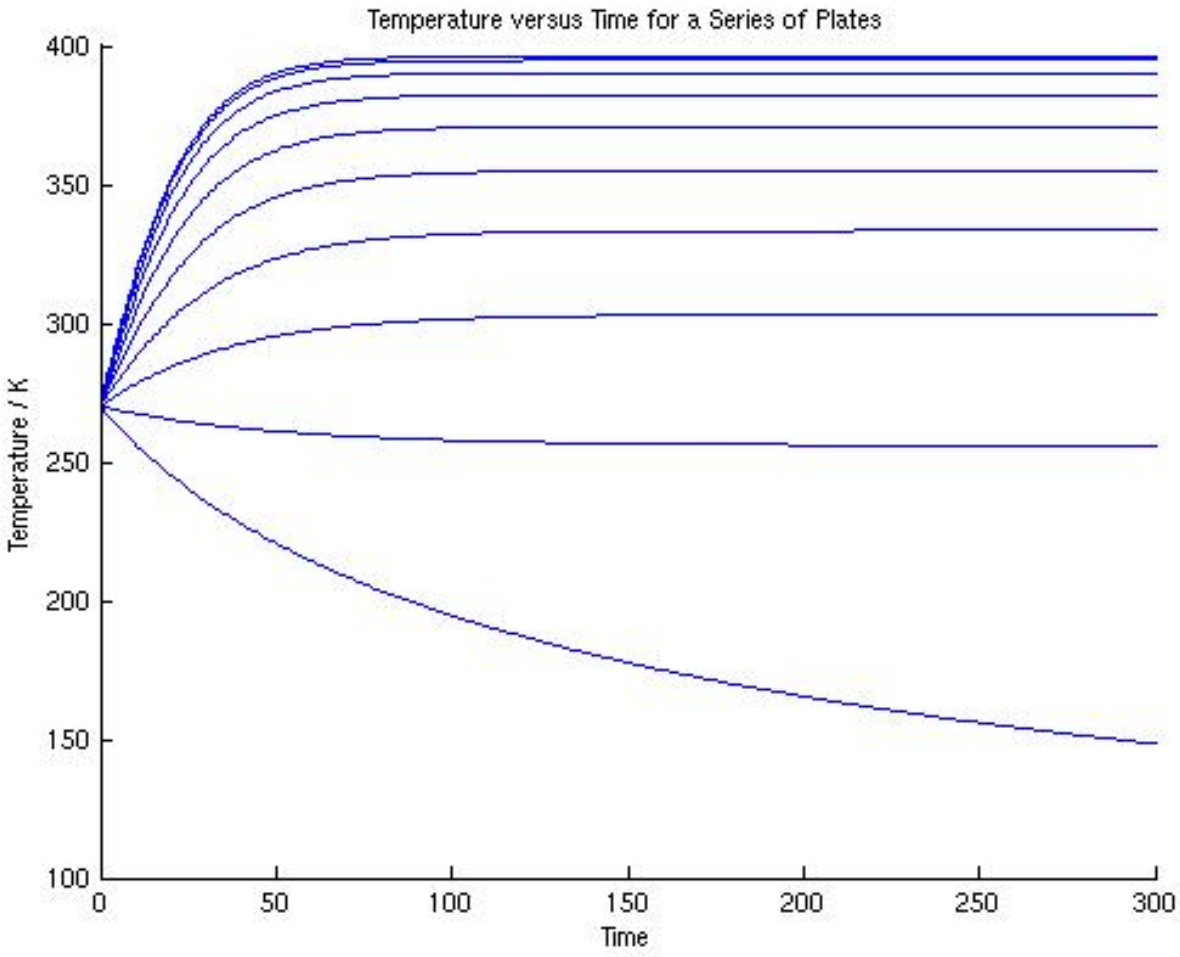


Figure 5.5: The variation of temperature with time over the 10 plates in the simulation.

The force due to emission should be directed opposite to the surface normal, and have magnitude equal to the ratio of power emission to speed of light, producing equilibrium values of 47, 46, 44, 40, 36, 30, 23, 16, 8, and 0 μN.

Figure 5.6 shows the variation with time of the force due to emission, confirming the analytic values.

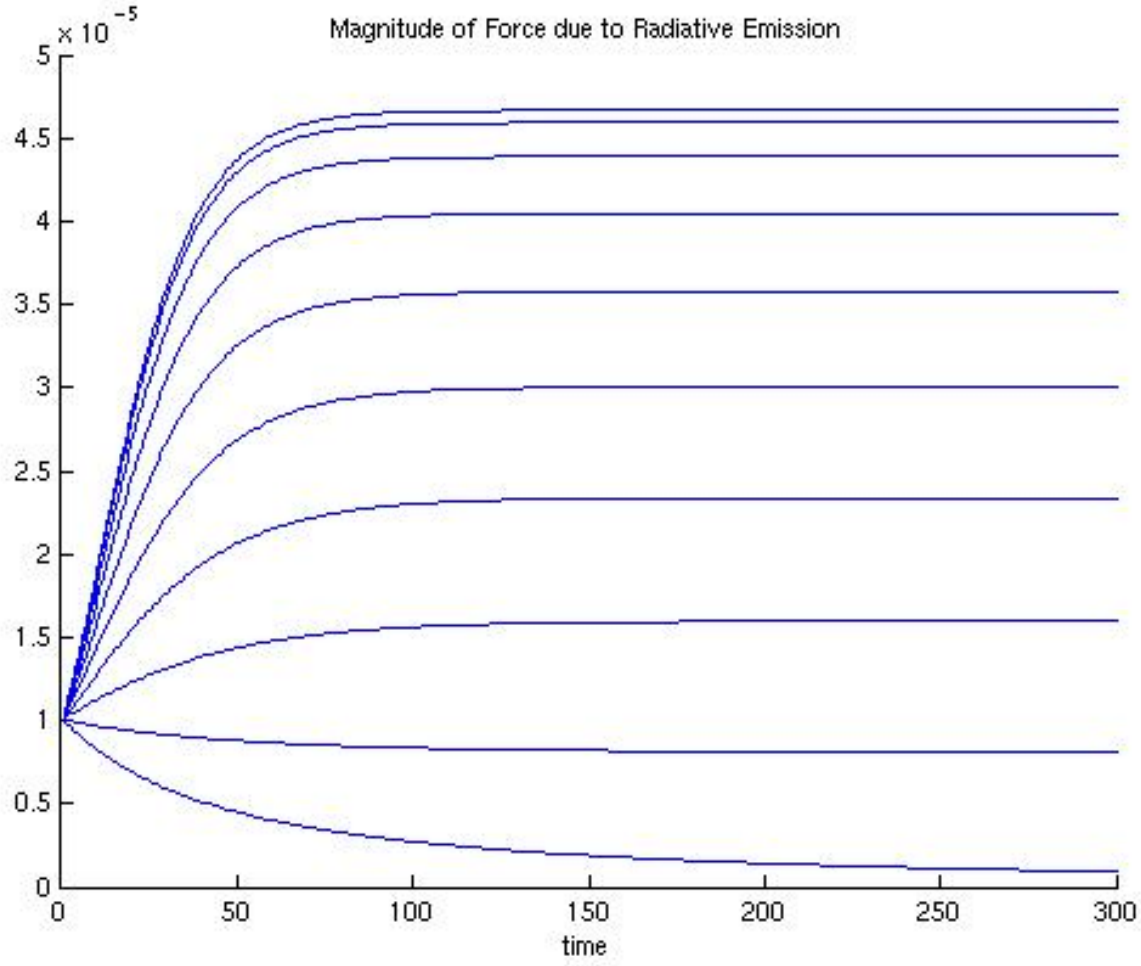


Figure 5.6: The variation with time of force, over the 10 plates in the simulation.

5.1.6 Verification of the Flux Calculations

This section is divided into two parts: the first considers a vehicle in Earth's shadow at several locations along the extension of the Sun-Earth vector, the second considers a vehicle moving perpendicular to the Sun-Earth vector, passing through Earth's shadow.

5.1.6.a Eclipse calculations along the extension of the Sun-Earth vector

Test RadiationPressure_5: Verification of the eclipse calculation for an object moving along the Sun-planet axis through the region of annular eclipse

Purpose:

The purpose of this simulation is to confirm that the code correctly identifies and calculates the extent of shadowing by an intervening body.

Requirements:

The results of this test, together with the results from RadiationPressure_6, together satisfy requirement RadiationPressure_10.

Procedure:

The simulation used in this test is available at *SIM_2_SHADOW_CALC/RUN_annular_eclipse*. The vehicle was placed at various locations along the dotted line shown in figure 5.7.

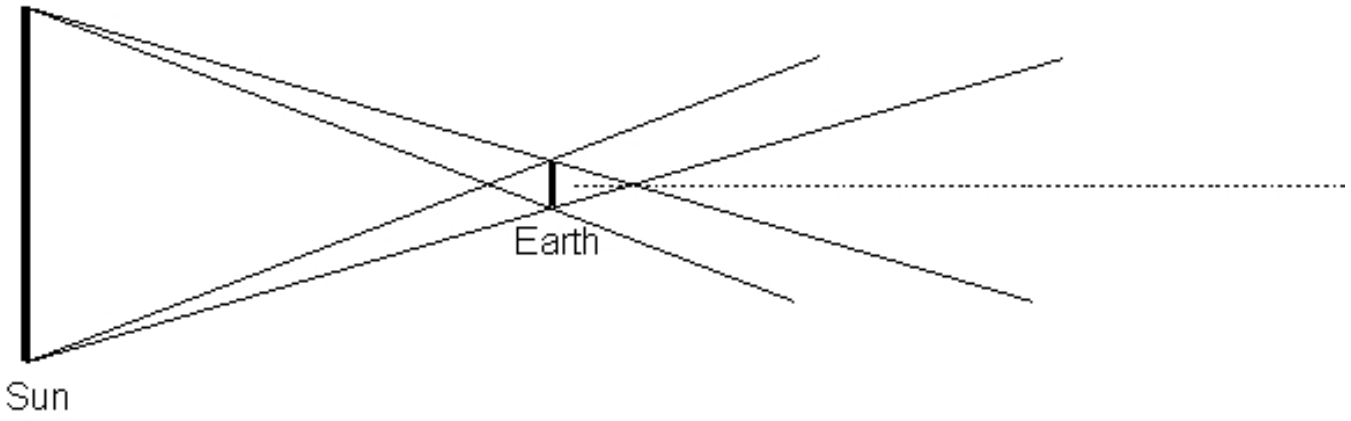


Figure 5.7: The location of the vehicle, with respect to Earth and Sun, during the simulation.

Two runs were made:

1. the first used the cylindrical shadow model,
2. the second used the conical shadow model.

The value of the flux vector was evaluated at several locations for each run.

Results:

Both runs produced results entirely in line with expectations.

1. For the first run (cylindrical shadow), there was no illumination at any location on the line.
2. For the second run (conical shadow), there was no illumination while the vehicle was inside the cone representing totality. At larger distances, the magnitude of the flux increased as a greater percentage of the solar disk became visible; at very large distances from Earth ($> 10^{10}$ m), the flux magnitude decreased with distance due to the inverse-square nature of the flux magnitude and the fact that the fraction of the solar disk that is exposed changes only very slowly at these distances. The data matched the theoretical predictions very closely.

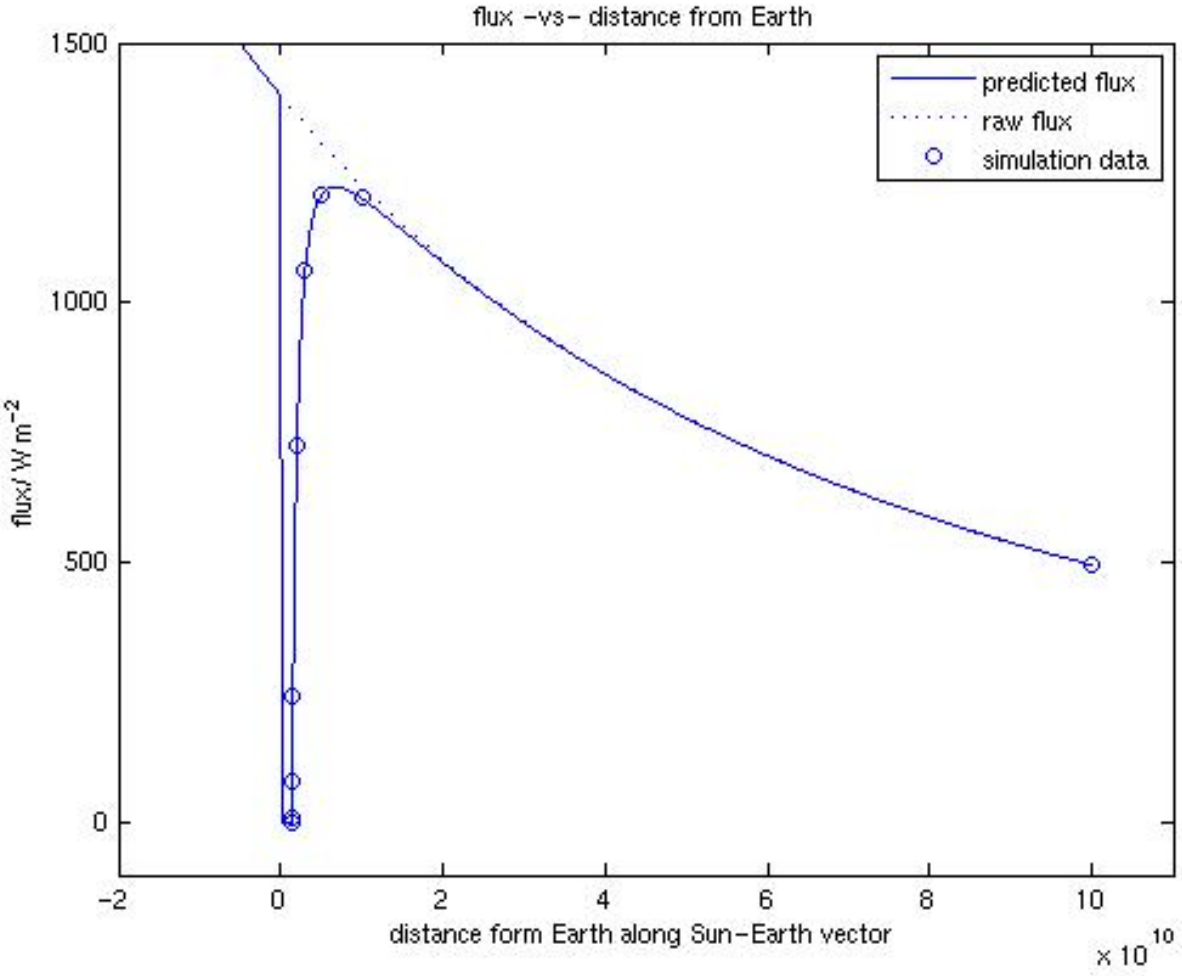


Figure 5.8: The flux of the vehicle as a function of distance from Earth on the Sun-Earth vector. The raw flux represents the solar flux without shadowing. The predicted flux represents the analytic function for flux with Earth shadowing included. Data points from the simulation are shown as circles.

5.1.6.b Eclipse calculations perpendicular to the extension of the Sun-Earth vector

Test RadiationPressure_6: Verification of the eclipse calculation for an object moving across the region of total eclipse

Purpose:

The purpose of this simulation is to confirm that the code correctly identifies and calculates the extent of shadowing by an intervening body.

Requirements:

The results of this test satisfy requirement RadiationPressure_9, and together with the results from RadiationPressure_5, satisfy requirement RadiationPressure_10.

Procedure:

The simulation used in this test is available at *SIM_2_SHADOW_CALC/RUN_transverse_shadow*. Two different techniques were used to investigate the response to the position within the shadow of the planet.

1. A series of runs was made with the vehicle traversing the shadow, perpendicular to the Sun-Earth vector at a selection of values of r_{par}, the component of the Earth-vehicle vector parallel to the Sun-Earth vector, as shown in Figure 5.9. The simulation included in the JEOD v5.4 release corresponds to line number 4 in this description. With an Earth-vehicle distance of $1.35 \times 10^9 m$, this corresponds to the apex of the cone representing totality. Line number 1 in this description is actually located at a distance of $1.35 \times 10^4 m$ from the center of Earth. This value is well within the planet; values closer to the planet center had complicating issues associated with Earth moving on its orbit during the simulation in such a way that the position of the vehicle was closer to Sun than was Earth.

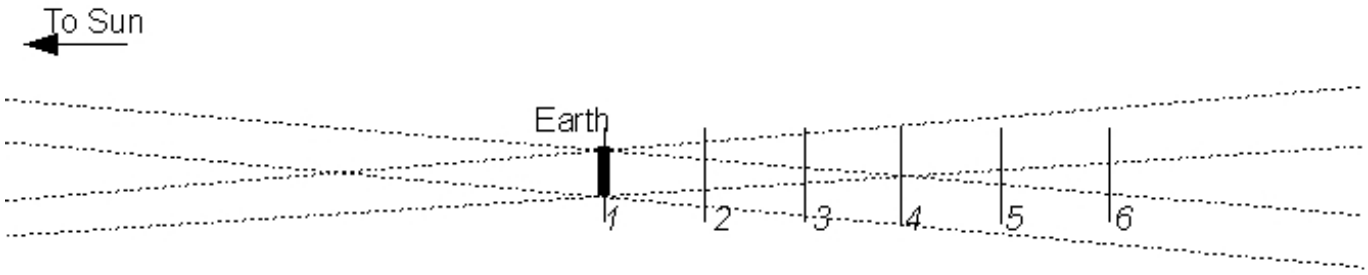


Figure 5.9: The 6 paths of the vehicle across Earth's shadow

2. Another run was made in which the inertial position of the vehicle was held fixed with respect to Earth, and the ephemeris allowed to evolve. Starting with the vehicle in the conical shadow, it gradually moved into full sunlight as the earth moved around its orbit, as shown in Figure 5.10.

Results:

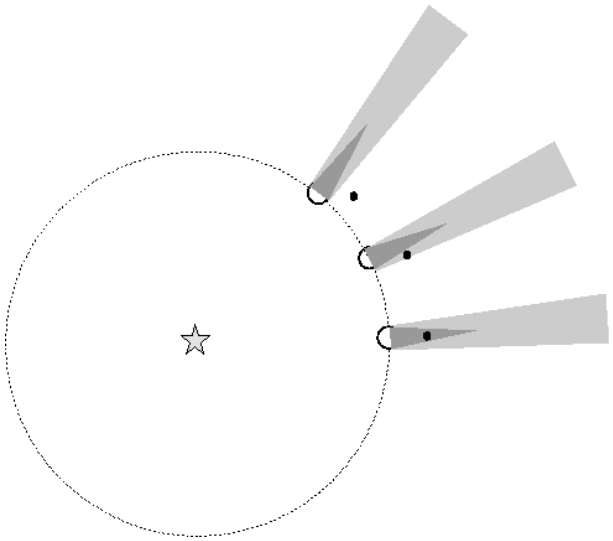


Figure 5.10: The position of the vehicle with respect to Sun as it transits Earth's shadow.

1. In the first run (vehicle passes through Earth shadow), the Sun-vehicle vector changes in both direction and magnitude, thus affecting the flux unit vector (flux_hat), the magnitude of the flux vector, and each individual component of the flux vector; these changes were consistent with theory.

   The six lines in Figure 5.11 each represent the magnitude of the force experienced by a vehicle as it traverses Earth's shadow. Each line represents a different value of r_{par} and is shown as a function of r_{perp}, the component of the Earth-vehicle vector that is perpendicular to the Sun-Earth vector.

   For passes that were sufficiently close to Earth that the vehicle passed through the region of totality (i.e. passes 1-3, with pass 4 satisfying this requirement only momentarily), we see the effects of both umbral and penumbral shadowing. The penumbral (partial shadowing) region is indicated by a sloping line; the umbral (total shadowing) region is indicated by a flat line and zero force.

   The first line (corresponding to a vehicle passing close to the center of Earth) shows no penumbral region, and an umbral region equal in size to Earth. Getting farther from Sun, the extent of the penumbral region grows, and that of the umbral region shrinks until it disappears by the fourth line, which is the limit of total eclipse. Farther yet from Earth, passes 5 and 6 represent annular eclipses, with the angular size of Earth now smaller than that of the sun. Annular eclipses do not have umbral regions; instead the flux should fall until the intervening body is completely within the solar disk, then stay constant as the intervening body traverses the solar disk, then rise again as it leaves the disk. Pass 6 in particular shows the annular eclipse lasting for some distance at constant illumination. Also, evident in the wings of the first three lines, beyond the points of total illumination, is the gradual decrease in illumination due to increased distance from Sun.

2. The second simulation (shadow moves with respect to a fixed-orientation of Earth and vehicle) demonstrated very similar results, with almost-negligible changes in the Sun-vehicle distance, and hence in the raw flux magnitude throughout the simulation.

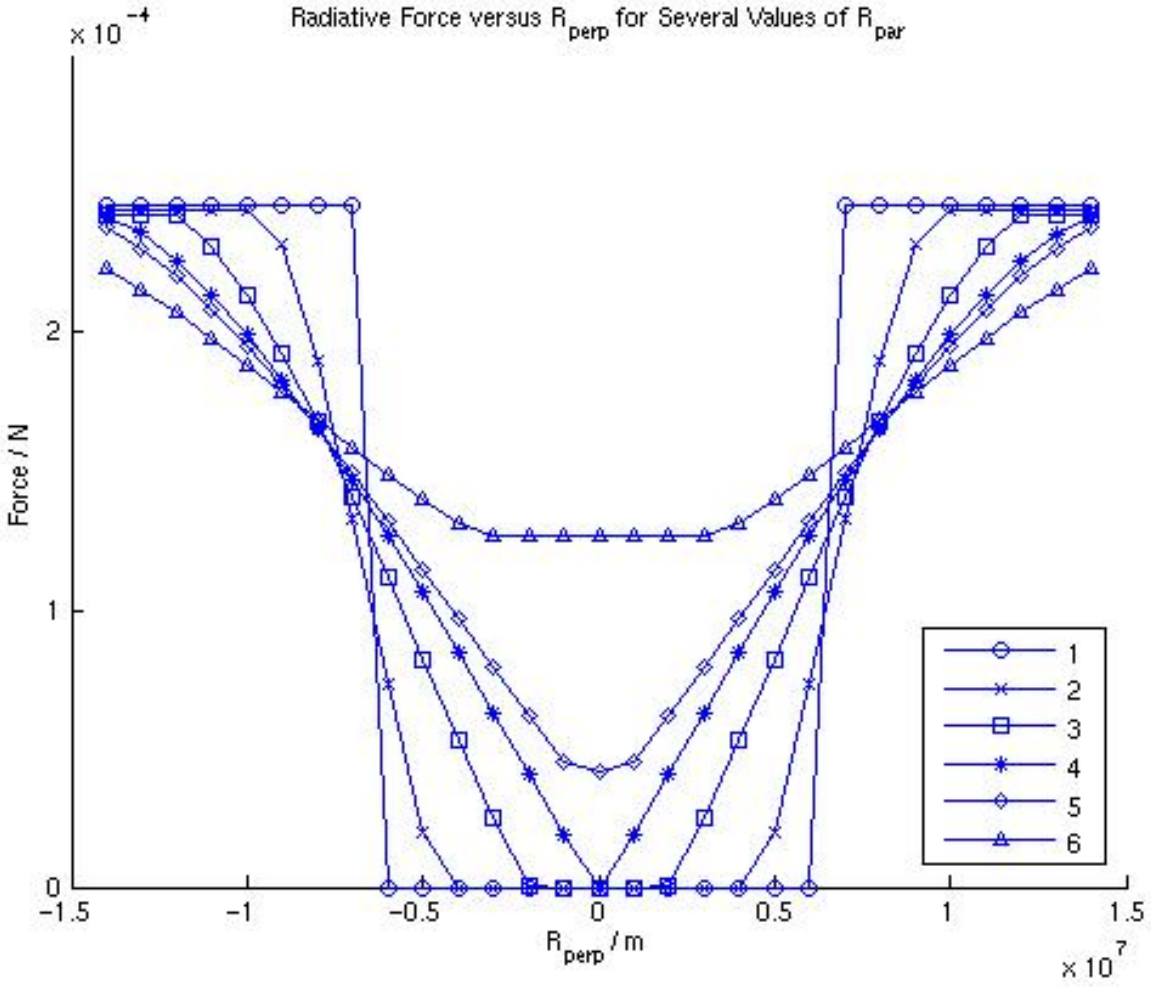


Figure 5.11: The magnitude of the radiative force as a function of r_{perp} for different values of r_{par}

5.1.6.c Confirmation of eclipsing bodies

Inspection RadiationPressure_5: Vehicle can be shadowed by any intervening body.

The functionality to calculate the effect of an eclipsing body is demonstrated in tests RadiationPressure_5 on page 97 and RadiationPressure_6 on page 99. The flexibility to include shadowing of more than one body is found in *RadiationSource::calculate_flux*, although this is not complete; where two intervening bodies overlap, the code will calculate the fractional part of the disk that is visible behind each (e.g. 90% and 80%) and multiply these together (72%). If the two bodies are blocking the same part of the disk, this is an invalid number. For a fully correct implementation, the "shadowing" of third-bodies by third-bodies also needs to be considered. Nevertheless, the requirement is for this functionality to be available as an extension, and in that sense, this inspection satisfies requirement RadiationPressure_11.

5.2 Validation

5.2.1 Validation of Flux-Vehicle Interaction-Model Principles

Test RadiationPressure_7: Flux-Vehicle Interaction Model Principles

Purpose:

The purpose of this test is to validate that radiation-interaction can be modeled to reasonable accuracy by a combination of specular reflection, diffuse reflection, absorption, and emission.

Requirements:

By passing this test, the model satisfies requirement RadiationPressure_7.

Procedure:

A number of test surfaces were run. Figure 5.12 shows the reflections for a series of four surfaces of progressively increasing roughness (and hence higher δ, the weighting towards diffuse reflection). Each of these surfaces was tested with 3 different albedos, across a range of incident angles. For each incident angle, 10^6 "photons" were "fired" at the surface and the subsequent reflection or absorption recorded (see Figure 5.12 for reflection characteristics).

The vectors associated with photon reflections were averaged and compared to the theoretical average for thin-film interactions, equation 3.3 on page 28.

$$\langle \hat{s} \rangle = \begin{bmatrix} -0.5 \sin\phi \left(1 + e^{-\frac{1}{c}}\right) \\ 0 \\ \cos\phi \, e^{-\frac{1}{c}} \end{bmatrix}$$

This theoretical value includes the possibility that a photon could be reflected downward as its terminal state, while the simulation took downward reflected photons and operated on them again until they were either absorbed by the surface, or reflected away from the surface.

In this representation, the x-axis is the negative of the projection of the incident radiation onto the surface; the z-axis is perpendicular to the surface, pointing outward the y-axis is parallel to the surface, and completes the right-hand coordinate system; c represents the "smoothness" of the surface.

The cumulative vector force calculated from the interactions of all photons for any given angle and surface was then compared with a best-fit bulk approximation that assigned some fraction of the incident photons to specular reflection, some to diffuse reflection, and some to absorption and subsequent emission.

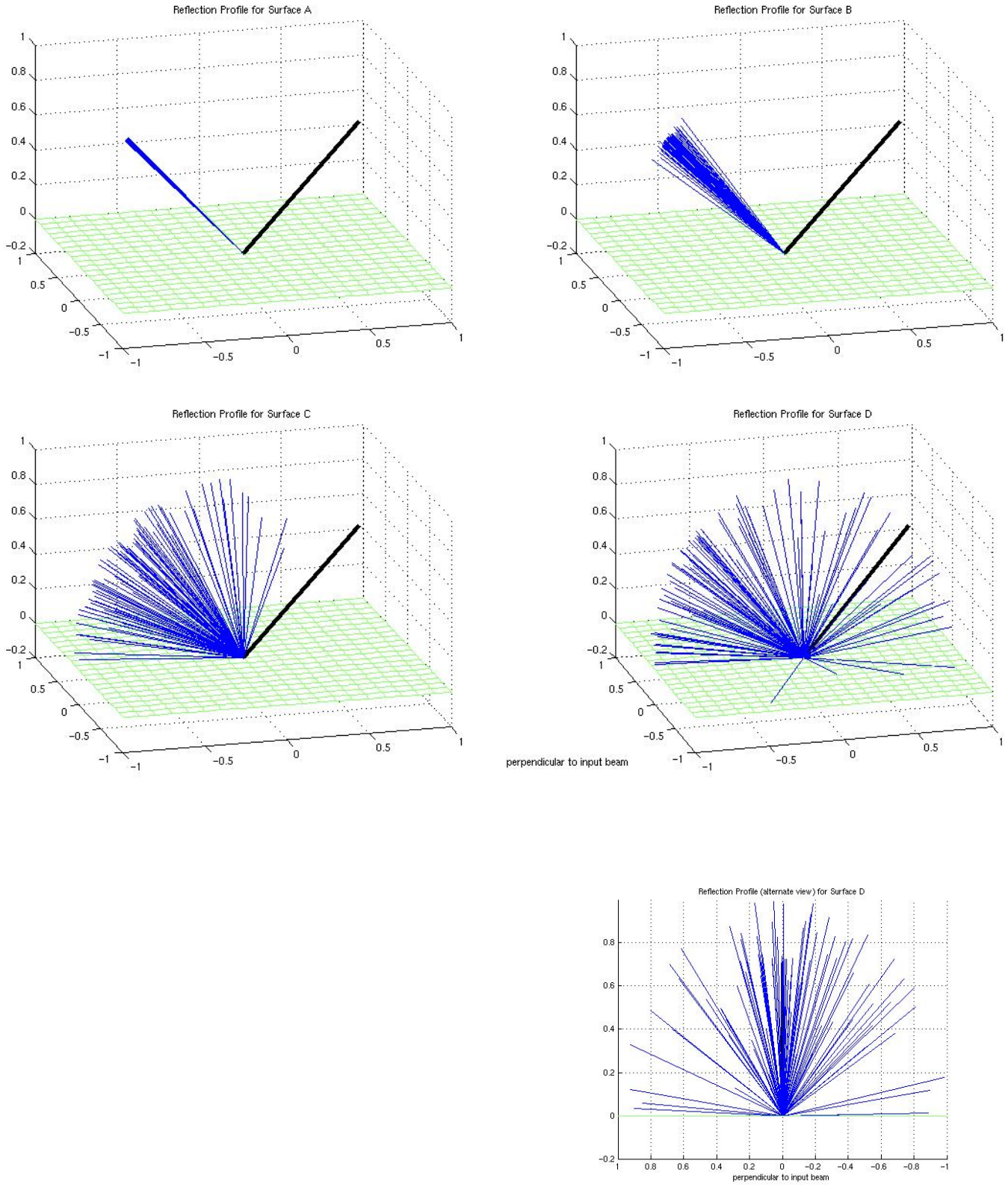


Figure 5.12: Reflection characteristics from four surfaces

Results:

The results section is divided into two parts: a comparison of the values for the unit-vector associated with reflection, and a comparison of the values for the force.

5.2.1.a Unit-vector Comparisons

The average unit-reflection-vector, $\langle \hat{s} \rangle$, was calculated from the numerical simulation and compared to the analytic value given above. Figure 5.13 shows the comparison between the numerical and analytical value for each of the three components of $\langle \hat{s} \rangle$ for surface #1.

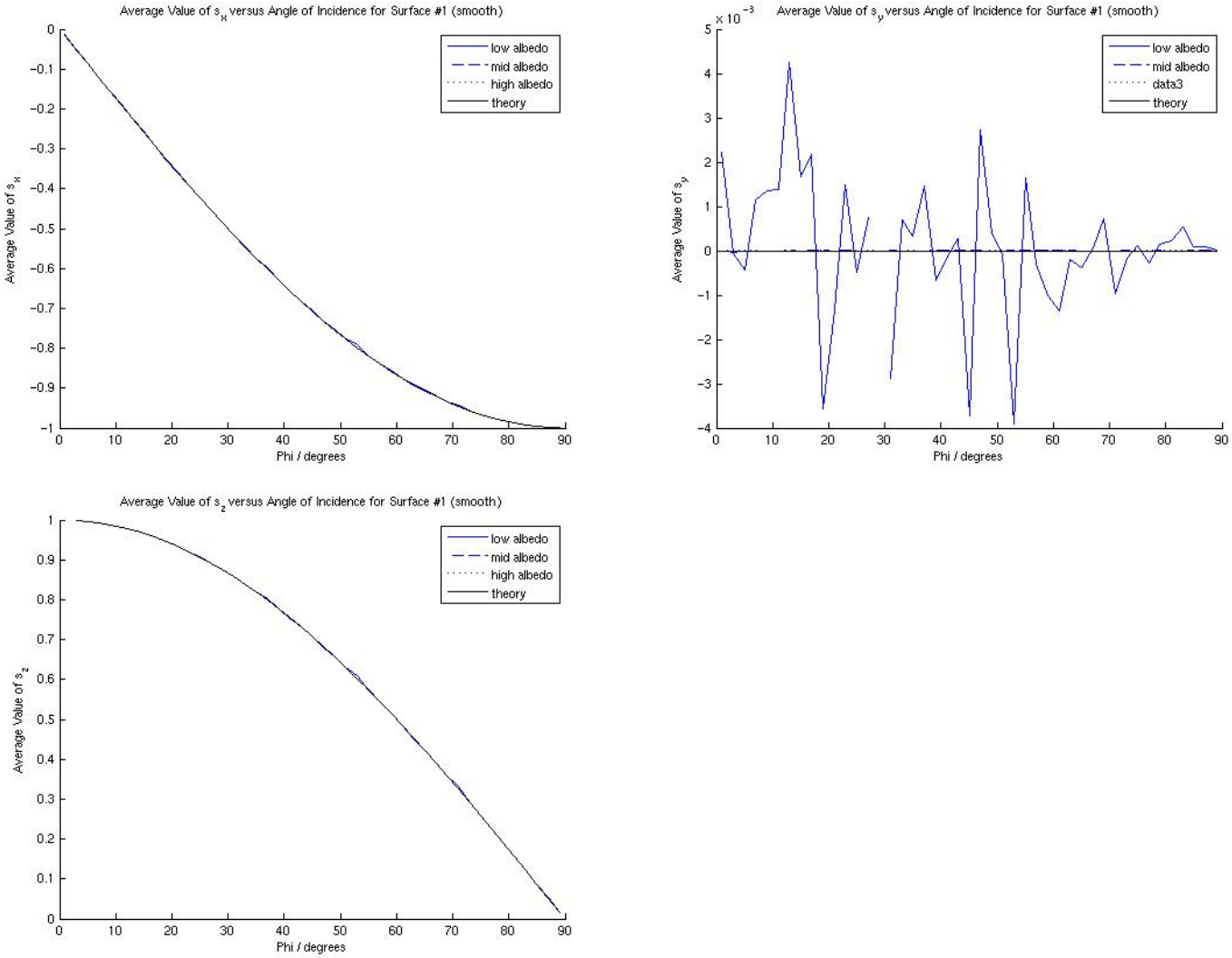


Figure 5.13: Analytical and numerical mean values for components of the reflection vector from a smooth surface as a function of incident angle

Figures 5.14, 5.15, and 5.16 show the differences along each axis for each of the four surfaces shown in Figure 5.12.

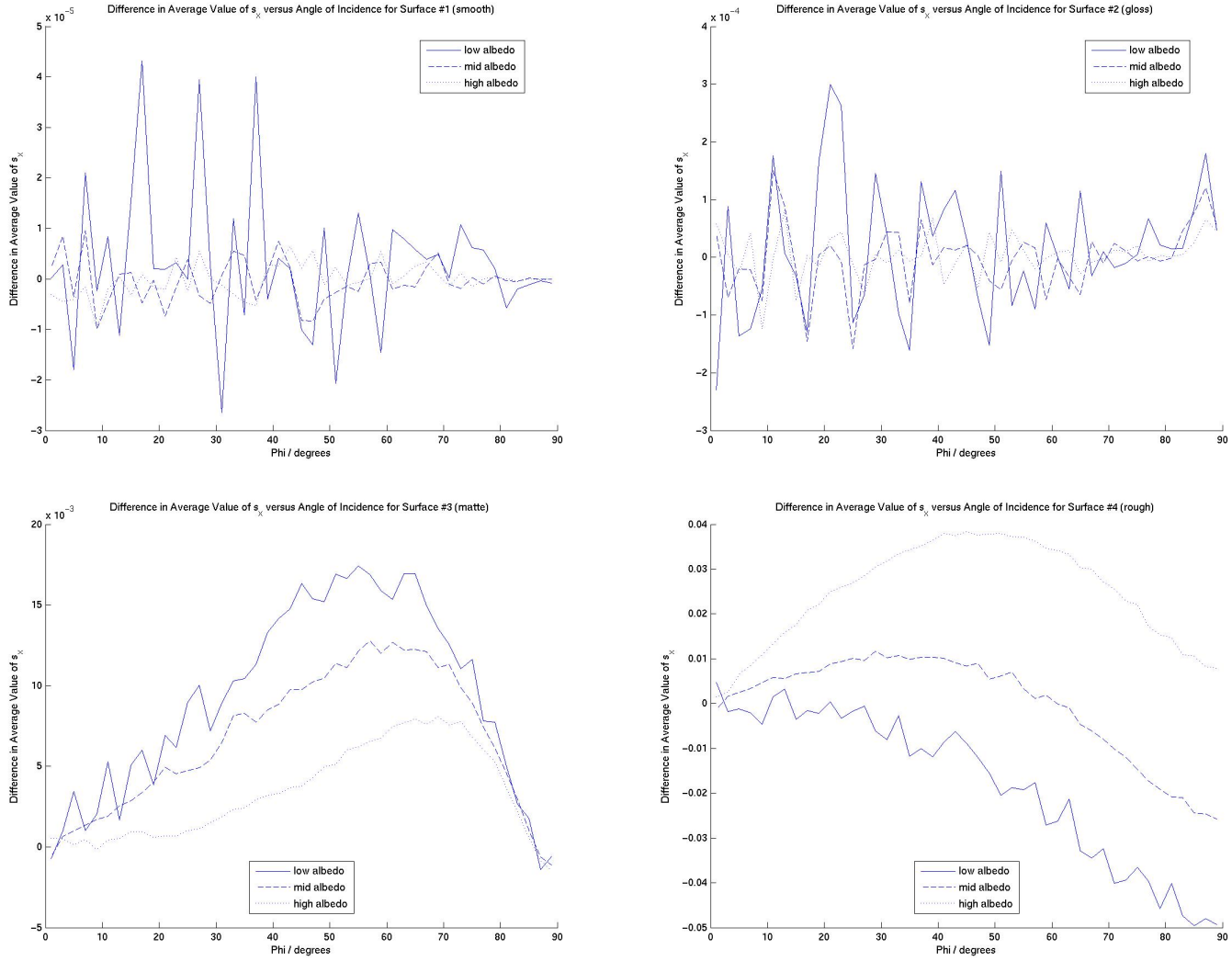


Figure 5.14: Differences between the analytic and numeric x-component of $\langle \hat{s} \rangle$ for each of the 4 surface-profiles.

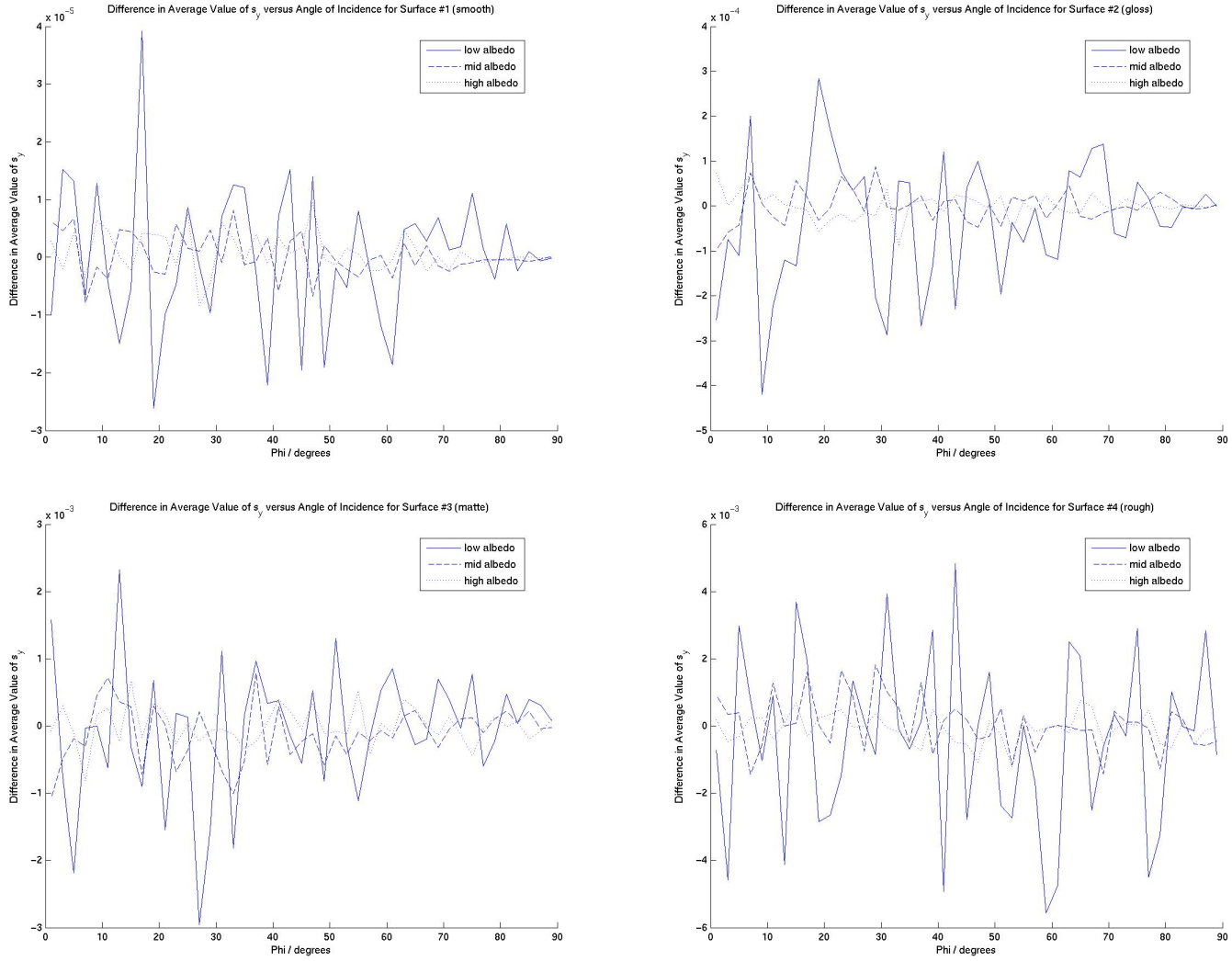


Figure 5.15: Differences between the analytic and numeric y-component of $\langle \hat{s} \rangle$ for each of the 4 surface-profiles.

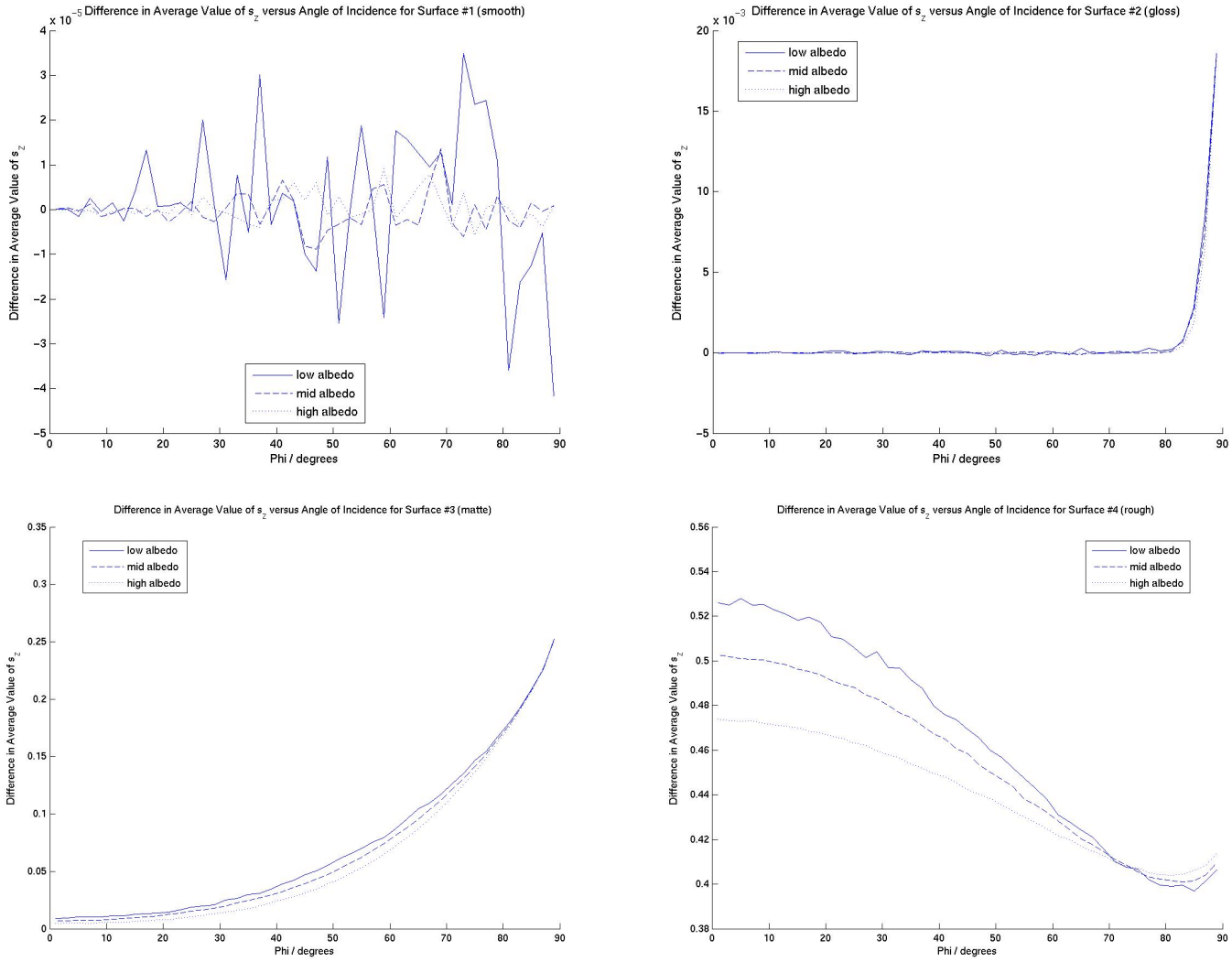


Figure 5.16: Differences between the analytic and numeric z-component of $\langle \hat{s} \rangle$ for each of the 4 surface-profiles.

5.2.2 Force Comparisons

For each angle of incidence, the total force calculated by each of the two models can be expressed as:

1. Numerical Model

$$\vec{F}_{num} = \sum_{i=1}^{N} d\vec{F}_i \tag{5.7}$$

where

- $d\vec{F}_i$ $=$
$$\begin{cases} f_\gamma\left(-\hat{p} - \hat{s}_i\right) & photon\ is\ reflected \\ f_\gamma\left(-\hat{p} - \frac{2}{3}\hat{k}\right) & photon\ is\ absorbed\ and\ re-emitted,\ or\ reflected\ diffusely \end{cases}$$
- $\hat{p}$,$\hat{s}$, and $\hat{k}$ are unit vectors for the incident radiation, specular reflection, and surface-normal vectors (Note: $\hat{p}$ is directed *against* the flux vector, away from the surface).
- f_γ represents the "force" associated with the momentum of each incident photon per unit time.
- N represents the number of "photons" being used.

2. Simplifying Model

$$\vec{F}_{model} = N f_\gamma \left(-\hat{p} - \alpha(1-\delta)\hat{s} - \frac{2}{3}(1-\alpha(1-\delta))\hat{k}\right) \tag{5.8}$$

where α and δ represent the fraction of incident light that is reflected, and the fraction of the reflected light that is reflected diffusely.

Equation 5.7 is evaluated for each angle of incidence, for each surface by calculating the force from each photon, and summing to provide the total force in each direction.

For any given surface, the force is calculated for a number of incident angles, then the values α and δ, that provide the best fit — across all of those angles — between equations 5.7 and 5.8, are determined.

First the difference, *per photon,* between equations 5.7 and 5.8 is calculated for some angle of incidence , ϕ_i :

$$(\Delta\vec{F}_{\phi_i}) = \frac{\vec{F}_{num} - \vec{F}_{model}}{N} \tag{5.9}$$

The RMS value is used for determining "best fit".

$$\Delta F_{RMS} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} |\Delta\vec{F}_{\phi_i}|^2} \tag{5.10}$$

where n represents the number of angles of incidence used in the numerical simulation.

The contribution to this value from each axis are also calculated for analysis.

$$\Delta F_x = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left((\Delta\vec{F}_{\phi_i})\cdot\hat{i}\right)^2} \tag{5.11}$$

$$\Delta F_y = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left((\Delta\vec{F}_{\phi_i})\cdot\hat{j}\right)^2} \tag{5.12}$$

$$\Delta F_z = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left((\Delta\vec{F}_{\phi_i})\cdot\hat{k}\right)^2} \tag{5.13}$$

Values of α and δ which minimize ΔF_{RMS} are now determined, and those values used in equation 5.8 to obtain a "best" estimate of the variation with incident angle of the force. The comparison of the numerical simulation to the best fit is shown in figures 5.17, 5.18, and 5.19. Note that in places, the best fit line is wholly above or below the data; this is because the data are showing 1 dimension at a time, whereas the best fit line is based on all dimensions together.

Notice that for all surfaces the behavior in the y-direction is small and unpredictable. For the smooth surface (surface #1), the x- and z- components of the force behave as expected. In the x-direction, a highly reflective surface should produce no force, while a highly absorbent surface should absorb all of the momentum from the incident radiation, which varies as $\sin\phi$. In the z-direction, the highly reflective surface should experience a force of magnitude $\cos\phi$ from each of incident and reflected radiation components, resulting in a curve that decreases from 2 at $\phi = 0$ to 0 at $\phi = 90$ degrees; the highly absorbent surface will experience the $\cos\phi$ force from the incident radiation, and a force with magnitude 2/3 from the re-emitted radiation, causing the curve to fall from 1.67 to 0.67. The forces in the y-direction should be generally small compared with those in the x- and z-direction. These features are all observed. In all cases, values of α and δ were found that provided a close fit between the numerical data and the simplified expression given in equation 5.8.

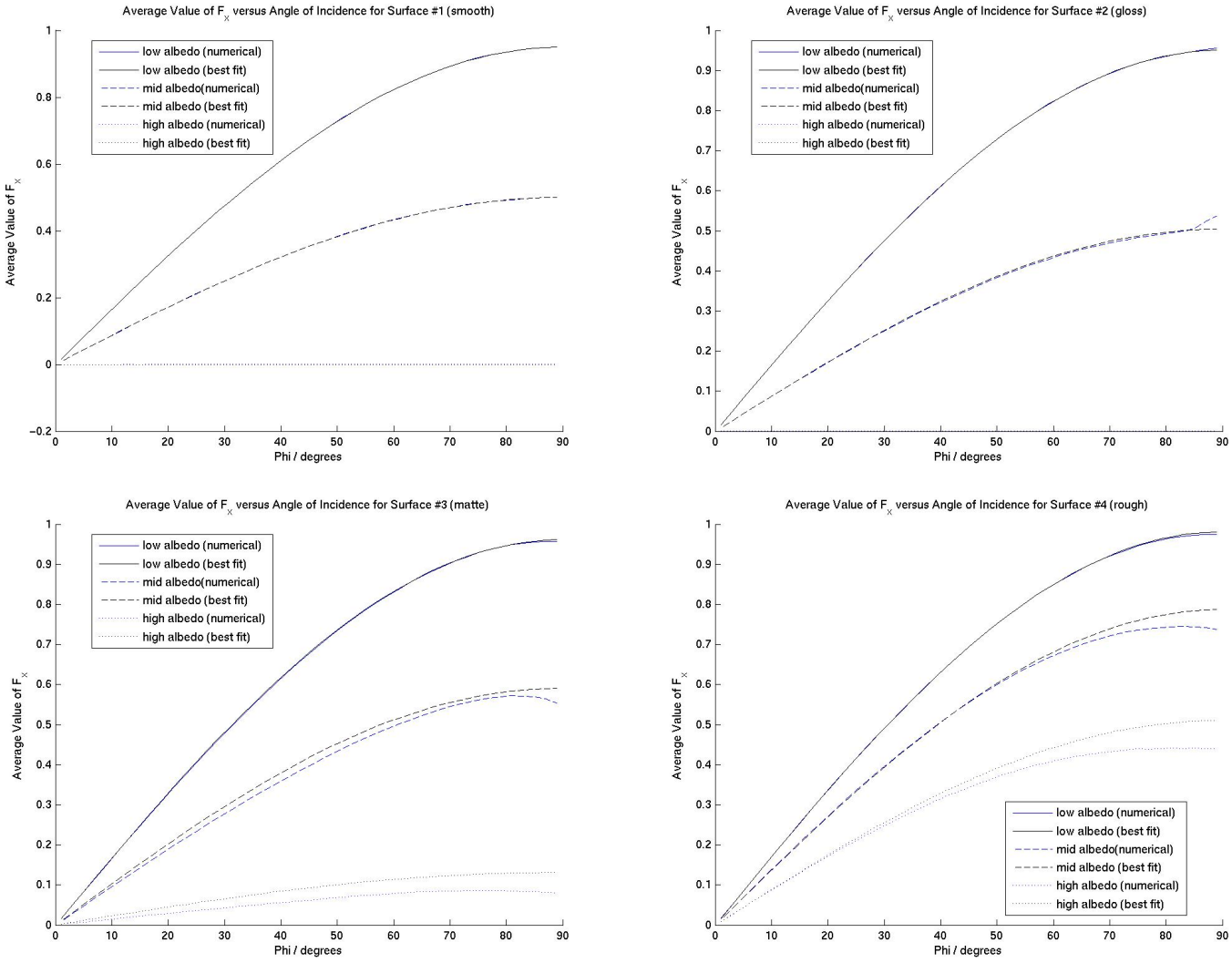


Figure 5.17: Mean values for the analytic and numeric x-component of the interaction force for each of the 4 surface-profiles.

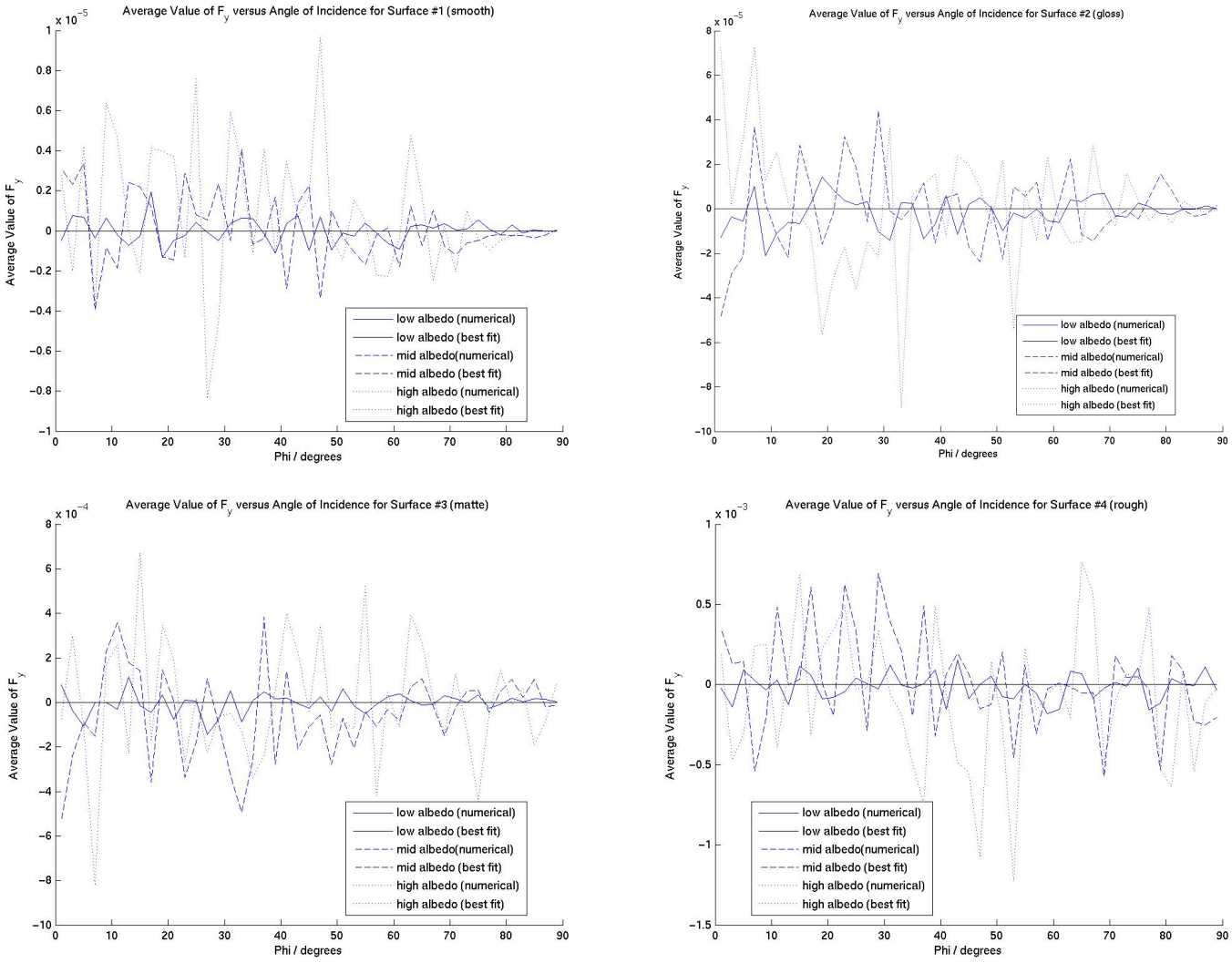


Figure 5.18: Mean values for the analytic and numeric y-component of the interaction force for each of the 4 surface-profiles.

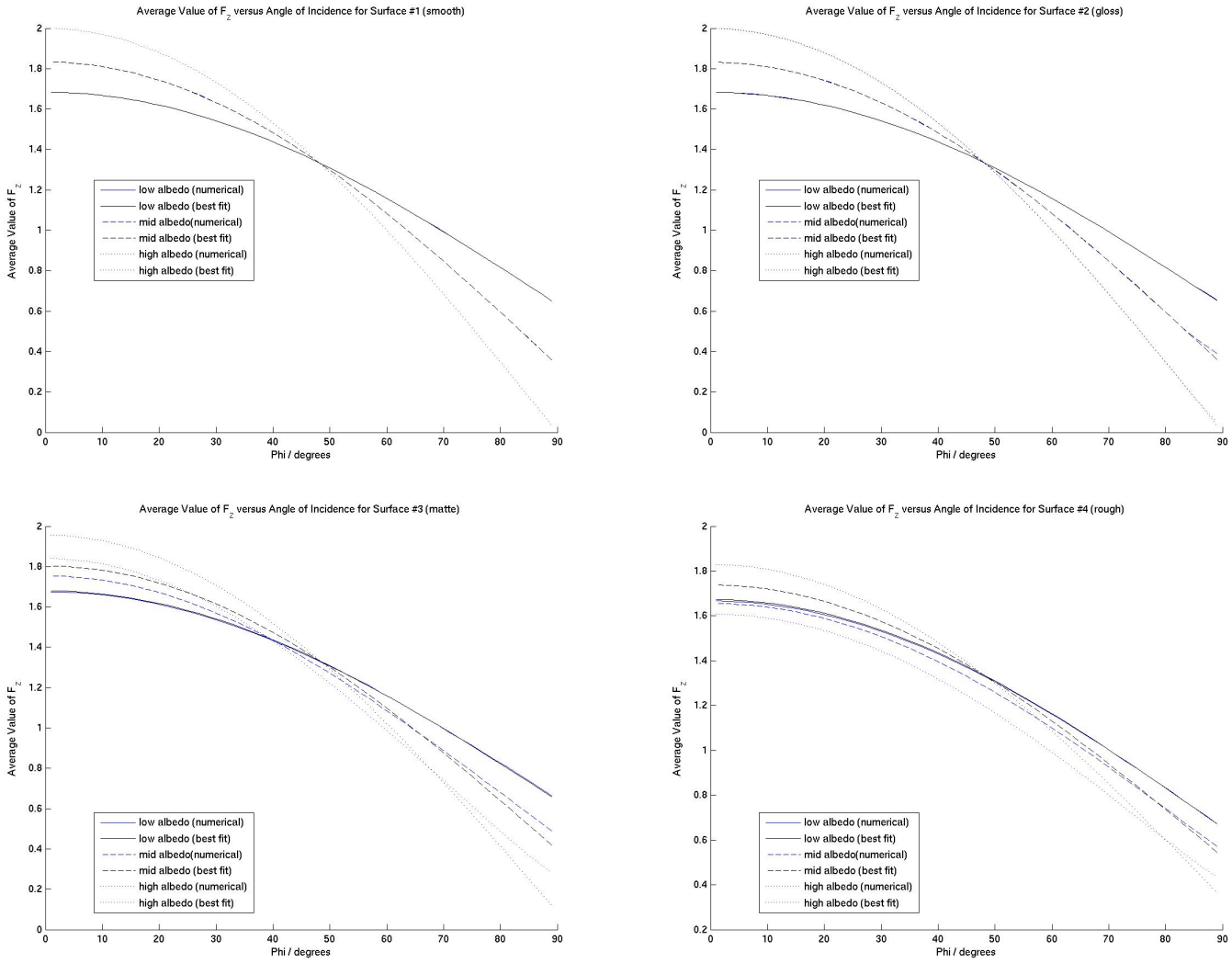


Figure 5.19: Mean values for the analytic and numeric z-component of the interaction force for each of the 4 surface-profiles.

On closer inspection, under certain circumstances, the best fit of α and δ to the data did not seem sensible. To investigate the degree to which the determination of the albedo and diffuse parameters are critical, plots have been generated showing the deviations ΔF_{RMS} , ΔF_x and ΔF_z as functions of α and δ . An example of this may be seen on the error associated with a matte surface at medium albedo, which should have mid-range values for both α and δ . From Figure 5.20, it is clear that appropriate values of 0.6 and 0.4 respectively find the region of minimum error. However, the algorithmic search would not be able to distinguish between this, and values of 1.0 and 0.6 for α and δ, which would be characteristic of a highly unusual surface that is both highly reflective and very rough.

These data, presented in Figures 5.20, 5.21, and 5.22, confirmed that "sensible" values existed with very similar error even in those cases where the algorithmic search for lowest error produced unusual results. It is also apparent that there are a number of solutions of approximately equivalent merit, and that the determination of α and δ is probably not critical to better than 1 part in 10, and certainly not beyond 1 part in 100. At low albedo particularly, there is very little difference between the four surfaces, and making significant investment in a precise determination of α and δ is futile.

For reference on these figures, recall that the force-error is expressed *per photon*, and that an error of value 1 is equivalent to the effect of the total incident radiation if it were all absorbed. The largest possible force-value on this scale is 2, which occurs in the vertical direction when $\phi = 0$, and $\alpha = 1$ on a perfectly smooth surface.

This observation is further supported by data presented in section RadiationPressure_8, Figures 5.29 to 5.34. While the two types of reflection produce Force-time graphs with very different patterns, the integration of those graphs yields impulses that are quite similar.

Of interest, there is a trend that as the surface gets rougher, α gets smaller and/or δ gets larger. This is expected behavior; as the surface gets rougher, more photons will be reflected "down" into the surface and be absorbed, reducing the albedo, those photons that are reflected upward are going to be more scattered, appearing as diffuse reflection more than specular reflection. For all four surfaces, the albedo settings (low, med, or high) are assigned by the value of the albedo of each microscopic plate.

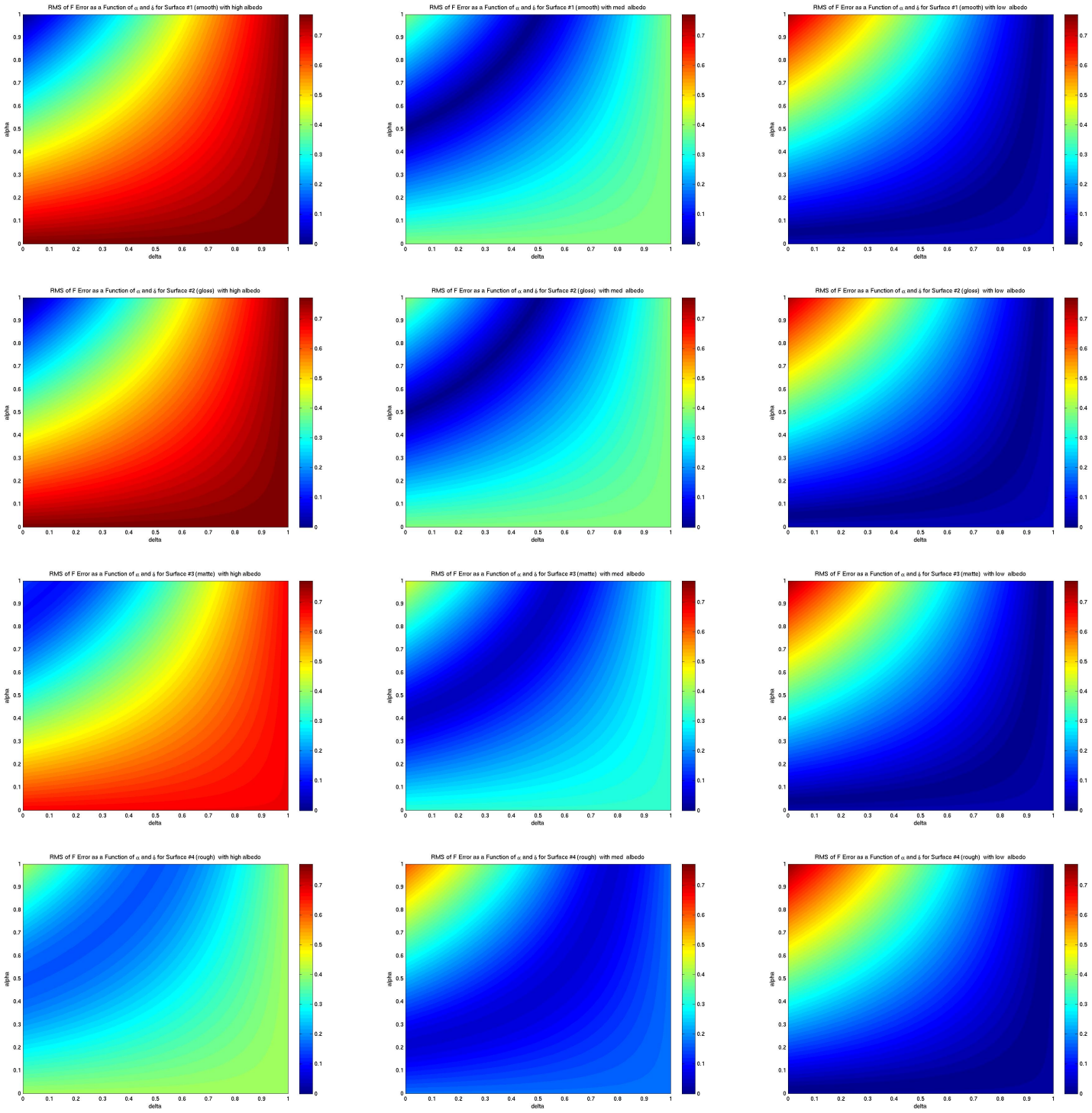


Figure 5.20: Differences in the value of the magnitude of the model force, comparing the numerical simulation to the simplified model, as a function of α and δ, presented in matrix formation with rows representing (from top) surface 1-4, and columns representing (from left) high to low albedo.

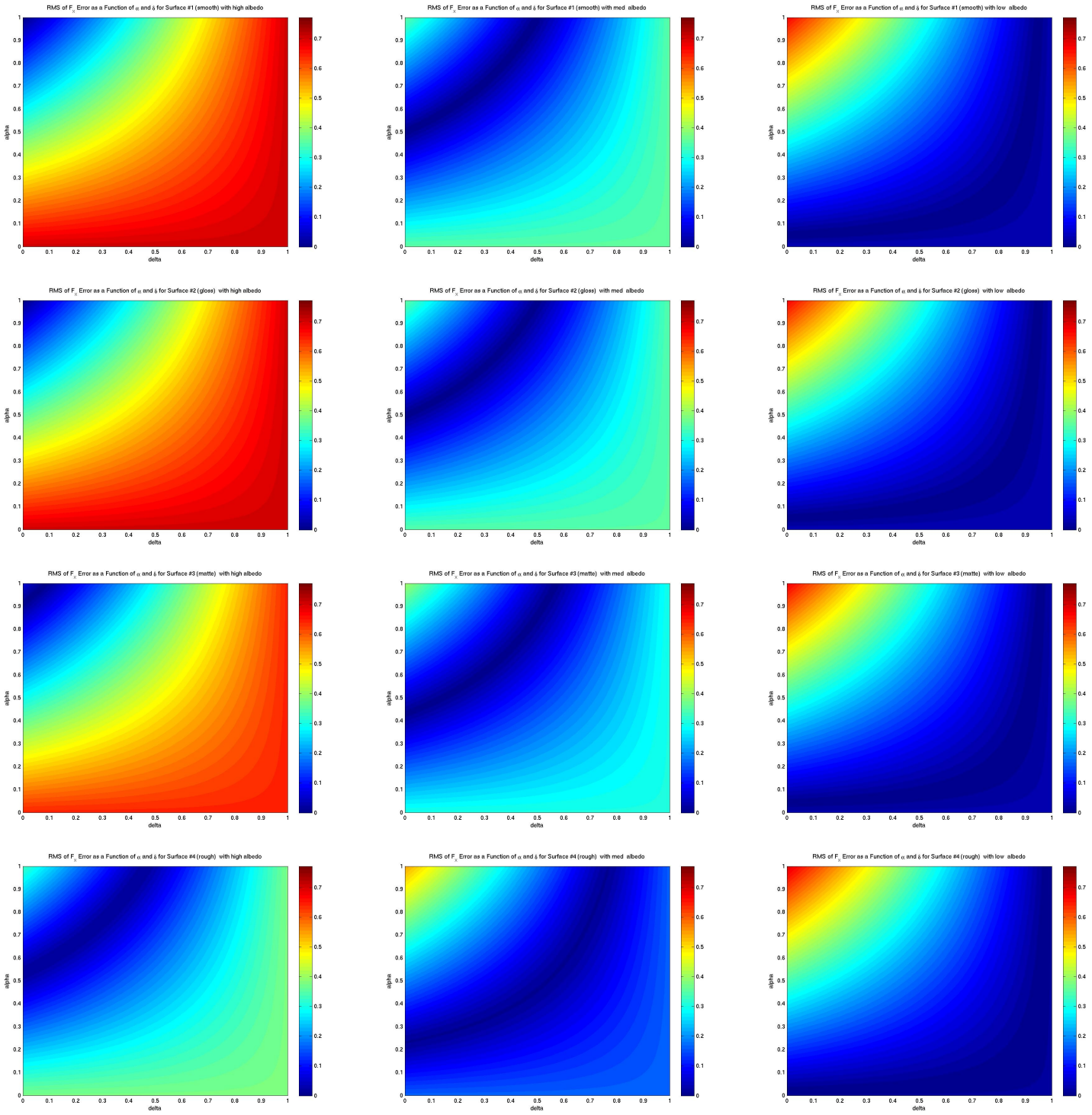


Figure 5.21: Differences in the value of the x-component of the model force, comparing the numerical simulation to the simplified model, as a function of α and δ, presented in matrix formation with rows representing (from top) surface 1-4, and columns representing (from left) high to low albedo.

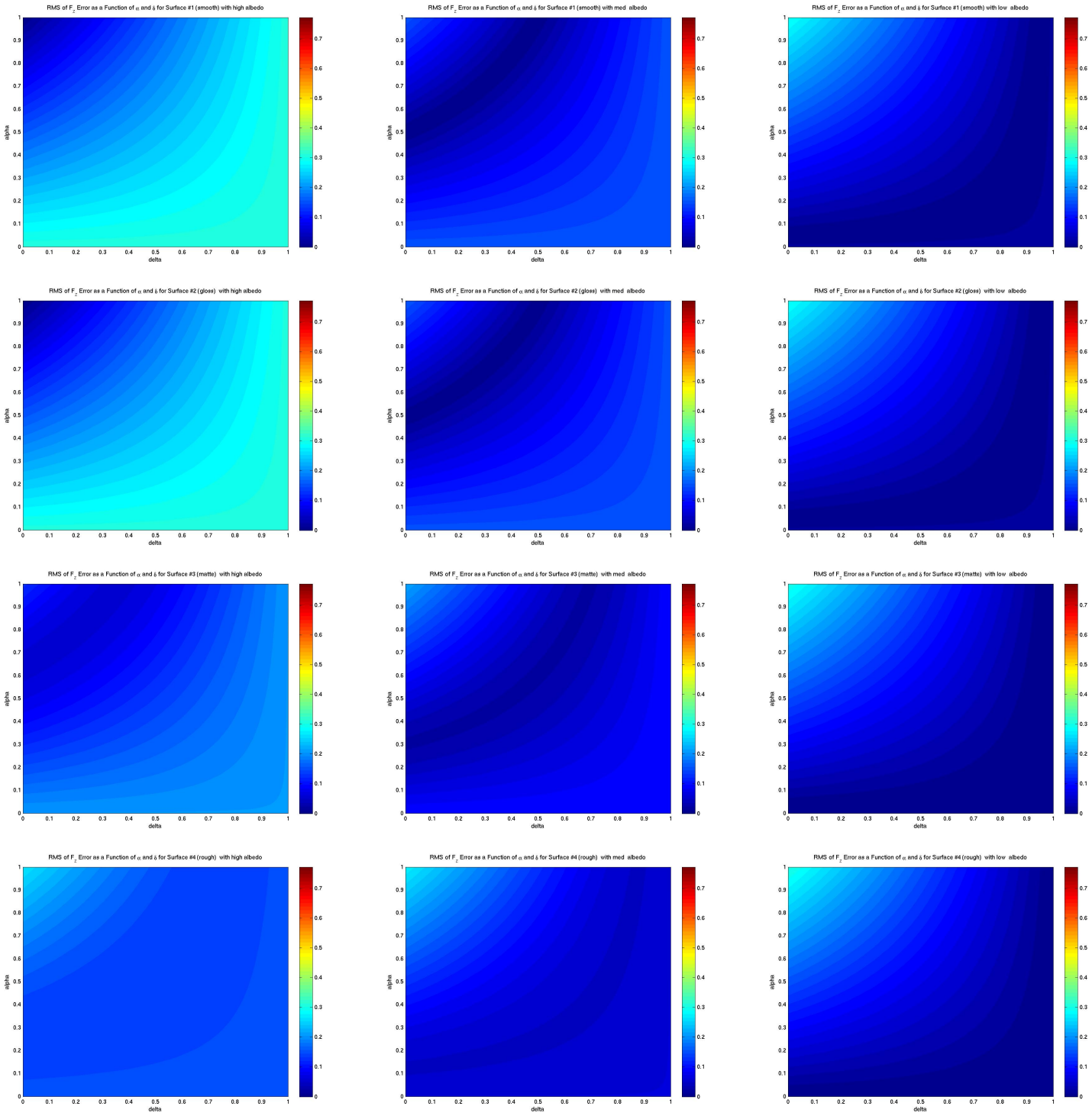


Figure 5.22: Differences in the value of the z-component of the model force, comparing the numerical simulation to the simplified model, as a function of α and δ, presented in matrix formation with rows representing (from top) surface 1-4, and columns representing (from left) high to low albedo.

5.2.3 Validation of the Plate Model Assumptions

Test RadiationPressure_8: Validation of the Plate Model Assumptions

Purpose:

To validate that, for the purposes of radiation pressure calculations, curved surfaces can be approximated by crude, gross-feature-only, flat plate models, and to assess the degree to which doing so introduces error into the system.

Procedure:

The response of a square cuboid — of side s and length L — to a radiation field is compared to that of a cylinder of comparable dimensions, with the cuboid representing the flat-plate approximation to the cylinder. The two objects were rotated in a constant radiation field with the long-axis perpendicular to the flux vector, as shown in figure 5.23.

This test was conducted using the JEOD v5.4 algorithms ported into MATLAB, while test RadiationPressure_9 on page 131 compares the difference directly between the spherical default surface, and a cubic flat-plate surface.

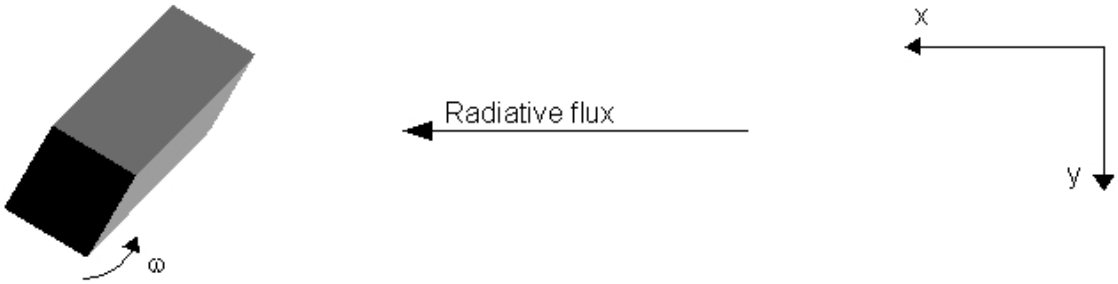


Figure 5.23: Orientation of the cylinder and cuboid.

The effects from the ends of the objects were ignored, which is essentially equivalent to assuming the objects to be long and thin. Rotation angles are measured counterclockwise from the lowest point on the object, as shown in figure 5.24.

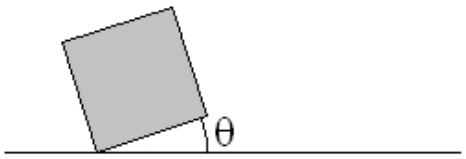


Figure 5.24: Definition of angle θ.

The cuboid starts 'sitting' on one face, as shown in figure 5.25.

The two objects were configured to allow for two extremes of temperature structures:

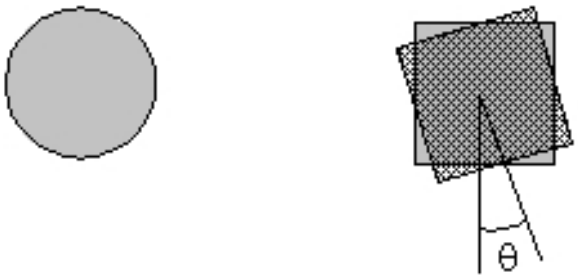


Figure 5.25: Orientation of the cylinder and cuboid.

1. The temperature was uniform across the object (isothermal approximation).
2. The temperature was spatially variable, with plates assumed to be thermally isolated for maximum temperature differential. For this, the cylinder was configured to comprise a large collection of thin strips, running the length of the cylinder.

Data are taken from the steady states of the two objects for each configuration (i.e. after initial transients have died away, the variability is due entirely to the rotation, and the temperature – and therefore the force – associated with each plate can be expressed as a function of θ only, with no other dependency on time).

Finally, the dimensions of the cuboid were adjusted to investigate whether it was more important to have equivalent surface area, or equivalent enclosed volume as an approximation to the cylinder.

Mathematical development

This section develops the equations used in these tests.

1. Cuboid
   (a) Absorption:
   The rate at which energy is absorbed is the product of the flux vector, the projected area, and the absorptivity (equal to 1-albedo). The force, or momentum flux, is related to the energy flux by the speed of light. The absorption will only produce a force in the direction of the incident radiation. It will not be a constant, as the cuboid projects different cross section as it rotates.
   Incident flux falls on two projected areas, one of area $sL\sin\theta$, and another of area $sL\cos\theta$,

$$\vec{F} = \left(\frac{1-\alpha}{c}\right)\phi sL(\sin\theta + \cos\theta)\hat{i} \tag{5.14}$$

   (b) Specular Reflection:
   Incident flux falls on the same two surfaces, where some fraction α is reflected, ($\alpha(1-\delta)$ is reflected specularly). There are two force "components", one from 'stopping' the incident radiation (in direction of radiation vector), and one 'recoiling'

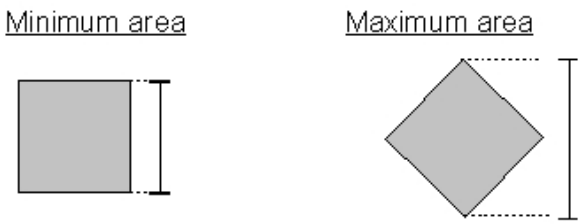


Figure 5.26: The surface area projected perpendicular to the illumination vector.

from the reflected radiation (in direction opposite the radiation vector). Use $\hat{r}_i$ to represent the direction of (specularly) reflected radiation from surface i.

$$\vec{F} = \frac{\alpha(1-\delta)}{c}\phi s L \left(\sin\theta(\hat{i} - \hat{r}_1) + \cos\theta(\hat{i} - \hat{r}_2)\right)$$

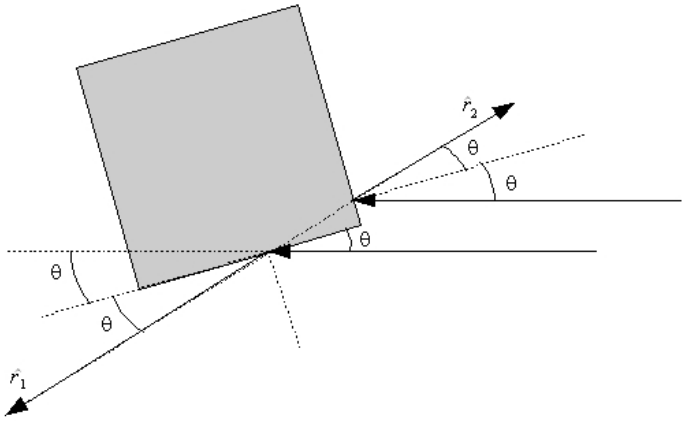


Figure 5.27: Schematic of the reflection from the two illuminated sides.

Figure 5.27 illustrates that by geometry, $\hat{r}_1 = -\hat{r}_2$, and that $\hat{r}_1 = \cos 2\theta\hat{i} + \sin 2\theta\hat{j}$. Hence,

$$\vec{F} = \frac{\alpha(1-\delta)}{c}\phi s L \left[\hat{i}\left(\sin\theta + \cos\theta + \cos 2\theta\left(\cos\theta - \sin\theta\right)\right) + \hat{j}\left(\sin 2\theta\left(\cos\theta - \sin\theta\right)\right)\right]$$

Simplifying,

$$\vec{F} = \frac{\alpha(1-\delta)}{c}\phi s L \left[\hat{i}2\left(\sin^3\theta + \cos^3\theta\right) + \hat{j}\left(\sin 2\theta\left(\cos\theta - \sin\theta\right)\right)\right] \qquad (5.15)$$

(c) Diffuse Reflection:
The same two sides again involved, this time

$$\vec{F} = \frac{\alpha\delta}{c}\phi sL\left[\sin\theta\left(\hat{i} + \frac{2}{3}\left(\sin\theta\hat{i} - \cos\theta\hat{j}\right)\right) + \cos\theta\left(\hat{i} + \frac{2}{3}\left(\cos\theta\hat{i} + \sin\theta\hat{j}\right)\right)\right]$$

which simplifies to

$$\vec{F} = \frac{\alpha\delta}{c}\phi sL\left(\sin\theta + \cos\theta + \frac{2}{3}\right)\hat{i} \tag{5.16}$$

(d) Emission:

The cuboid comprises four sides, all of which emit.

i. Non-isothermal case:

Each side will emit differently, because each side will have an independent temperature. As a side rotates into illumination, it will pass through a minimum temperature, and reach a maximum temperature shortly before passing back into shadow, as shown in Figure 5.28. In order to calculate the force as a function of angle, first the temperature must be known as a function of angle. In this test, that is completed by numerical integration.

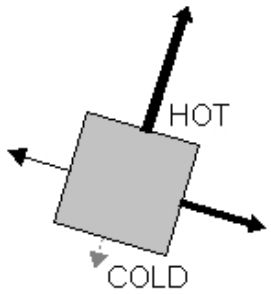


Figure 5.28: The temperature variation across the non-isothermal cuboid.

Consider an arbitrary plate, call it plate#1:

The rate at which the temperature changes is given by:

$$\frac{dT}{dt} = \begin{cases} \frac{4sL}{C}\left((1-\alpha)\phi\sin\theta - \epsilon\sigma T^4\right) & illuminated \\ -\frac{4sL}{C}\epsilon\sigma T^4 & shaded \end{cases}$$

where C is the heat capacity of the entire cuboid (hence the heat capacity of one "side" is C/4).

The emissive force acting on the plate is:

$$\vec{F}(\theta) = \frac{2}{3}\frac{1}{c}\frac{d\vec{E}}{dT}\hat{n} = \frac{2sL\epsilon\sigma T(\theta)^4}{3c}\left(\sin\theta\hat{i} - \cos\theta\hat{j}\right)$$

Each of the other plates, $i = 2, 3, 4$, can be considered as having rotated through an additional angle of $\pi(i-1)/2$ and the overall force obtained from the summation over the four plates, with $\theta_i = \theta + \pi(i-1)/2$.

$$\vec{F}(\theta) = \frac{2sL\epsilon\sigma}{3c}\sum_{i=1}^{4} T(\theta_i)^4\left(\sin\theta_i\hat{i} - \cos\theta_i\hat{j}\right) \tag{5.17}$$

ii. Isothermal case:
Each side will emit at the same rate, and the emissive forces will cancel to zero. Because the emitting area is constant, but the receiving area is variable, the temperature varies in time.

$$\frac{dT}{dt} = \frac{sL}{C}\left((1-\alpha)\phi\left(|\sin\theta| + |\cos\theta|\right) - 4\epsilon\sigma T^4\right) \tag{5.18}$$

2. Cylinder

(a) Absorption:
The rate at which energy is absorbed is the product of the flux vector, the projected area, and the absorptivity (1-albedo). The force is related to the energy rate by the speed of light.

$$\vec{F} = \frac{1-\alpha}{c} 2RL\phi\hat{i} \tag{5.19}$$

(b) Specular reflection:
Similar to the case for the cuboid, there are two considerations to the force, one from 'stopping' the incident radiation (in direction of radiation vector), and one 'recoiling' from the reflected radiation (in direction opposite the radiation vector). Consider a thin strip of the cylinder, of length L subtending an angle $d\theta$ located at rotation angle θ.

$$\vec{F} = \frac{\alpha(1-\delta)}{c}\phi RLd\theta\left[\hat{i}\left(\sin\theta - \cos 2\theta\sin\theta\right) - \hat{j}\left(\sin 2\theta\sin\theta\right)\right]$$

Such strips will contribute to the force as long as $0 \leqslant \theta \leqslant \pi$. Integrating produces

$$\vec{F} = \frac{8\alpha(1-\delta)}{3c}\phi RL\hat{i} \tag{5.20}$$

(c) Diffuse Reflection:
Using a similar analysis based on the development for the cuboid and integrating again yields

$$\vec{F} = \frac{\alpha\delta}{c}\phi RL\left(2 + \frac{\pi}{3}\right)\hat{i} \tag{5.21}$$

(d) Emission:

i. Non-isothermal case:
The cylinder comprises $\frac{2\pi}{d\theta}$ sides, compared to the cuboid's four. Similar to the cuboid, each side will have an independent temperature. In order to calculate the force as a function of angle, first the temperature must be known as a function of angle.
Consider an elemental strip: The rate at which the temperature changes is given by:

$$\frac{dT}{dt} = \begin{cases} \frac{2RL\pi}{C}\left((1-\alpha)\phi\sin\theta - \epsilon\sigma T^4\right) & \textit{illuminated} \\ -\frac{2RL\pi}{C}\epsilon\sigma T^4 & \textit{shaded} \end{cases}$$

where C is the heat capacity of the entire cylinder.

The force acting on an element is:

$$d\vec{F}(\theta) = \frac{2Rd\theta L\epsilon\sigma\left(T(\theta)\right)^4}{3c}\left(\sin\theta\hat{i} - \cos\theta\hat{j}\right) \tag{5.22}$$

which can be integrated numerically over the entire cylinder once the steady state temperature structure is known.

ii. Isothermal case:
The net force from emission for an isothermal cylinder is zero, by symmetry. Also by symmetry, there is no temporal temperature variation.

Results:

Data were collected for cases of low, medium, and high values of albedo, for cases of low, medium and high values for the heat capacity (essentially, this affects only the temperature response to the absorbed radiation; this variation is equivalent to varying the rotational period), and for the cases of totally diffuse and totally specular reflection.

The x-component of the force is larger than the y-component in all cases.

The dominant features, shown in Figures 5.29 to 5.32 are that the forces due to radiation are:

- Strongly affected by the extent to which the plate temperature varies.
- Weakly affected by the approximation of the flat-plate model to the curved surface.

Figure 5.33 shows the integration of the force over time, or the total impulse applied to the surface as a function of time; this is particularly useful because it smooths out the fluctuations that result from the flat-plate approximation to the smooth surface, and allows a comparison to be made of the average force applied. From figures 5.29 to 5.32 it is clear that the variation with time of the force on the cuboid depends strongly on the values used to represent:

1. The albedo
2. The relative strength of the specular to diffuse reflection
3. The extent to which the temperature varies.

Conversely, the cylinder shows no time variation regardless of variations in those values. However, the analysis leading to Figure 5.33 demonstrates that, providing that the size is handled appropriately, the time-integrated difference between the cylinder and cuboid is negligible, and hence it can be concluded that the cuboid is a reasonable approximation to the cylinder across a range of values for the albedo, the temperature fluctuations. Furthermore, Figure 5.34 demonstrates that the difference between specular and diffuse reflection, averaged over time, are also not very significant.

Finally, figure 5.35 contrasts two sizes for the cuboid. It can safely be concluded that it is more important to conserve the surface area of the approximated vehicle than to conserve the enclosed volume of the approximated vehicle. Data presented in Figures 5.29 to 5.34 have the cylinder and cuboid with equal surface area.

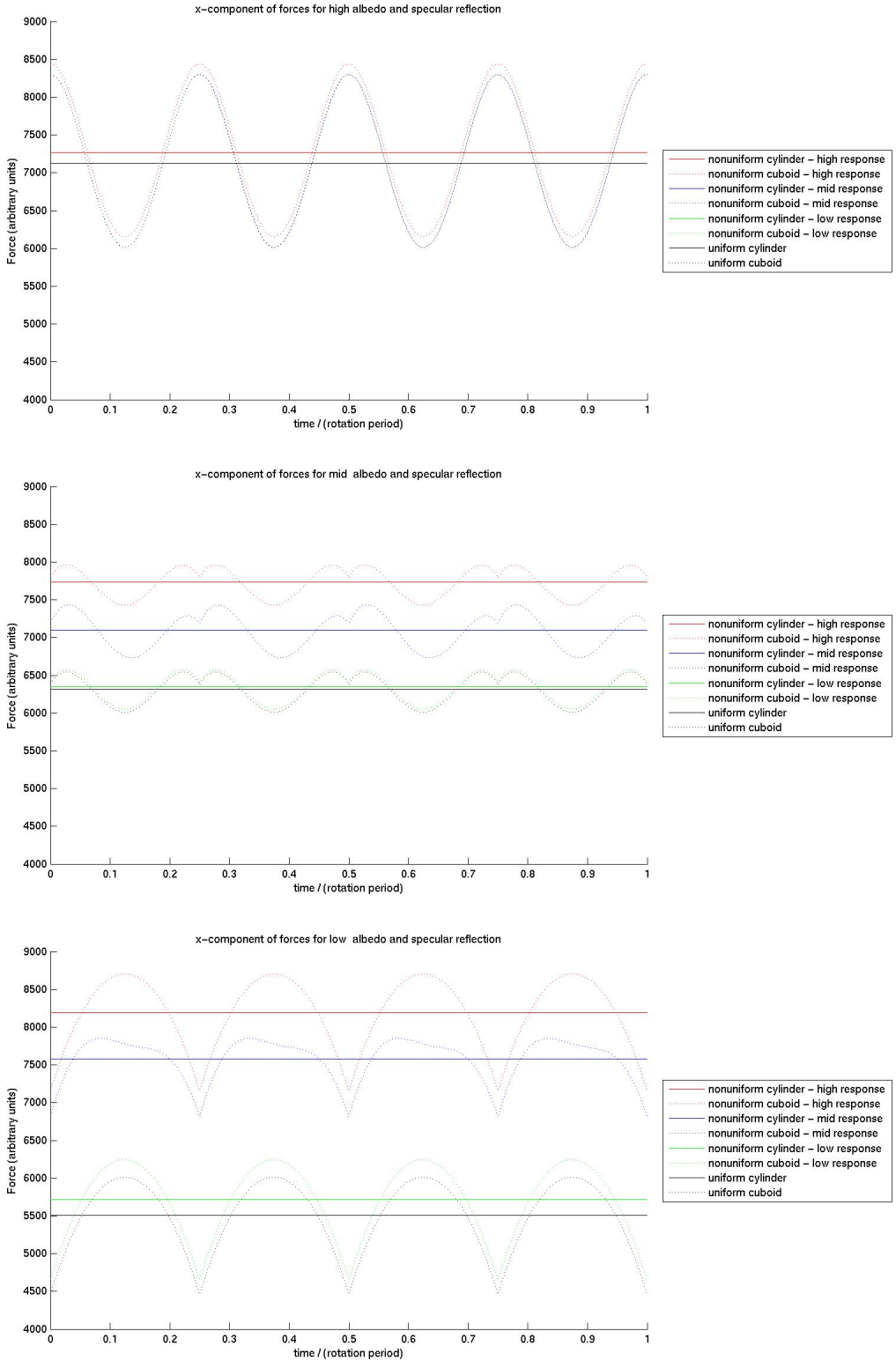


Figure 5.29: The x-component of force due to radiation with specular reflection. Each chart represents a different albedo, and contains plots for different temperature-response models. A model with low heat capacity will have a high temperature response.

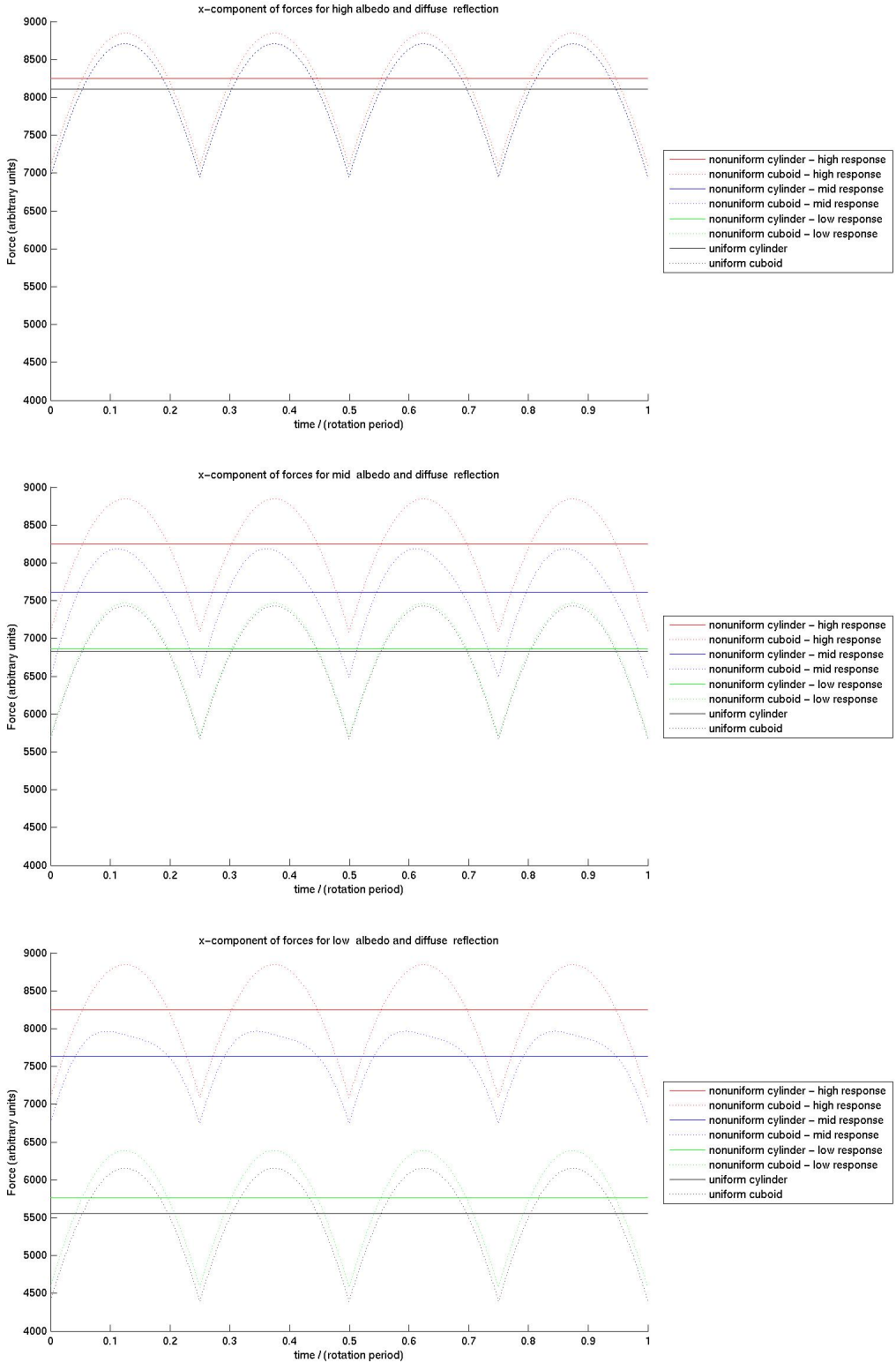


Figure 5.30: The x-component of force due to radiation with diffuse reflection. Each chart represents a different albedo, and contains plots for different temperature-response models. A model with low heat capacity will have a high temperature response.

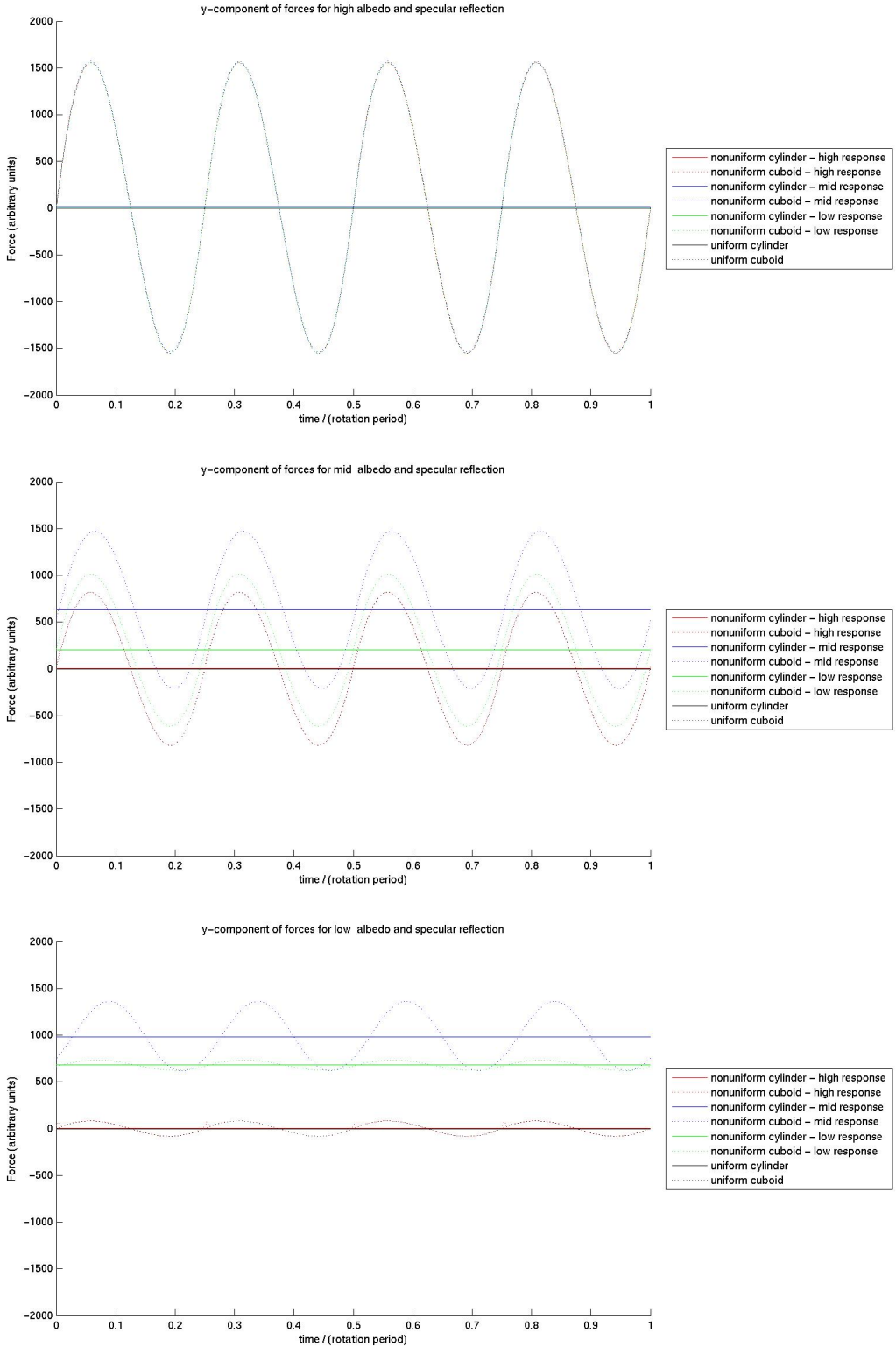


Figure 5.31: The y-component of force due to radiation with specular reflection. Each chart represents a different albedo, and contains plots for different temperature-response models. A model with low heat capacity will have a high temperature response.

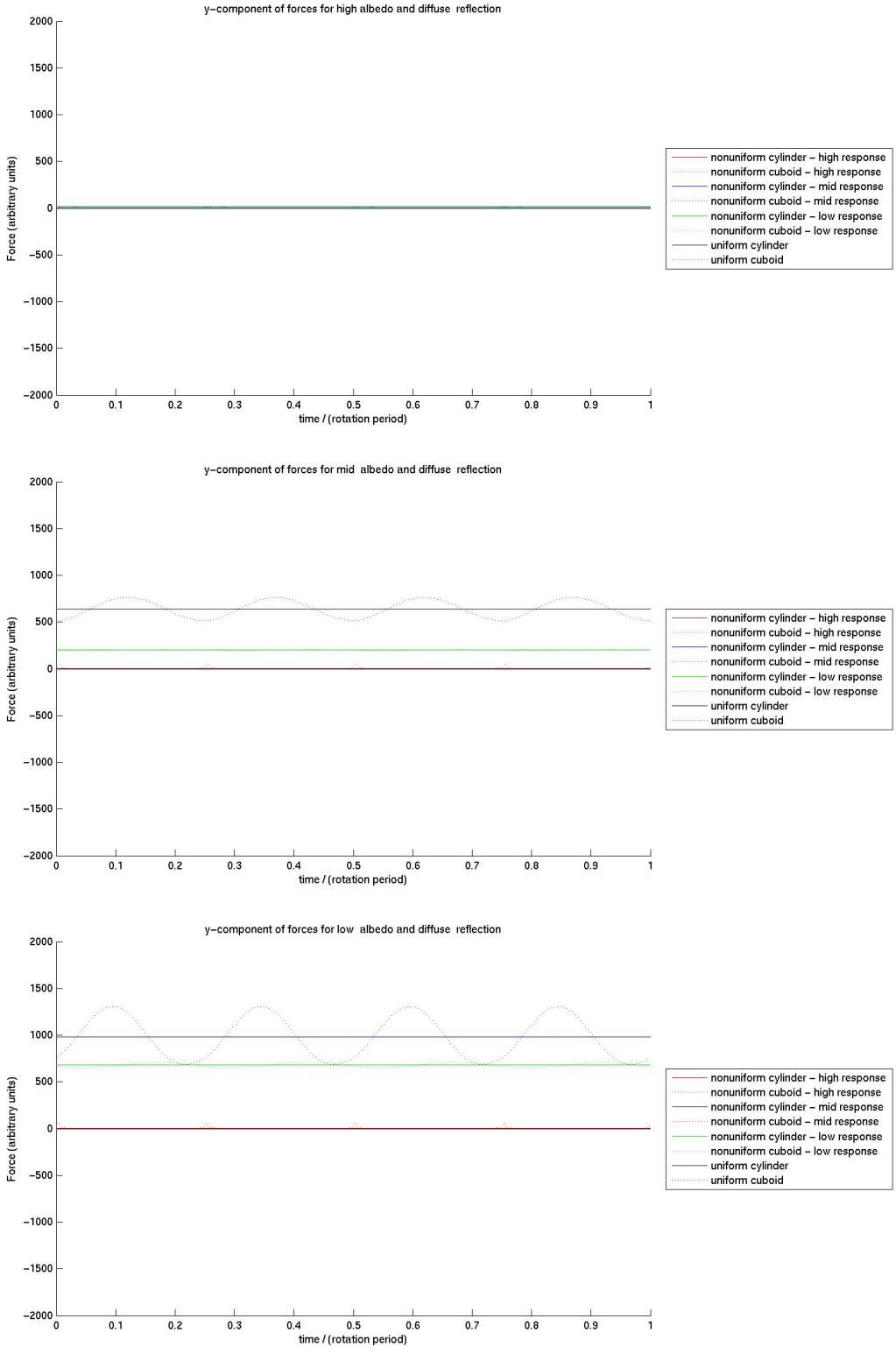


Figure 5.32: The y-component of force due to radiation with diffuse reflection. Each chart represents a different albedo, and contains plots for different temperature-response models. A model with low heat capacity will have a high temperature response.

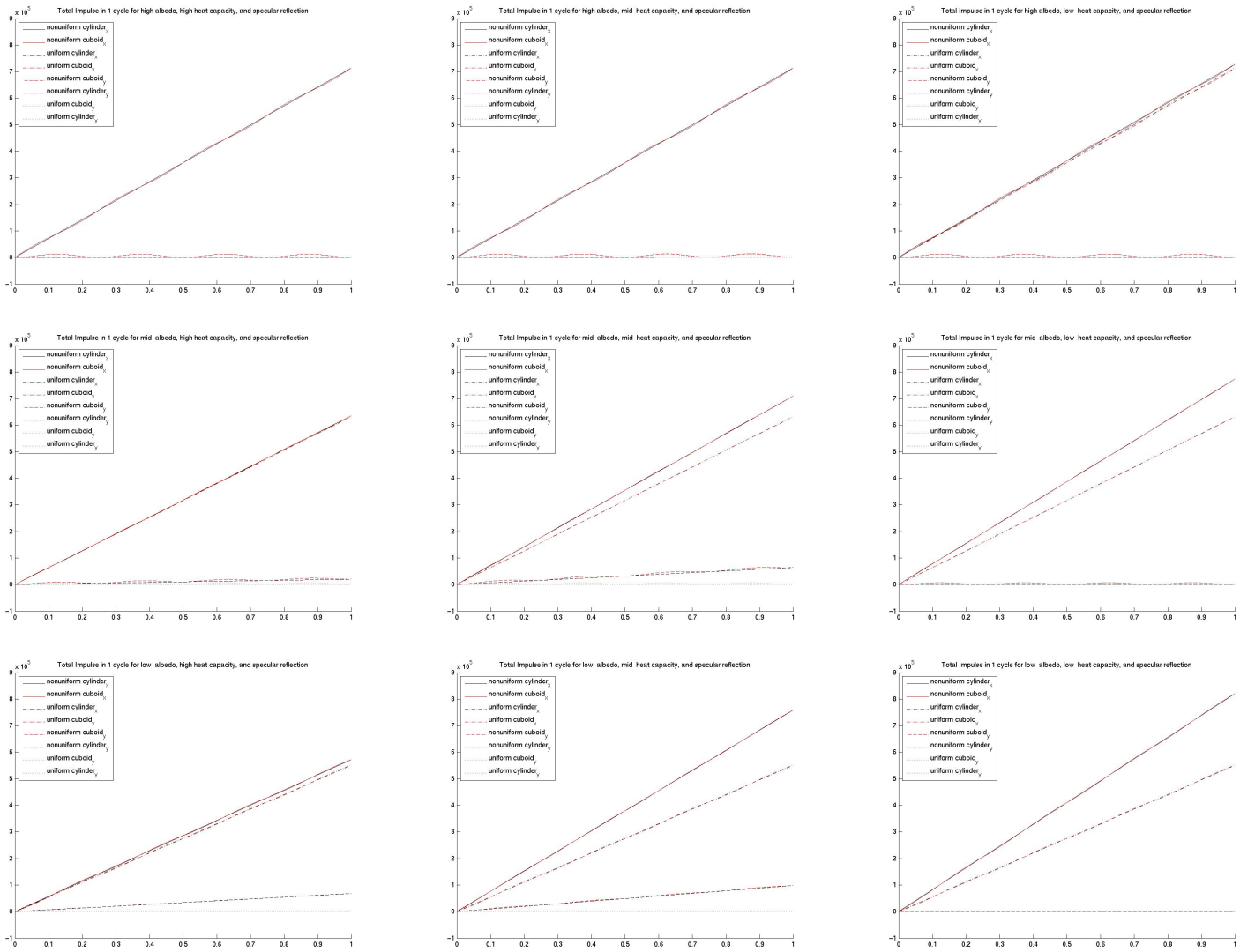


Figure 5.33: The integrated force for the cylinder and cuboid of varying degrees of temperature responsiveness. The graphs are presented on a matrix grid, with each row representing constant albedo (high at the top), and each column representing constant heat capacity (heat capacity high, temperature response low, to the left). These data are generated with the assumption of specular reflection. For each graph, the x-axis represents time in cycles, and the y-axis represents accumulated impulse. The upper cluster of lines represents the force parallel to the flux vector, and the lower cluster the force perpendicular to the flux vector on the same scale.

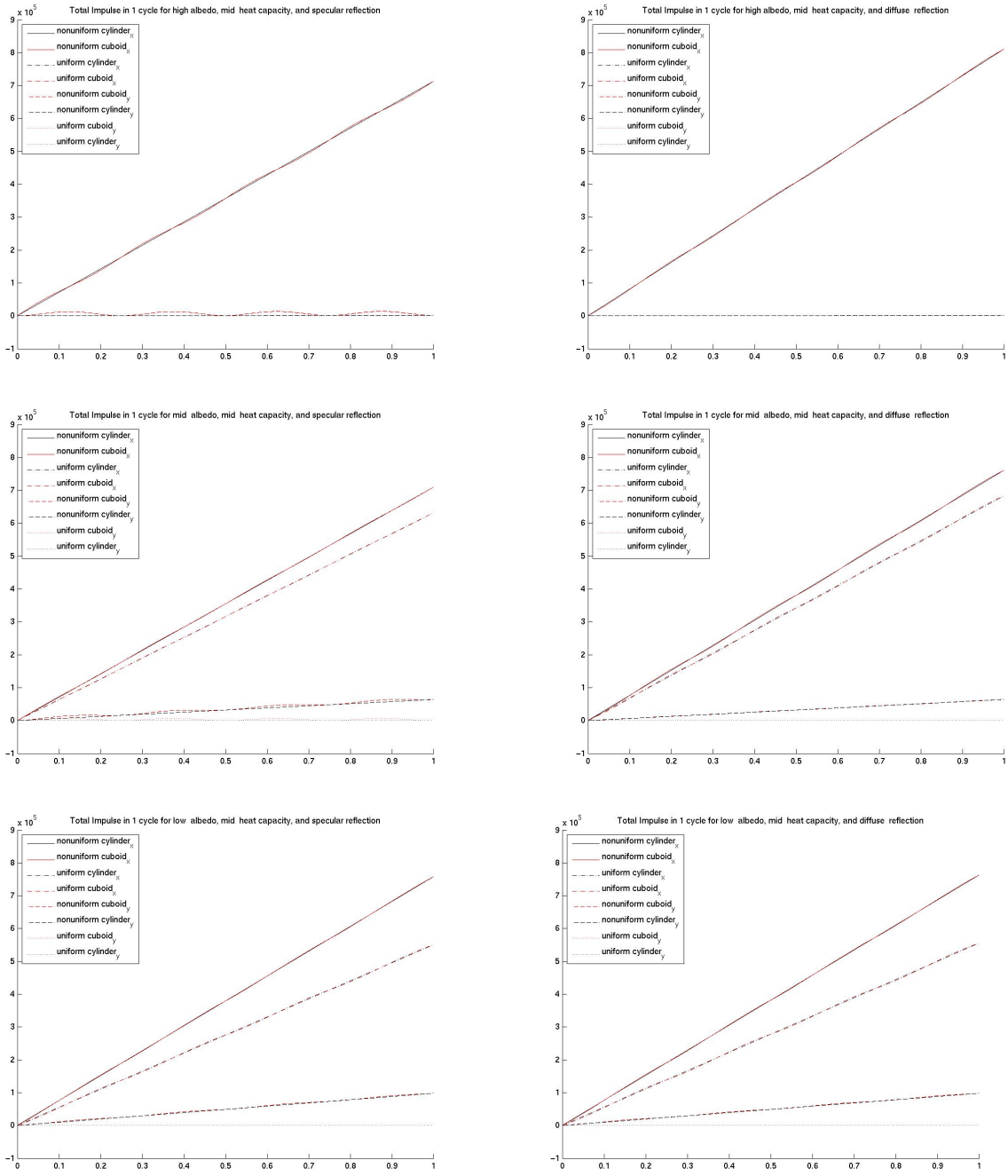


Figure 5.34: The data in the left column is a copy of the center column from figure 5.33, with specular reflection. The data in the right-column is from similar runs but with diffuse reflection. For each graph, the x-axis represents time in cycles, and the y-axis represents accumulated impulse.

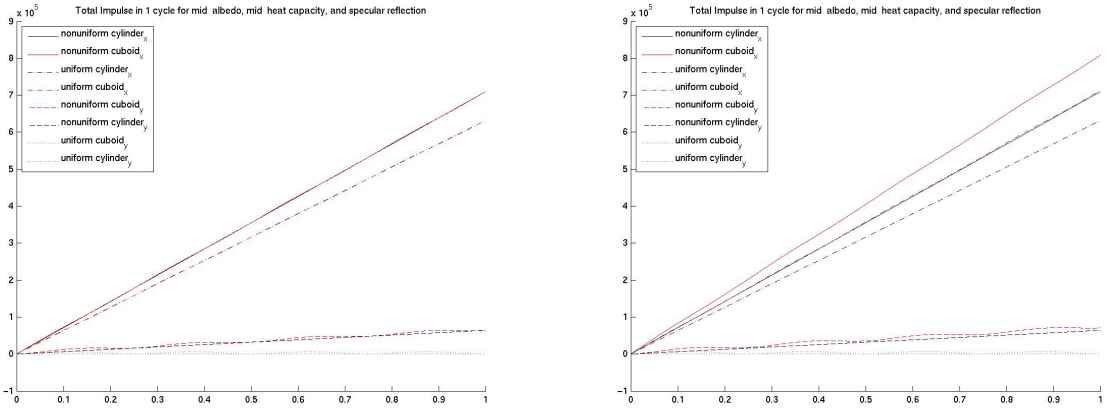


Figure 5.35: The data presented in the left graph is the center panel from figure 5.33, for which the cuboid surface area is equivalent to the cylinder surface area. The data presented in the second graph is the impulse when the cylinder is approximated as a cuboid while maintaining constant enclosed volume. For each graph, the x-axis represents time in cycles, and the y-axis represents accumulated impulse.

Test RadiationPressure_9: Comparison of Cube to Default Sphere

Purpose:

The purpose of this test is to continue the investigation into relative sizing, in the particular case of using the default sphere as an approximation to an angular surface.

Procedure:

The simulation used in this test is available at *SIM_4_DEFAULT/RUN_compare*. Several surfaces were simulated, including:

1. A cube.
2. A sphere fitting entirely inside the cube.
3. A sphere with a cross-sectional area equal to the area of one side of the cube.
4. A sphere with the same surface area as the cube.
5. A sphere that completely encloses the cube.

The force data for each of the spherical surfaces were then compared against the data for the cubic surface to identify the closest match.

Results:

Confirming the results from test RadiationPressure_8, the best approximation to an angular surface is to use a sphere with the same cross-sectional area. Figure 5.36 shows the comparison of the forces applied to each surface. Notice that the default spherical surfaces are all smooth, and the cubic surface has some irregularities associated with the faces turning into the flux vector.

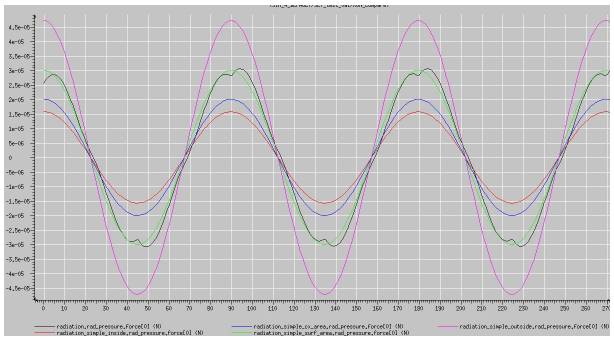


Figure 5.36: The x-component of the radiation forces for each of the surfaces; *radiation.rad_pressure* is the cubic surface; *radiation_simple_inside* fits inside the cube; *radiation_simple_cx_area* has the same cross-sectional area as the cube; *radiation_simple_surf_area* has the same surface area as the cube; *radiation_simple_outside* envelopes the cube.

5.2.4 Investigation of the Effect of the Isothermal Approximation on Forces and Temperature-Time Variation

Test RadiationPressure_10: Isothermal Approximation Effect on Forces and Temperature Variation

Purpose:

To identify the strength of the assumption that an object may be treated as a collection of equal-temperature plates. This is particularly relevant to simulations where it may be desirable to make the facets thermally *inactive*; this saves significantly on computational time, but may lead to unacceptable error.

Requirements:

This test further facilitates the modeling process, and underlies the importance of requirements RadiationPressure_4 and RadiationPressure_8

Procedure:

The same analysis is used for this test as for the validation of the flat-plate model (RadiationPressure_8), and the figures referenced in this section are located in that section.

Expectation:

At low albedo, the absorption/emission process dominates the radiation interaction; this is highly temperature dependent, and so the correct temperature structure becomes critical. At high albedo, the temperature-independent reflection process dominates, and the temperature structure is not needed to such precision. It is therefore expected that errors (the difference in the force between propagating the temperature, and holding it constant) will be more significant for a low albedo plate.

In the direction parallel to the plate, the force-error will be maximized when the temperature response is most rapid; a rapid temperature response produces a maximum temperature at the point where the plate-normal and flux vector are anti-aligned. Perpendicular to the plate, the error will be maximized when the temperature response of the plate is somewhat slower — a plate with a rapidly responding temperature will cool very quickly as it moves into shadow, making the two sides perpendicular to the flux vector comparable in temperature, while a very slowly responding plate will produce only very small variations in temperature, and also have comparable temperatures on the sides; neither will produce a significant force.

Results:

Figures 5.29 to 5.32 show that the forces parallel to, and perpendicular to, the flux vector are very much affected by the temperature variation with time, particularly on surfaces with low albedo.

The impulse graphs (Figure 5.33) confirm this observation; the difference between the graphlines associated with uniform and non-uniform surfaces is small at high albedo, and again where the temperature response is minimal (i.e. high heat capacity), but can be quite large, particularly at low albedo and high response (graph at bottom-right of Figure 5.33, with the largest error parallel to flux) and at low albedo and medium response (graph at bottom-center of Figure 5.33, with the largest error perpendicular to the flux).

Conclusions:

The expectations were seen in the data and are now quantified.

- At high albedo, it is appropriate to treat the spacecraft as a single-temperature body, even when the plates are not thermally connected.
- It is also appropriate to treat the body as having uniform temperature in the case that there is insufficient time for large surface temperature variations (i.e. either the heat capacity or the rotational velocity is large).
- For darker objects, with significant periods of illumination and darkness, the approximation of uniform temperature is not particularly effective, with potential for large error in the radiation force term.

Since the thermal *activity* of each facet can be turned on or off independently, it is possible to pick and choose; those facets that will be relatively unaffected by illumination can be held *inactive*, while those that are particularly susceptible to illumination should be made thermally *active*.

5.2.5 Validation of Orbital Simulations

Test RadiationPressure_11: Long Duration

Purpose:

To validate the long-term stability of the simulation, and provide data on the significance of radiation pressure at geostationary altitudes.

Procedure:

The simulation used in this test is available at *SIM_3_ORBIT*. There are two simulation runs, *RUN_baseline* has no radiation pressure, and *RUN_radiation* does. In both simulation runs, a vehicle was placed in Earth orbit, close to the altitude of a geosynchronous satellite, and propagated over a 2-year period.

Results:

On each orbit, the vehicle goes into Earth's shadow for a short period of time, causing the flux to fall to zero before returning to its background level (see Figure 5.37), which itself oscillates with a period of 1 year (see Figure 5.38, which shows the long-term variability of the background flux of approximately ±3.3% over a 1-year period. This long-term variation is attributed to the eccentricity of Earth's orbit and validates the communication between the flux-calculation routine and the ephemeris routines.

A second validating feature is in the difference between the orbital period and eclipse period. Over the simulation, there were 728.7 orbital periods, but only 726.7 eclipse periods. This is attributed to the difference — of 1 cycle per year — between the sidereal and synodic periods.

Test RadiationPressure_12: Comparison to Nonspherical Gravitational Terms

Purpose:

The radiation effect and effect from higher-order gravity terms should be comparable at geosynchronous orbits ([3]). This test verifies that results are consistent with this expectation.

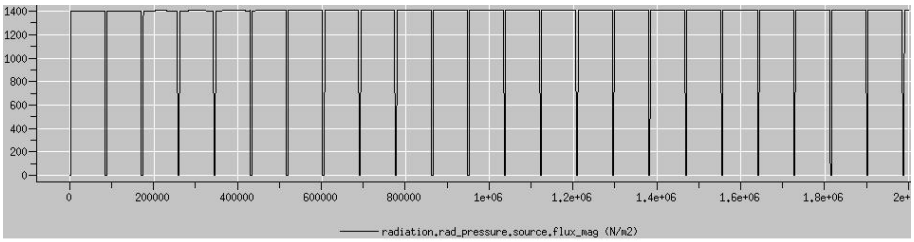


Figure 5.37: The long-term fluctuation in the solar flux observed by a satellite with period approximately 1 day. The oscillation is attributed to shadowing by Earth.

Requirements:

This test partially satisfies requirement RadiationPressure_9

Procedure:

A vehicle was placed in Earth orbit, close to the altitude of a geosynchronous satellite. It was propagated for 3 scenarios: with spherical gravity and no radiation (baseline scenario), with nonspherical gravity and no radiation, and with spherical gravity and radiation.

Results:

It must be noted that without details about the satellite cited in [3], an order of magnitude or so must be considered satisfactorily close. While the shape and mass of the vehicle do not affect the magnitude of the influence of higher order gravity terms, they are essential in the calculation of the radiation effect. Figure 5.39 shows the short-term effect on position of including radiation pressure, compared to the baseline configuration, and Figure 5.40 shows the same effect on the velocity vector. Over a longer duration, Figure 5.41 shows the deviation in position from the baseline scenario for the gravity-enhanced and radiation-enhanced simulations.

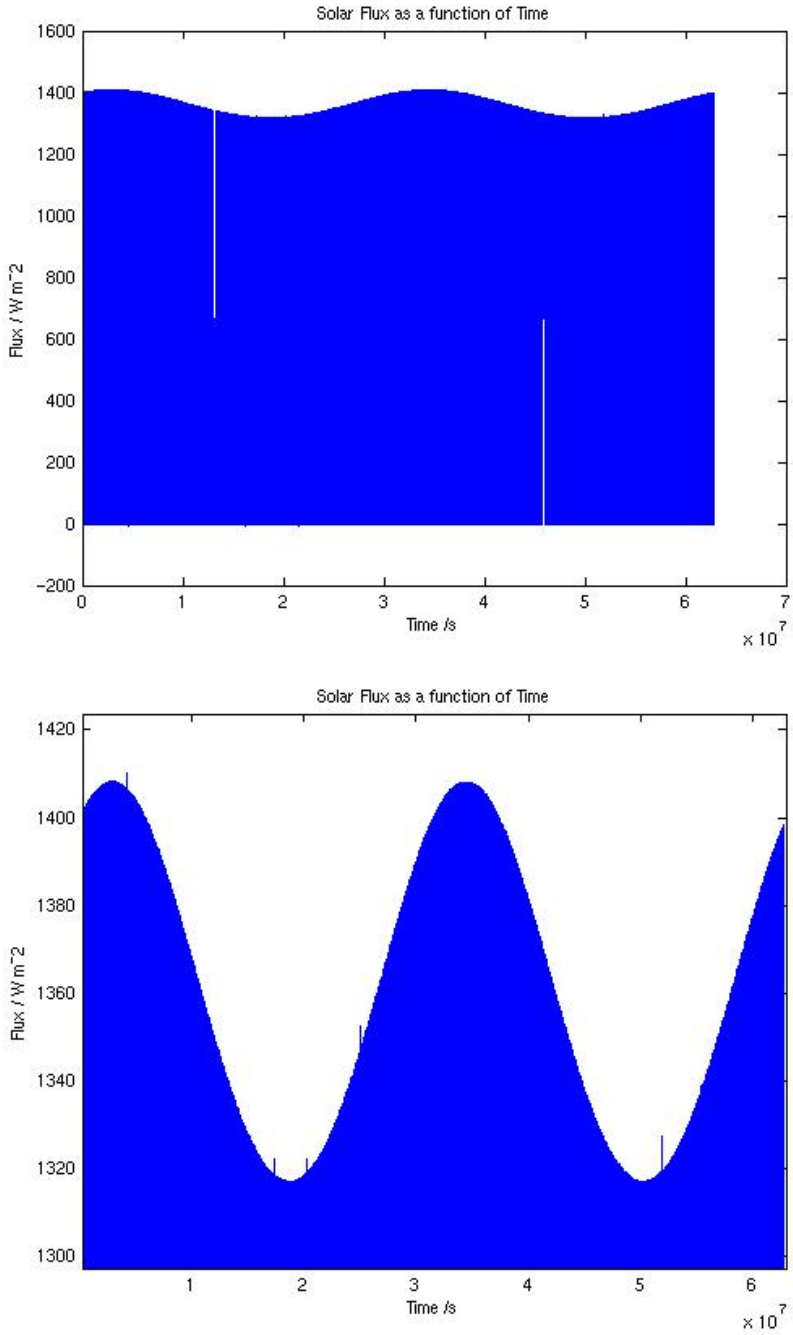


Figure 5.38: The long-term fluctuation in the solar flux observed by a satellite with period approximately 1 day. The upper graph gives an indication of the relative magnitude of the oscillation, while the lower graph gives a better indication of the absolute magnitude of the oscillation. The graph is solid because it also includes the daily oscillation between 0 and the background flux.

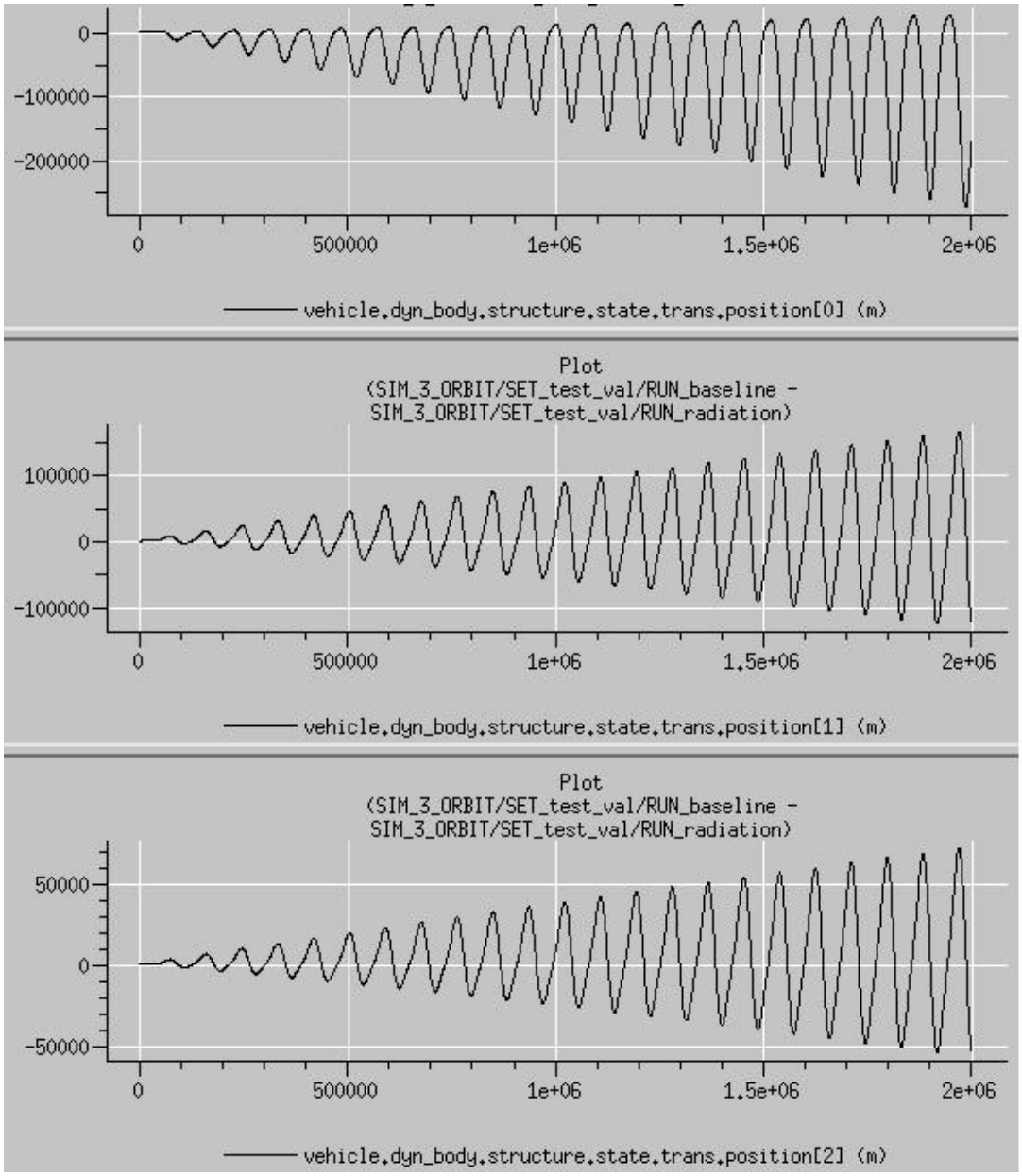


Figure 5.39: The effect on position of including the forces due to radiation pressure on an otherwise simple simulation of an Earth satellite at altitude comparable to that of a geosynchronous orbit.

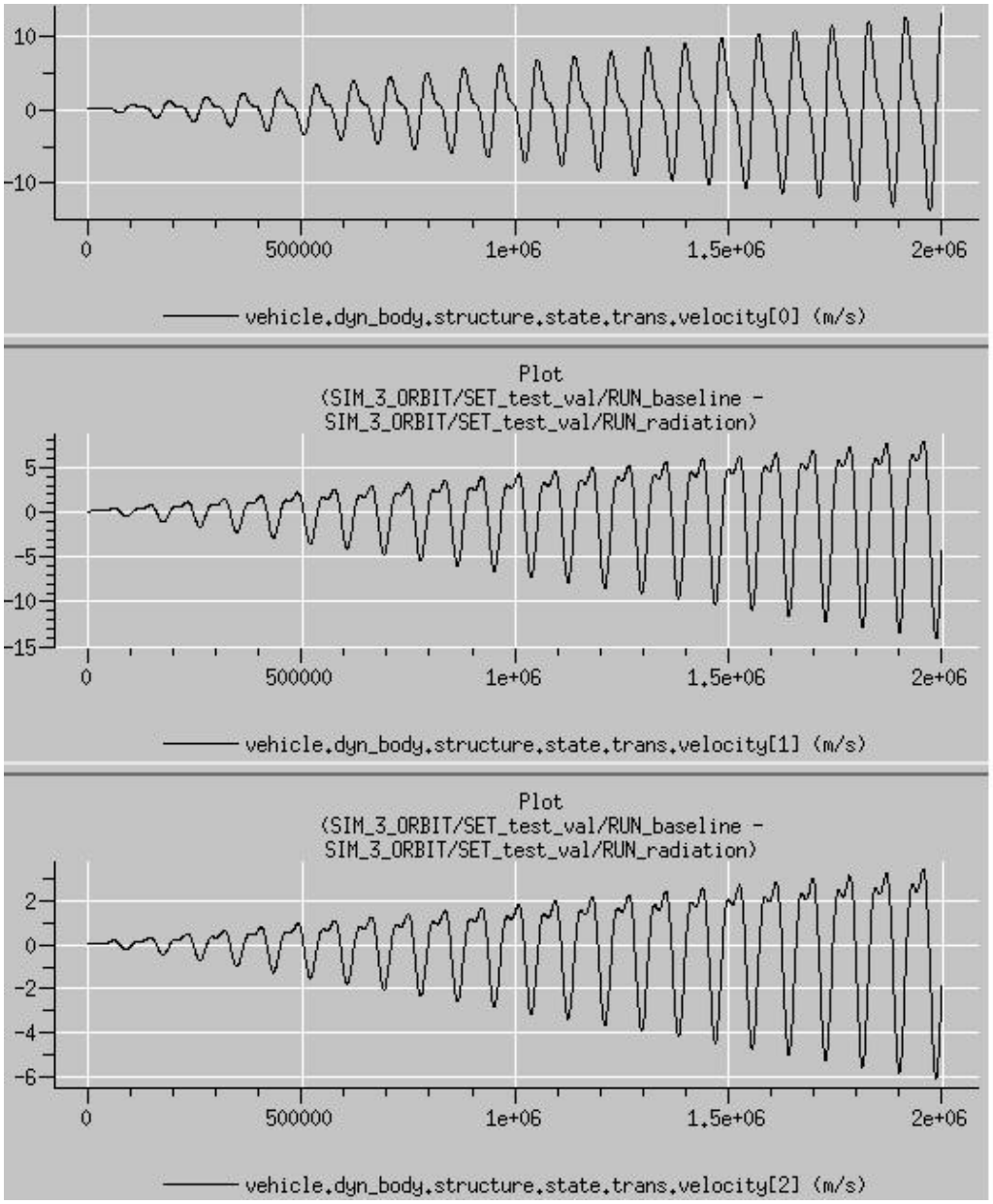


Figure 5.40: The effect on velocity of including the forces due to radiation pressure on an otherwise simple simulation of an Earth satellite at altitude comparable to that of a geosynchronous orbit.

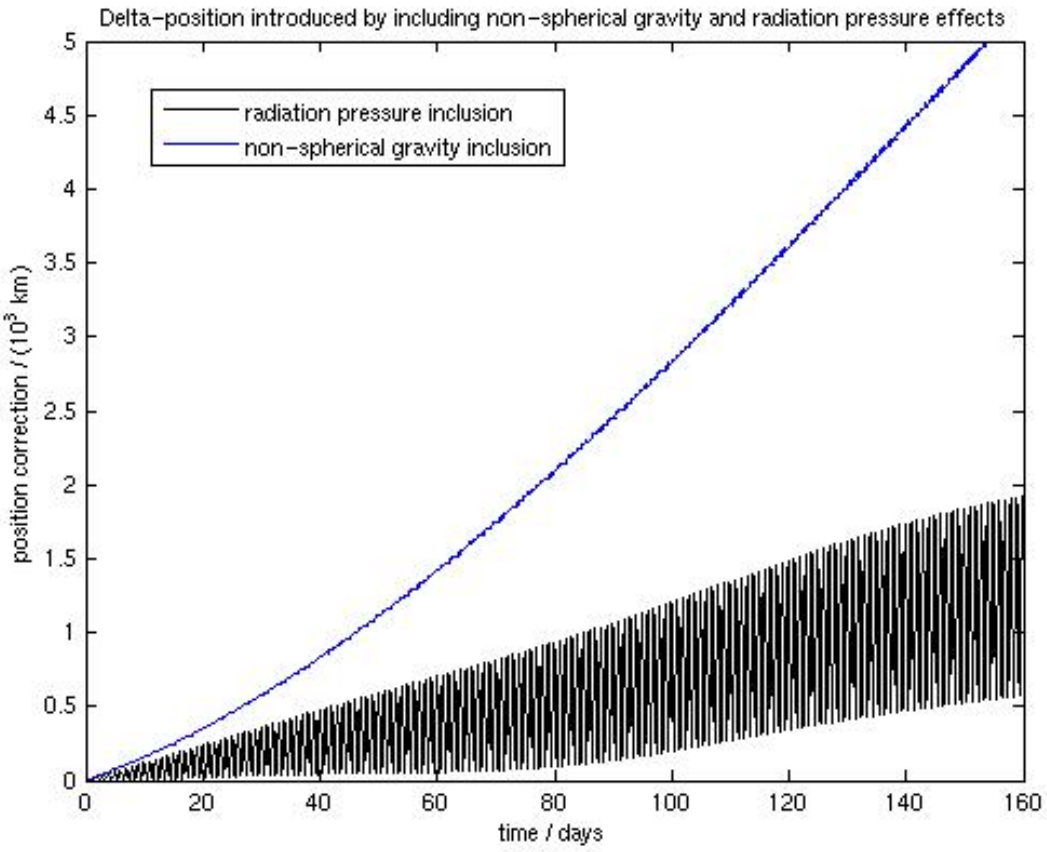


Figure 5.41: The result of including the effects of nonspherical gravity compared to the result of including the effects of radiation pressure, on an otherwise simple simulation of an Earth satellite at altitude comparable to that of a geosynchronous orbit.

5.3 Metrics

5.3.1 Code Metrics

Table 5.1 presents coarse metrics on the source files that comprise the model.

Table 5.1: Coarse Metrics

File Name	Number of Lines			
	Blank	**Comment**	**Code**	**Total**
include/flat_plate_radiation_facet.hh	25	82	30	137
include/flat_plate_radiation_factory.hh	19	70	22	111
include/radiation_base_facet.hh	36	133	40	209
include/radiation_default_surface.hh	29	98	37	164
include/radiation_facet.hh	28	97	39	164
include/radiation_messages.hh	19	89	20	128
include/radiation_params.hh	16	77	22	115
include/radiation_pressure.hh	39	118	56	213
include/radiation_source.hh	44	145	54	243
include/radiation_surface.hh	35	94	48	177
include/radiation_surface_factory.hh	18	77	21	116
include/radiation_third_body.hh	49	178	85	312
src/cmake_file_list.cmake	2	0	19	21
src/flat_plate_radiation_facet.cc	27	70	77	174
src/flat_plate_radiation_factory.cc	23	56	69	148
src/radiation_base_facet.cc	18	49	57	124
src/radiation_default_surface.cc	34	67	178	279
src/radiation_facet.cc	16	43	39	98
src/radiation_messages.cc	17	32	13	62
src/radiation_pressure.cc	31	110	257	398

Continued on next page

Table 5.1: Coarse Metrics (continued from previous page)

File Name	Number of Lines			
	Blank	Comment	Code	Total
src/radiation_pressure__default_surface.cc	20	58	47	125
src/radiation_pressure__surface_model.cc	23	76	50	149
src/radiation_source.cc	27	69	47	143
src/radiation_surface.cc	41	94	213	348
src/radiation_surface_factory.cc	18	48	36	102
src/radiation_third_body.cc	44	120	308	472
Total	698	2150	1884	4732

Table 5.2 presents the extended cyclomatic complexity (ECC) of the methods defined in the model.

Table 5.2: Cyclomatic Complexity

Method	File	Line	ECC
jeod::RadiationDefaultSurface::set_name(std::set_name (std::string name_in)	include/radiation_default_surface.hh	147	1
jeod::RadiationFacet::get_thermal_integrator ()	include/radiation_facet.hh	142	1
jeod::RadiationSource::set_name(std::set_name (std::string name_in)	include/radiation_source.hh	226	1
jeod::RadiationThirdBody::set_name(std::set_name (std::string name_in)	include/radiation_third_body.hh	242	1
jeod::RadiationThirdBody::accumulate_refl_flux (RadiationBaseFacet * veh_surf_elem JEOD_UNUSED, bool calculate_forces JEOD_UNUSED)	include/radiation_third_body.hh	250	1

Continued on next page

Table 5.2: Cyclomatic Complexity (continued)

Method	File	Line	ECC
jeod::RadiationThirdBody::accumulate_rad_flux (RadiationBaseFacet * veh_surf_elem JEOD_UNUSED, bool calculate_forces JEO D_UNUSED)	include/radiation_third_body.hh	259	1
jeod::RadiationThirdBody::is_interactive ()	include/radiation_third_body.hh	268	1
jeod::RadiationThirdBody::get_added_to_model ()	include/radiation_third_body.hh	277	1
jeod::RadiationThirdBody::set_added_to_model (bool value)	include/radiation_third_body.hh	288	1
jeod::FlatPlateRadiation Facet::define_facet (Flat Plate * flat_plate)	src/flat_plate_radiation_facet.cc	50	1
jeod::FlatPlateRadiation Facet::initialize_geom (double center_grav[3])	src/flat_plate_radiation_facet.cc	64	1
jeod::FlatPlateRadiation Facet::incident_radiation (const double flux_mag, const double flux_struct_hat[3], const bool calculate_forces)	src/flat_plate_radiation_facet.cc	76	3
jeod::FlatPlateRadiation Facet::radiation_pressure ()	src/flat_plate_radiation_facet.cc	131	3
jeod::FlatPlateRadiation Factory::JEOD_DECLAR E_ATTRIBUTES (Flat PlateRadiationFacet)	src/flat_plate_radiation_factory.cc	53	1
jeod::FlatPlateRadiation Factory::create_facet (Facet * facet, FacetParams * params)	src/flat_plate_radiation_factory.cc	64	3
jeod::FlatPlateRadiation Factory::is_correct_factory (Facet * facet)	src/flat_plate_radiation_factory.cc	124	2
jeod::RadiationBaseFacet::initialize ()	src/radiation_base_facet.cc	47	7

Continued on next page

Table 5.2: Cyclomatic Complexity (continued)

Method	File	Line	ECC
jeod::RadiationBaseFacet:: interact_with_third_body (RadiationThirdBody * third_body_ptr, const bool calculate_forces)	src/radiation_base_facet.cc	90	1
jeod::RadiationBaseFacet:: initialize_runtime_values ()	src/radiation_base_facet.cc	107	1
jeod::RadiationDefault Surface::initialize ()	src/radiation_default_ surface.cc	55	21
jeod::RadiationDefault Surface::incident_radiation (const double flux_mag, const double flux_struc_ hat[3], const bool calculate_ forces)	src/radiation_default_ surface.cc	203	2
jeod::RadiationDefault Surface::thermal_update ()	src/radiation_default_ surface.cc	247	3
jeod::RadiationDefault Surface::add_thermal_ integrator_to (DynBody * dyn_body)	src/radiation_default_ surface.cc	263	1
jeod::RadiationFacet:: initialize ()	src/radiation_facet.cc	49	9
jeod::RadiationFacet::define_ facet_core (Facet *, const ThermalFacetRider & facet_thermal, Radiation Params * params)	src/radiation_facet.cc	76	1
jeod::RadiationPressure:: RadiationPressure ()	src/radiation_pressure.cc	56	1
jeod::RadiationPressure::~ RadiationPressure ()	src/radiation_pressure.cc	67	1
jeod::RadiationPressure:: initialize_environment (Dyn Manager * dyn_mgr_ptr)	src/radiation_pressure.cc	75	2
jeod::RadiationPressure:: update (RefFrame & veh_ struc_frame, double center_ grav[3], double scale_factor, double real_time)	src/radiation_pressure.cc	90	4

Continued on next page

Table 5.2: Cyclomatic Complexity (continued)

Method	File	Line	ECC
jeod::RadiationPressure:: third_body_adjustments (double real_time, Ref Frame & veh_struc_frame)	src/radiation_pressure.cc	131	3
jeod::RadiationPressure::add_ third_body (RadiationThird Body * third_body_ptr)	src/radiation_pressure.cc	152	9
jeod::RadiationPressure::set_ third_body_active (const std::string & third_body_ name)	src/radiation_pressure.cc	266	4
jeod::RadiationPressure::set_ third_body_inactive (const std::string & third_body_ name)	src/radiation_pressure.cc	306	6
jeod::RadiationPressure::find_ third_body (const std:: string & third_body_name)	src/radiation_pressure.cc	353	3
jeod::RadiationPressure::set_ calculate_forces (bool value)	src/radiation_pressure.cc	378	2
jeod::RadiationPressure:: initialize (DynManager & dyn_mgr, RadiationDefault Surface * surf_ptr)	src/radiation_pressure__ default_surface.cc	64	2
jeod::RadiationPressure:: update_default_surface ()	src/radiation_pressure__ default_surface.cc	87	6
jeod::RadiationPressure:: initialize (DynManager & dyn_mgr, RadiationSurface * surf_ptr, double center_ grav[3])	src/radiation_pressure__ surface_model.cc	65	2
jeod::RadiationPressure:: update_facet_surface ()	src/radiation_pressure__ surface_model.cc	95	7
jeod::RadiationSource:: calculate_flux (RefFrame & veh_struc_frame, const double center_gravity[3])	src/radiation_source.cc	68	2
jeod::RadiationSource:: initialize (DynManager * dyn_mgr_ptr)	src/radiation_source.cc	113	2

Continued on next page

Table 5.2: Cyclomatic Complexity (continued)

Method	File	Line	ECC
jeod::RadiationSurface::JEOD_DECLARE_ATTRIBUTES (RadiationFacet)	src/radiation_surface.cc	54	1
jeod::RadiationSurface::initialize (double center_grav[3])	src/radiation_surface.cc	65	10
jeod::RadiationSurface::allocate_array (unsigned int size)	src/radiation_surface.cc	130	3
jeod::RadiationSurface::allocate_interaction_facet (Facet * facet, InteractionFacetFactory * factory, FacetParams * params, unsigned int index)	src/radiation_surface.cc	159	4
jeod::RadiationSurface::incident_radiation (double flux_mag, const double flux_struc_hat[3], bool calculate_forces)	src/radiation_surface.cc	224	2
jeod::RadiationSurface::interact_with_third_body (RadiationThirdBody * third_body_ptr, const bool calculate_forces)	src/radiation_surface.cc	238	2
jeod::RadiationSurface::accumulate_thermal_sources ()	src/radiation_surface.cc	252	2
jeod::RadiationSurface::thermal_integrator ()	src/radiation_surface.cc	263	3
jeod::RadiationSurface::equalize_absorption_emission ()	src/radiation_surface.cc	281	2
jeod::RadiationSurface::radiation_pressure ()	src/radiation_surface.cc	292	2
jeod::RadiationSurface::initialize_runtime_values ()	src/radiation_surface.cc	309	2
jeod::RadiationSurface::add_thermal_integrators_to (DynBody * dyn_body)	src/radiation_surface.cc	320	2

Continued on next page

Table 5.2: Cyclomatic Complexity (continued)

Method	File	Line	ECC
jeod::RadiationSurface::~ RadiationSurface ()	src/radiation_surface.cc	334	1
jeod::RadiationSurface Factory::RadiationSurface Factory ()	src/radiation_surface_ factory.cc	49	1
jeod::RadiationSurface Factory::add_facet_params (FacetParams * to_add)	src/radiation_surface_ factory.cc	59	2
jeod::RadiationSurface Factory::~RadiationSurface Factory ()	src/radiation_surface_ factory.cc	88	1
jeod::RadiationThirdBody:: initialize (DynManager * dyn_mgr_ptr)	src/radiation_third_body.cc	53	11
jeod::RadiationThirdBody:: calculate_shadow ()	src/radiation_third_body.cc	187	13
jeod::RadiationThirdBody:: generate_alpha (double rho_ adj, double delta)	src/radiation_third_body.cc	325	1
jeod::RadiationThirdBody:: convert_shadow_from_int (int old_shadow_geometry)	src/radiation_third_body.cc	344	1
jeod::RadiationThirdBody:: process_third_body (double real_time, RefFrame & veh_ struc_frame JEOD_UNUSE D)	src/radiation_third_body.cc	365	4
jeod::RadiationThirdBody:: test_for_state_update (double real_time)	src/radiation_third_body.cc	413	4
jeod::RadiationThirdBody:: update_third_body_state ()	src/radiation_third_body.cc	434	2

Bibliography

[1] Generated by doxygen. *Radiation Pressure Reference Manual*. National Aeronautics and Space Administration, Johnson Space Center, Software, Robotics & Simulation Division, Simulation and Graphics Branch, 2101 NASA Parkway, Houston, Texas, 77058, January 2026.

[2] Jackson, A., Thebeau, C. JSC Engineering Orbital Dynamics. Technical Report JSC-61777-docs, NASA, Johnson Space Center, Houston, Texas, January 2026.

[3] A. Milani, A.M. Nobili, and P. Farinella. *Non-Gravitational Perturbations and Satellite Geodesy*. Adam Hilger, Bristol, 1987.

[4] NASA. NASA Software Engineering Requirements. Technical Report NPR-7150.2, NASA, NASA Headquarters, Washington, D.C., September 2004.

[5] Spencer, A. Surface Model. Technical Report JSC-61777-utils/surface_model, NASA, Johnson Space Center, Houston, Texas, January 2026.

[6] Turner, G. Thermal Model. Technical Report JSC-61777-interactions/thermal_rider, NASA, Johnson Space Center, Houston, Texas, January 2026.